

Understanding and Overcoming the Challenges of Efficient Transformer Quantization

Yelysei Bondarenko, Markus Nagel, Tijmen Blankevoort

Qualcomm AI Research*

{ybond, markusn, tijmen}@qti.qualcomm.com

Abstract

Transformer-based architectures have become the de-facto standard models for a wide range of Natural Language Processing tasks. However, their memory footprint and high latency are prohibitive for efficient deployment and inference on resource-limited devices. In this work, we explore quantization for transformers. We show that transformers have unique quantization challenges – namely, high dynamic activation ranges that are difficult to represent with a low bit fixed-point format. We establish that these activations contain structured outliers in the residual connections that encourage specific attention patterns, such as attending to the special separator token. To combat these challenges, we present three solutions based on post-training quantization and quantization-aware training, each with a different set of compromises for accuracy, model size, and ease of use. In particular, we introduce a novel quantization scheme – per-embedding-group quantization. We demonstrate the effectiveness of our methods on the GLUE benchmark using BERT, establishing state-of-the-art results for post-training quantization. Finally, we show that transformer weights and embeddings can be quantized to ultra-low bit-widths, leading to significant memory savings with a minimum accuracy loss. Our source code is available at <https://github.com/qualcomm-ai-research/transformer-quantization>.

1 Introduction

Recently, transformer architectures have shown remarkable improvement in many Natural Language Processing (NLP) tasks and beyond. Based on the original Transformer (Vaswani et al., 2017), language models pre-trained from large corpora of unlabeled text, such as BERT (Devlin et al., 2019), RoBERTa (Liu et al., 2019), XLNet (Yang

et al., 2019), Transformer-XL (Dai et al., 2019), GPT family (Radford et al., 2018, 2019; Brown et al., 2020), have become an indispensable building block in modern NLP pipelines. They are also increasingly adopted in other areas, including vision (Carion et al., 2020; Dosovitskiy et al., 2020; Chen et al., 2020; Girdhar et al., 2019) and audio (Dong et al., 2018; Child et al., 2019).

Despite cutting edge results in many applications, pre-trained transformer-based models are extremely large, sometimes exceeding billions of parameters. Hence, efficient deployment of these models on resource-constrained embedded systems, and even sometimes in data centers, has become an important problem due to high latency and prohibitively large memory footprint and energy consumption.

One effective method to tackle this problem is neural network quantization. Quantization reduces memory consumption by using low-bit precision for weight and activation tensors. It also reduces inference time, and improves energy efficiency by employing low-bit fixed-point arithmetic instead of floating-point arithmetic (Horowitz, 2014).

Quantization, however, is not free. It introduces additional noise in the network that can lead to a drop in the model’s performance. While prior work has demonstrated the feasibility of integer-only inference for computer vision models (Lin et al., 2016; Jacob et al., 2018; Krishnamoorthi, 2018; Zhang et al., 2018; Choukroun et al., 2019; Dong et al., 2019; Esser et al., 2019; Nagel et al., 2019, 2020), there is relatively little work done on quantizing NLP models (Wang et al., 2018b; Xu et al., 2018), and specifically on transformer models.

Understanding the challenges of transformer quantization and designing a robust and easy-to-use quantization pipeline for them constitute the primary goal of this paper. The contributions of our work include:

*Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

Quantizing a real-valued tensor $\mathbf{x}$ is performed by first mapping it to an unsigned integer grid:

$$\mathbf{x}^{(\mathbb{Z})} = \text{clip} \left(\left\lfloor \frac{\mathbf{x}}{s} \right\rfloor + z; 0, 2^b - 1 \right), \quad (1)$$

It is possible to approximately recover the real-valued input $\mathbf{x}$ through an operation that is often referred to as *de-quantization*:

$$\hat{\mathbf{x}} := q(\mathbf{x}; s, z, b) = s \left(\mathbf{x}^{(\mathbb{Z})} - z \right) \approx \mathbf{x}. \quad (2)$$

In the case of symmetric quantization, we restrict the quantization grid to be symmetric around z .

It is common to have a single set of quantization parameters per tensor, known as *per-tensor quantization*. One could also increase the *quantization granularity* by defining separate quantizers for individual segments of a tensor. This will improve the accuracy of a network, but at the cost of an additional compute and memory overhead.

An important class of quantization methods is *post-training quantization* (PTQ) algorithms, which take a pre-trained FP32 network and convert it directly into a fixed-point network without the need for the original training pipeline (Krishnamoorthi, 2018). A vital step in the PTQ process is finding good quantization ranges for each quantizer. One way of doing this is *static range estimation*, which determines quantization parameters for the network by passing a few batches of calibration data through the model before inference. It yields more efficient inference since all the quantization parameters are known in advance and fixed. Several of the most common range estimators include:

current min-max or simply *min-max*, uses the full dynamic range of the tensor (Zhou et al., 2016; Wu et al., 2018b; Zhu et al., 2020);

running min-max uses exponential moving average of the min and max over multiple batches (Krishnamoorthi, 2018);

MSE finds quantization parameters that minimize mean squared error between quantized and floating-point tensors (Choukroun et al., 2019; Banner et al., 2018).

An alternative to PTQ is to train a neural network with the simulated quantization operations in the network, known as *quantization-aware training* (QAT, Jacob et al. 2018; Gupta et al. 2015; Krishnamoorthi 2018). It allows the model to

better adapt to the introduced quantization noise compared to PTQ, at the cost of longer training times, the need for labeled data and doing a hyper-parameter search. Gradients through the non-differentiable quantization step are usually approximated using the *straight-through* estimator (Bengio et al., 2013). Ranges for both weights and activations can be set using PTQ range estimators or learned jointly with the weights during training, as in Esser et al. (2019); Jain et al. (2019).

Finally, it is possible to assign different bit-widths for different layers or parts of the network, a technique known as *mixed precision* (Lin et al., 2016; Wu et al., 2018a; Zhou et al., 2018; Dong et al., 2019; Wang et al., 2019; van Baalen et al., 2020).

Transformer quantization Junczys-Dowmunt et al. (2018) applied knowledge distillation and 8-bit post-training quantization to speed up transformer models for neural machine translation. Bhandare et al. (2019) also applied 8-bit post-training quantization to the transformer model for machine translation and demonstrated how to utilize specialized hardware to accelerate the inference process.

Zafir et al. (2019) proposed an 8-bit quantization scheme for BERT-like models and achieves compression of up to 25% of the original model size. Shen et al. (2020) applies mixed-precision quantization on BERT, where they assign a different precision to different layers according to their sensitivity defined by Hessian information. Kim et al. (2021) proposed a fully integer-only arithmetic inference scheme based on second-order polynomial approximations for GELU, Softmax, and LayerNorm non-linearities. Some examples of transformer-based model quantization with alternative quantization schemes include Fan et al. (2020); Darvish Rouhani et al. (2020).

Note that all the mentioned approaches for BERT-like transformer quantization employ some form of QAT and either do not discuss PTQ alternatives or only use them as weak baselines.

3 Problem investigation

First, we investigate what happens when we apply standard 8-bit post-training quantization to the BERT model and evaluate it on eight downstream tasks from the GLUE benchmark (Wang et al., 2018a). To quantize fine-tuned models, we use uniform affine quantization with static range

PEG quantization with permutation can still be simulated on hardware that only supports per-tensor operations. First, we can share the same permutation for FFN’s input, output and sum since we expect the outliers in the output dominate the ones from the input. Second, we use the permutation-equivariant properties of LayerNorm and linear layers (weights are permuted accordingly before inference). We first permute the output of the first LayerNorm, proceed as described above, and then apply inverse permutation before the next LayerNorm.

Note that the PEG quantization has a negligible memory overhead, introducing only $d + 2 \cdot 3 \cdot K$ extra parameters per attention layer (permutation indices and scale & zero points per group for FFN’s input, output, and sum), which is less than 0.04% of the total size of BERT-base model.

Quantization-aware training Finally, we consider a variant of QAT with learnable ranges for both weights and activations by adapting the procedure from Esser et al. (2019); Jain et al. (2019) for BERT-like transformer models. Simulating the quantization process during fine-tuning allows the model to adapt to quantization noise and often significantly increases performance compared to post-training quantization.

Criterion	MP-PTQ	PEG-PTQ	QAT
Post-training	✓	✓	✗
Per-tensor	✓	✗	✓
Same bit-width	✗	✓	✓

Table 3: Comparison between proposed techniques (MP = mixed precision, PEG = per-embedding-group).

Comparison of methods We summarize different trade-offs for the proposed techniques in Table 3. As discussed in Section 2, usually PTQ methods are preferred over QAT algorithms since they are faster and require either no data or only a small calibration dataset. Additionally, they typically require almost no hyperparameter tuning, enabling easy and computationally efficient quantization. Allocating a higher bit-width to certain parts of the network will reduce the efficiency gain from quantization, because higher bit-width layers are more computationally expensive. It is also not supported by all target hardware. Per-embedding-group quantization has a smaller granularity compared to per-tensor quantization. It leads to a minor amount of

extra compute (and potential latency) due to the additional summation and re-quantization that occurs and might not be supported natively on every fixed-point platform. Meanwhile, we have shown a way to simulate this scheme on a hardware that only support per-tensor quantization operations.

5 Experiments

In this section, we evaluate the proposed quantization techniques for the BERT model on GLUE downstream tasks.

Experimental setup In all experiments, we use uniform affine quantization – symmetric weights, asymmetric activations – with the static activation range setting, as discussed in Section 2. We quantize all layer’s weights and activations. For 8-bit weight quantization, we use the best range settings found in the experiment from Section 3, which can be found in Appendix B.2. However, for low (<8) bit weight and token embedding quantization, we always use the MSE range estimator, as recommended by Choukroun et al. (2019); Banner et al. (2018). We set activation ranges based on min and max from a single input sequence. For PTQ experiments, we report the median score over five runs with different random seeds.

For QAT experiments, we initialize all quantization parameters from the PTQ setup described above. Similarly to full-precision fine-tuning, we use Adam (Kingma and Ba, 2014) and a maximum sequence length of 128, with padding using a special [PAD] token for shorter sequences. We use a typical learning rate schedule from the transformer literature (Devlin et al., 2019; Liu et al., 2019; Lan et al., 2019) – a linear warmup for the first 10% of training steps followed by a linear decay to zero. We perform a hyper-parameter search over the maximum learning rate, batch size, number of epochs, and the self-attention dropout rate for every task and report the best median score over three runs with different random seeds. For reproducibility, we included more details on the search space and selected hyper-parameters in Appendix B.3.

Mixed precision PTQ First, we present the results for mixed precision post-training quantization (MP-PTQ) in Table 4, where we start from 8-bit activations and progressively keep more and more operations in 16-bit precision. We see that for classification tasks (MNLI, QNLI, RTE), it is sufficient to keep a few of the most problematic parts in 16-bit

- Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. Deep learning with limited numerical precision. In *International conference on machine learning*, pages 1737–1746. PMLR.
- M. Horowitz. 2014. [1.1 computing’s energy problem \(and what we can do about it\)](#). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799. PMLR.
- Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2017. Quantized neural networks: Training neural networks with low precision weights and activations. *The Journal of Machine Learning Research*, 18(1):6869–6898.
- Forrest Iandola, Albert Shaw, Ravi Krishna, and Kurt Keutzer. 2020. [SqueezeBERT: What can computer vision teach NLP about efficient neural networks?](#) In *Proceedings of SustainNLP: Workshop on Simple and Efficient Natural Language Processing*, pages 124–135, Online. Association for Computational Linguistics.
- Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713.
- Sambhav R Jain, Albert Gural, Michael Wu, and Chris Dick. 2019. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *arXiv preprint arXiv:1903.08066*, 6.
- Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. [TinyBERT: Distilling BERT for natural language understanding](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174, Online. Association for Computational Linguistics.
- Marcin Junczys-Dowmunt, Kenneth Heafield, Hieu Hoang, Roman Grundkiewicz, and Anthony Aue. 2018. Marian: Cost-effective high-quality neural machine translation in c++. *arXiv preprint arXiv:1805.12096*.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR.
- Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. 2021. I-bert: Integer-only bert quantization. *arXiv preprint arXiv:2101.01321*.
- Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*.
- Raghuraman Krishnamoorthi. 2018. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*.
- Hector Levesque, Ernest Davis, and Leora Morgenstern. 2012. The winograd schema challenge. In *Thirteenth International Conference on the Principles of Knowledge Representation and Reasoning*. Citeseer.
- Darryl Lin, Sachin Talathi, and Sreekanth Annappureddy. 2016. Fixed point quantization of deep convolutional networks. In *International conference on machine learning*, pages 2849–2858. PMLR.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. 2020. Up or down? adaptive rounding for post-training quantization. In *International Conference on Machine Learning*, pages 7197–7206. PMLR.
- Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. 2019. Data-free quantization through weight equalization and bias correction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1325–1334.
- Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Blankevoort Tijmen. 2021. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*.
- Sai Prasanna, Anna Rogers, and Anna Rumshisky. 2020. When bert plays the lottery, all tickets are winning. *arXiv preprint arXiv:2005.00561*.
- Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training.

Up or Down? Adaptive Rounding for Post-Training Quantization

Markus Nagel^{*1} Rana Ali Amjad^{*1} Mart van Baalen¹ Christos Louizos¹ Tijmen Blankevoort¹

Abstract

When quantizing neural networks, assigning each floating-point weight to its nearest fixed-point value is the predominant approach. We find that, perhaps surprisingly, this is not the best we can do. In this paper, we propose AdaRound, a better weight-rounding mechanism for post-training quantization that adapts to the data and the task loss. AdaRound is fast, does not require fine-tuning of the network, and only uses a small amount of unlabelled data. We start by theoretically analyzing the rounding problem for a pre-trained neural network. By approximating the task loss with a Taylor series expansion, the rounding task is posed as a quadratic unconstrained binary optimization problem. We simplify this to a layer-wise local loss and propose to optimize this loss with a soft relaxation. AdaRound not only outperforms rounding-to-nearest by a significant margin but also establishes a new state-of-the-art for post-training quantization on several networks and tasks. Without fine-tuning, we can quantize the weights of Resnet18 and Resnet50 to 4 bits while staying within an accuracy loss of 1%.

1. Introduction

Deep neural networks are being used in many real-world applications as the standard technique for solving tasks in computer vision, machine translation, voice recognition, ranking, and many other domains. Owing to this success and widespread applicability, making these neural networks efficient has become an important research topic. Improved efficiency translates into reduced cloud-infrastructure costs and makes it possible to run these networks on heterogeneous devices such as smartphones, internet-of-things appli-

cations, and even dedicated low-power hardware.

One effective way to optimize neural networks for inference is neural network quantization (Krishnamoorthi, 2018; Guo, 2018). In quantization, neural network weights and activations are kept in a low-bit representation for both memory transfer and calculations in order to reduce power consumption and inference time. The process of quantizing a network generally introduces noise, which results in a loss of performance. Various prior works adapt the quantization procedure to minimize the loss in performance while going as low as possible in the number of bits used.

As Nagel et al. (2019) explained, the practicality of neural network quantization methods is important to take into consideration. Although many methods exist that do quantization-aware training (Jacob et al., 2018; Louizos et al., 2019) and get excellent results, these methods require a user to spend significant time on re-training models and hyperparameter tuning.

On the other hand, much attention has recently been dedicated to *post-training quantization* methods (Nagel et al., 2019; Cai et al., 2020; Choukroun et al., 2019; Banner et al., 2019), which can be more easily applied in practice. These types of methods allow for network quantization to happen on-the-fly when deploying models, without the user of the model spending time and energy on quantization. Our work focuses on this type of network quantization.

Rounding-to-nearest is the predominant approach for all neural network weight quantization work that came out thus far. This means that the weight vector $\mathbf{w}$ is rounded to the nearest representable quantization grid value in a fixed-point grid by

$$\hat{\mathbf{w}} = s \cdot \text{clip} \left(\left\lfloor \frac{\mathbf{w}}{s} \right\rfloor, n, p \right), \quad (1)$$

where s denotes the quantization scale parameter and, n and p denote the negative and positive integer thresholds for clipping. We could round any weight down by replacing $\lfloor \cdot \rfloor$ with $\lfloor \cdot \rfloor$, or up using $\lceil \cdot \rceil$. But, rounding-to-nearest seems the most sensible, as it minimizes the difference per-weight in the weight matrix. Perhaps surprisingly, we show that for post-training quantization, rounding-to-nearest is not optimal.

Our contributions in this work are threefold:

^{*}Equal contribution ¹Qualcomm AI Research, an initiative of Qualcomm Technologies, Inc.. Correspondence to: Markus Nagel <markusn@qti.qualcomm.com>, Rana Ali Amjad <ramjad@qti.qualcomm.com>, Tijmen Blankevoort <tijmen@qti.qualcomm.com>.

Proceedings of the 37th International Conference on Machine Learning, Online, PMLR 119, 2020. Copyright 2020 by the author(s).

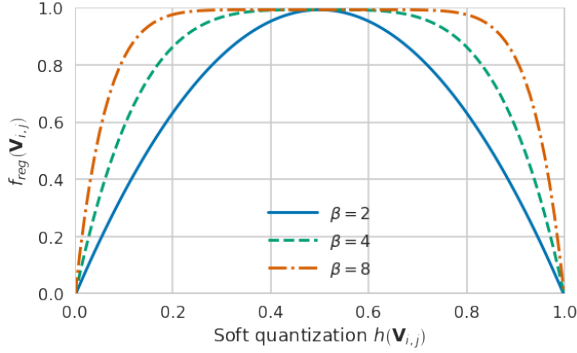


Figure 2. Effect of annealing β on regularization term (24).

encourage $h(\mathbf{V}_{i,j})$ to move to the extremities. For regularization we use

$$f_{reg}(\mathbf{V}) = \sum_{i,j} 1 - |2h(\mathbf{V}_{i,j}) - 1|^\beta, \quad (24)$$

where we anneal the parameter β . This allows most of the $h(\mathbf{V}_{i,j})$ to adapt freely in the initial phase (higher β) to improve the MSE and encourages it to converge to 0 or 1 in the later phase of the optimization (lower β), to arrive at the binary solution that we are interested in. The effect of annealing β is illustrated in Fig. 2. Fig. 3 shows how this combination of rectified sigmoid and f_{reg} leads to many weights learning a rounding that is different from rounding to the nearest, to improve the performance, while ultimately converging close to 0 or 1.

This method of optimizing (21) is a specific instance of the general family of Hopfield methods used for binary constrained optimization problems. These types of methods are commonly used as an efficient approximation algorithm for large scale combinatorial problems (Hopfield & Tank, 1985; Smith et al.).

To quantize the whole model, we optimize (21) layer-by-layer sequentially. However, this does not account for the quantization error introduced due to the previous layers. In order to avoid the accumulation of quantization error for deeper networks as well as to account for the activation function, we use the following asymmetric reconstruction formulation

$$\arg \min_{\mathbf{V}} \left\| f_a(\mathbf{W}\mathbf{x}) - f_a(\widetilde{\mathbf{W}}\hat{\mathbf{x}}) \right\|_F^2 + \lambda f_{reg}(\mathbf{V}), \quad (25)$$

where $\hat{\mathbf{x}}$ is the layer’s input with all preceding layers quantized and f_a is the activation function. A similar formulation of the loss has been used previously in (Zhang et al., 2016; He et al., 2017), albeit for different purposes. (25) defines our final objective that we can optimize via stochastic gradient descent. We call this algorithm AdaRound, as it adapts to the statistics of the input data as well as to (an approximation of) the task loss. In section 5 we elaborate on the influence of our design choices as well as the asymmetric reconstruction loss on the performance.

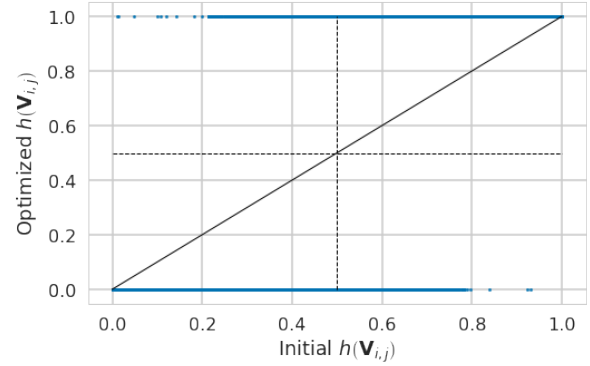


Figure 3. Comparison of $h(\mathbf{V}_{i,j})$ before (x-axis, corresponding to floating point weights) vs after (y-axis) optimizing (21). We see that all $h(\mathbf{V}_{i,j})$ have converged to 0 or 1. Top left and lower right quadrants indicate the weights that have different rounding using (21) vs rounding-to-nearest.

4. Background and related work

In the 1990s, with the resurgence of the field of neural networks, several works designed hardware and optimization methods for running low-bit neural networks on-device. Hammerstrom (1990) created hardware for 8 and 16-bit training of networks, Holi & Hwang (1993) did an empirical analysis on simple neural networks to show that 8 bits are sufficient in most scenarios, and Hoehfeld & Fahlman (1992) developed a stochastic rounding scheme to push neural networks below 8 bits.

More recently, much attention has gone to quantizing neural networks for efficient inference. This is often done by simulating quantization during training, as described in Jacob et al. (2018) and Gupta et al. (2015), and using a straight-through estimator to approximate the gradients. Many methods have since then extended these training frameworks. Choi et al. (2018) learns the activations to obey a certain quantization range, while Esser et al. (2020); Jain et al. (2019) learn the quantization min and max ranges during training so that they do not have to be set manually. Louizos et al. (2019) also learn the grid and formulate a probabilistic version of the quantization training procedure. Uhlich et al. (2020) learn both the quantization grid, and the bit-width per layer, resulting in automatic bit-width selection during training. Works like Kim et al. (2019); Mishra & Marr (2017) exploit student-teacher training to improve quantized model performance during training. Although quantization-aware training is potent and often gives good results, the process is often tedious and time-consuming. Our work seeks to get high accuracy models without this hassle.

Several easy-to-use methods for quantization of networks without quantization-aware training have been proposed as of recent. These methods are often referred to as *post-training quantization* methods. Krishnamoorthi (2018) show several results of network quantization without fine-

- Hopfield, J. J. and Tank, D. W. “neural” computation of decisions in optimization problems. *Biological Cybernetics*, 52(3):141–152, 1985.
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- Jain, S. R., Gural, A., Wu, M., and Dick, C. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *arxiv preprint arxiv:1903.08066*, 2019.
- Kim, J., Bhalgat, Y., Lee, J., Patel, C., and Kwak, N. QKD: quantization-aware knowledge distillation. *arxiv preprint arxiv:1911.12491*, 2019.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *International Conference for Learning Representations (ICLR)*, 2015.
- Kochenberger, G., Hao, J.-K., Glover, F., Lewis, M., Lü, Z., Wang, H., and Wang, Y. The unconstrained binary quadratic programming problem: a survey. *Journal of Combinatorial Optimization*, 28(1):58–81, Jul 2014.
- Krishnamoorthi, R. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.
- Lin, D. D., Talathi, S. S., and Annapureddy, V. S. Fixed point quantization of deep convolutional networks. In *International Conference on Machine Learning*, 2016.
- Louizos, C., Welling, M., and Kingma, D. P. Learning sparse neural networks through l_0 regularization. *International Conference on Learning Representations (ICLR)*, 2018.
- Louizos, C., Reisser, M., Blankevoort, T., Gavves, E., and Welling, M. Relaxed quantization for discretized neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.
- Mishra, A. K. and Marr, D. Apprentice: Using knowledge distillation techniques to improve low-precision network accuracy. *arXiv preprint arxiv:1711.05852*, 2017.
- Nagel, M., van Baalen, M., Blankevoort, T., and Welling, M. Data-free quantization through weight equalization and bias correction. *International Conference on Computer Vision (ICCV)*, 2019.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. In *Neural Information Processing Systems (NeurIPS)*, 2019.
- Rubinstein, R. The cross-entropy method for combinatorial and continuous optimization. *Methodology And Computing In Applied Probability*, 1(2):127–190, Sep 1999.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015.
- Smith, K. A., Palaniswami, M., and Krishnamoorthy, M. Neural techniques for combinatorial optimization with applications. *IEEE Trans. Neural Networks*, 9(6):1301–1318.
- Stock, P., Joulin, A., Gribonval, R., Graham, B., and Jégou, H. And the bit goes down: Revisiting the quantization of neural networks. In *International Conference on Learning Representations*, 2020.
- Uhlich, S., Mauch, L., Yoshiyama, K., Cardinaux, F., García, J. A., Tiedemann, S., Kemp, T., and Nakamura, A. Mixed precision dnns: All you need is a good parametrization. *International Conference on Learning Representations (ICLR)*, 2020.
- Wang, P., Hu, Q., Zhang, Y., Zhang, C., Liu, Y., and Cheng, J. Two-step quantization for low-bit neural networks. *Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4376–4384, 2018.
- Zhang, X., Zou, J., He, K., and Sun, J. Accelerating very deep convolutional networks for classification and detection. *IEEE Trans. Pattern Anal. Mach. Intell.*, 38(10): 1943–1955, 2016.
- Zhao, R., Hu, Y., Dotzel, J., Sa, C. D., and Zhang, Z. Improving neural network quantization without retraining using outlier channel splitting. *International Conference on Machine Learning, ICML*, 2019.

A White Paper on Neural Network Quantization

Markus Nagel*
Qualcomm AI Research[†]
markusn@qti.qualcomm.com

Marios Fournarakis*
Qualcomm AI Research[†]
mfournar@qti.qualcomm.com

Rana Ali Amjad
Qualcomm AI Research[†]
ramjad@qti.qualcomm.com

Yelysei Bondarenko
Qualcomm AI Research[†]
ybodaren@qti.qualcomm.com

Mart van Baalen
Qualcomm AI Research[†]
mart@qti.qualcomm.com

Tijmen Blankevoort
Qualcomm AI Research[†]
tijmen@qti.qualcomm.com

Abstract

While neural networks have advanced the frontiers in many applications, they often come at a high computational cost. Reducing the power and latency of neural network inference is key if we want to integrate modern networks into edge devices with strict power and compute requirements. Neural network quantization is one of the most effective ways of achieving these savings but the additional noise it induces can lead to accuracy degradation.

In this white paper, we introduce state-of-the-art algorithms for mitigating the impact of quantization noise on the network's performance while maintaining low-bit weights and activations. We start with a hardware motivated introduction to quantization and then consider two main classes of algorithms: Post-Training Quantization (PTQ) and Quantization-Aware-Training (QAT). PTQ requires no re-training or labelled data and is thus a lightweight push-button approach to quantization. In most cases, PTQ is sufficient for achieving 8-bit quantization with close to floating-point accuracy. QAT requires fine-tuning and access to labeled training data but enables lower bit quantization with competitive results. For both solutions, we provide tested pipelines based on existing literature and extensive experimentation that lead to state-of-the-art performance for common deep learning models and tasks.

1 Introduction

With the rise in popularity of deep learning as a general-purpose tool to inject intelligence into electronic devices, the necessity for small, low-latency and energy efficient neural networks solutions has increased. Today neural networks can be found in many electronic devices and services, from smartphones, smart glasses and home appliances, to drones, robots and self-driving cars. These devices are typically subject to strict time restrictions on the execution of neural networks or stringent power requirements for long-duration performance.

One of the most impactful ways to decrease the computational time and energy consumption of neural networks is quantization. In neural network quantization, the weights and activation tensors are stored in lower bit precision than the 16 or 32-bit precision they are usually trained in. When

*Equal contribution.

[†]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

$\mathbf{x}_i$, is given by:

$$\begin{aligned}
\frac{\partial \hat{\mathbf{x}}_i}{\partial \mathbf{x}_i} &= \frac{\partial q(\mathbf{x}_i)}{\partial \mathbf{x}_i} \\
&= s \cdot \frac{\partial}{\partial \mathbf{x}_i} \text{clamp} \left(\left\lfloor \frac{\mathbf{x}_i}{s} \right\rfloor; n, p \right) + 0 \\
&= \begin{cases} s \cdot \frac{\partial \lfloor \mathbf{x}_i/s \rfloor}{\partial (\mathbf{x}_i/s)} \frac{\partial (\mathbf{x}_i/s)}{\partial \mathbf{x}_i} & \text{if } q_{\min} \leq \mathbf{x}_i \leq q_{\max}, \\ s \cdot \frac{\partial n}{\partial \mathbf{x}_i} & \text{if } \mathbf{x}_i < q_{\min}, \\ s \cdot \frac{\partial p}{\partial \mathbf{x}_i} & \text{if } \mathbf{x}_i > q_{\max}, \end{cases} \\
&= \begin{cases} 1 & \text{if } q_{\min} \leq \mathbf{x}_i \leq q_{\max}, \\ 0 & \text{otherwise.} \end{cases} \tag{37}
\end{aligned}$$

Using this gradient definition we can now back-propagate through the quantization blocks. Figure 10 shows a simple computational graph for the forward and backward pass used in quantization-aware training. The forward pass is identical to that of figure 4, but in the backward pass we effectively skip the quantizer block due to the STE assumption. In earlier QAT work the quantization ranges

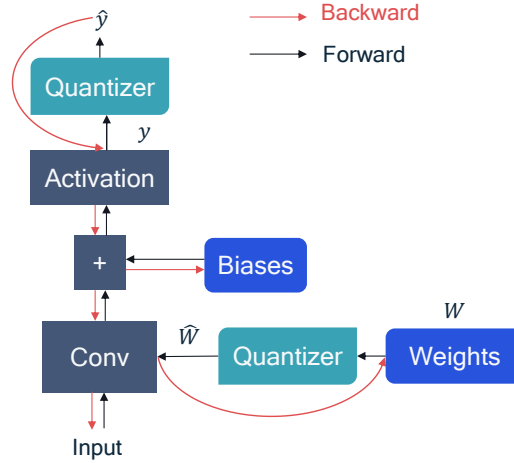


Figure 10: Forward and backward computation graph for quantization aware training with STE assumption.

for weights and activations were updated at each iteration most commonly using the min-max range (Krishnamoorthi, 2018). In later work (Essex et al., 2020; Jain et al., 2019; Bhalgat et al., 2020), the STE is used to calculate the gradient *w.r.t.* the quantization parameters, z and s . Using the chain rule and the STE, we first calculate the gradient *w.r.t.* the scale-factor:

$$\begin{aligned}
\frac{\partial \hat{\mathbf{x}}_i}{\partial s} &= \frac{\partial}{\partial s} \left[s \cdot \text{clamp} \left(\left\lfloor \frac{\mathbf{x}_i}{s} \right\rfloor; n, p \right) \right] \\
&= \begin{cases} -\mathbf{x}_i/s + \lfloor \mathbf{x}_i/s \rfloor & \text{if } q_{\min} \leq \mathbf{x}_i \leq q_{\max}, \\ n & \text{if } \mathbf{x}_i < q_{\min}, \\ p & \text{if } \mathbf{x}_i > q_{\max}. \end{cases} \tag{38}
\end{aligned}$$

Originally, we restricted the zero-point to be an integer. To make zero-point learnable we convert into a real number and apply the rounding operator. The modified quantization function is defined as:

$$\hat{\mathbf{x}} = q(\mathbf{x}; s, z) = s \cdot \left[\text{clamp} \left(\left\lfloor \frac{\mathbf{x}}{s} \right\rfloor + \lfloor z \rfloor; n, p \right) - \lfloor z \rfloor \right] \tag{39}$$

The gradient *w.r.t.* to z is calculated by applying the STE once again to the rounding operator:

$$\frac{\partial \hat{\mathbf{x}}_i}{\partial z} = \begin{cases} 0 & q_{\min} \leq \mathbf{x}_i \leq q_{\max}, \\ -s & \text{otherwise.} \end{cases} \tag{40}$$

Model (FP32 accuracy)	ResNet18 (69.68)		MobileNetV2 (71.72)	
	PTQ	QAT	PTQ	QAT
W4A8 w/ min-max weight init	0.12	69.61	0.56	69.96
W4A8 w/ MSE weight init	18.58	69.76	12.99	70.13
W4A4 w/ min-max act init	7.51	68.23	0.22	66.55
W4A4 w/ MSE act init	9.62	68.41	0.71	66.29

Table 8: Ablation study for various ways to initialize the quantization grid. The learning rate is individually optimized for each configuration. ImageNet validation accuracy (%) averaged over 3 runs.

this modeling approach is on par or better for per-channel quantization compared to static folding as we can see from the last two rows of table 7.

4.3 Initialization for QAT

In this section, we will explore the effect of initialization for QAT. It is common practice in literature to start from a pre-trained FP32 model (Esser et al., 2020; Krishnamoorthi, 2018; Jacob et al., 2018; Jain et al., 2019). While it is clear that starting from an FP32 model is beneficial, the effect of the quantization initialization on the final QAT result is less studied. Here we explore the effect of using several of our PTQ techniques as an initial step before doing QAT.

Effect of range estimation To assess the effect of the initial range setting (see section 3.1) for weights and activations, we perform two sets of experiments, which are summarized in table 8. In the first experiment, we quantize the weights to 4-bits and keep the activations in 8-bits. We compare the min-max initialization with the MSE based initialization for the weights quantization range. While the MSE initialized model has a significantly higher starting accuracy, the gap closes after training for 20 epochs.

To explore the same effect for activation quantization, we perform a similar experiment, where we now quantize the activation to 4-bits and compare min-max initialization with MSE based initialization. The observations from weight range initialization hold here as well. In figure 11 we show the full training curve of this experiment. In the first few epochs, there is a significant advantage for using MSE initialization, which almost vanishes in the later stage of training. In conclusion, a better initialization can lead to better QAT results, but the gain is usually small and vanishes the longer the training lasts.

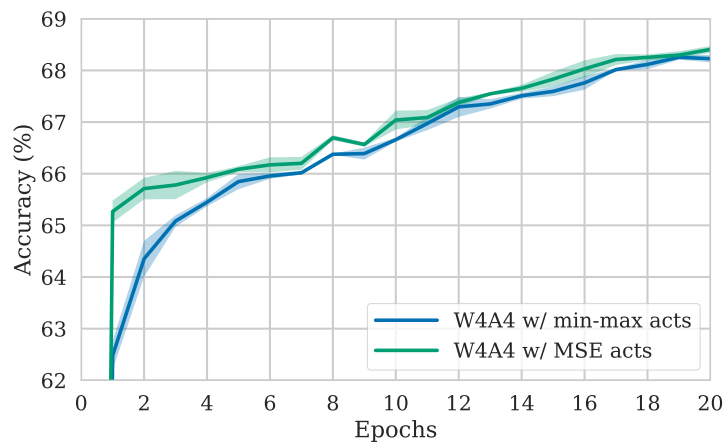


Figure 11: Influence of the initial activation range setting on the QAT training behavior of ResNet18. Average ImageNet validation accuracy (%) after each training epoch over 3 runs (and standard deviation shaded).

- Esser, S. K., McKinstry, J. L., Bablani, D., Appuswamy, R., and Modha, D. S. Learned step size quantization. *International Conference on Learning Representations (ICLR)*, 2020. 20, 22
- Finkelstein, A., Almog, U., and Grobman, M. Fighting quantization bias with bias. *arXiv preprint arxiv:1906.03193*, 2019. 13
- Gupta, S., Agrawal, A., Gopalakrishnan, K., and Narayanan, P. Deep learning with limited numerical precision. *International Conference on Machine Learning, ICML*, 2015. 14
- Horowitz, M. 1.1 computing’s energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pp. 10–14, 2014. doi: 10.1109/ISSCC.2014.6757323. 2, 3
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017. 10
- Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Bach, F. and Blei, D. (eds.), *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 448–456, Lille, France, 07–09 Jul 2015. PMLR. URL <http://proceedings.mlr.press/v37/ioffe15.html>. 6
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 3, 6, 21, 22
- Jain, S. R., Gural, A., Wu, M., and Dick, C. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *CoRR*, abs/1903.08066, 2019. URL <http://arxiv.org/abs/1903.08066>. 20, 22
- Krishnamoorthi, R. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018. 6, 10, 12, 20, 21, 22
- Meller, E., Finkelstein, A., Almog, U., and Grobman, M. Same, same but different: Recovering neural network quantization error through weight factorization. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, pp. 4486–4495, 2019. URL <http://proceedings.mlr.press/v97/meller19a.html>. 11, 12, 13
- Moons, B., Noorzad, P., Skliar, A., Mariani, G., Mehta, D., Lott, C., and Blankevoort, T. Distilling optimal neural networks: Rapid search in diverse spaces. *arXiv preprint arXiv:2012.08859*, 2020. 15
- Nagel, M., van Baalen, M., Blankevoort, T., and Welling, M. Data-free quantization through weight equalization and bias correction. *International Conference on Computer Vision (ICCV)*, 2019. 10, 11, 12, 13
- Nagel, M., Amjad, R. A., Van Baalen, M., Louizos, C., and Blankevoort, T. Up or down? Adaptive rounding for post-training quantization. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 7197–7206. PMLR, 13–18 Jul 2020. URL <http://proceedings.mlr.press/v119/nagel20a.html>. 13, 14, 15
- Nascimento, M. G. d., Fawcett, R., and Prisacariu, V. A. Dsconv: Efficient convolution operator. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019. 5
- Ramachandran, P., Zoph, B., and Le, Q. V. Searching for activation functions. *CoRR*, abs/1710.05941, 2017. URL <http://arxiv.org/abs/1710.05941>. 7
- Rouhani, B., Lo, D., Zhao, R., Liu, M., Fowers, J., Ovtcharov, K., Vinogradsky, A., Massengill, S., Yang, L., Bittner, R., Forin, A., Zhu, H., Na, T., Patel, P., Che, S., Koppaka, L. C., Song, X., Som, S., Das, K., Tiwary, S., Reinhardt, S., Lanka, S., Chung, E., and Burger, D. Pushing the limits of narrow precision inferencing at cloud scale with microsoft floating point. In *Neural Information Processing Systems (NeurIPS 2020)*. ACM, November 2020. 5

MIXED PRECISION DNNs: ALL YOU NEED IS A GOOD PARAMETRIZATION

Stefan Uhlich*, Lukas Mauch*, Fabien Cardinaux*, Kazuki Yoshiyama
Javier Alonso García, Stephen Tiedemann, Thomas Kemp
 Sony Europe B.V., Germany
 firstname.lastname@sony.com

Akira Nakamura
 Sony Corporate, Japan
 akira.b.nakamura@sony.com

ABSTRACT

Efficient deep neural network (DNN) inference on mobile or embedded devices typically involves quantization of the network parameters and activations. In particular, mixed precision networks achieve better performance than networks with homogeneous bitwidth for the same size constraint. Since choosing the optimal bitwidths is not straight forward, training methods, which can learn them, are desirable. Differentiable quantization with straight-through gradients allows to learn the quantizer’s parameters using gradient methods. We show that a suited parametrization of the quantizer is the key to achieve a stable training and a good final performance. Specifically, we propose to parametrize the quantizer with the step size and dynamic range. The bitwidth can then be inferred from them. Other parametrizations, which explicitly use the bitwidth, consistently perform worse. We confirm our findings with experiments on CIFAR-10 and ImageNet and we obtain mixed precision DNNs with learned quantization parameters, achieving state-of-the-art performance.

1 INTRODUCTION

Quantized DNNs apply quantizers $Q : \mathbb{R} \rightarrow \{q_1, \dots, q_I\}$ to discretize the weights and/or activations of a DNN (Han et al., 2015; Zhou et al., 2017; Li et al., 2016; Liu & Mattina, 2019; Cardinaux et al., 2018; Jain et al., 2019; Bai et al., 2018). They require considerably less memory and have a lower computational complexity, since discretized values $\{q_1, \dots, q_I\}$ can be stored, multiplied and accumulated efficiently. This is particularly relevant for inference on mobile or embedded devices with limited computational power.

However, gradient based training of quantized DNNs is difficult, as the gradient of a quantization function vanishes almost everywhere, i.e., backpropagation through a quantized DNN almost always returns a zero gradient. Different solutions to this problem have been proposed in the literature: A first possibility is to use DNNs with stochastic weights from a categorical distribution and to optimize the *evidence lower bound* (ELBO) to obtain an estimate of the posterior distribution of the weights. As proposed in (Jang et al., 2016; Maddison et al., 2016; Louizos et al., 2019), the categorical distribution can be relaxed to a concrete distribution – a smoothed approximation of the categorical distribution – such that the ELBO becomes differentiable under reparametrization. A second possibility is to use the *straight through estimator* (STE) (Bengio et al., 2013). STE allows the gradients to be backpropagated through the quantizers and, thus, the network weights can be adapted with standard gradient descent (Hubara et al., 2016). Compared to STE based methods, stochastic methods suffer from large gradient variance, which makes training of large quantized DNNs difficult. Therefore, STE based methods are more popular in practice.

More recent research (Jain et al., 2019; Esser et al., 2019; Wang et al., 2018; Elthakeb et al., 2018) focuses on methods which can also learn the optimal quantization parameters, e.g., the stepsize, dynamic range and bitwidth, in parallel to the network weights. This is a promising approach as DNNs with learned quantization parameters almost always outperform DNNs with handcrafted ones.

*Equal contribution.

The source code can be found at <https://github.com/sony/ai-research-code>.

Recently, and in parallel to our work, (Jain et al., 2019) explored the use of STE to define the gradient with respect to the quantizers’s dynamic range. The authors applied a per-tensor quantization and used the dynamic range as an additional trainable parameter also learned with gradient descent. Similarly, (Esser et al., 2019) learned the stepsize using gradient descent. However, neither of them learned the optimal bitwidth of the quantizers.

One approach was proposed in (Wang et al., 2018; Elthakeb et al., 2018). They learn the bitwidth with reinforcement learning, i.e., they learn an optimal bitwidth assignment policy. Their experiments show that a DNN with a learned and heterogeneous bitwidth assignment outperforms quantized DNNs with a homogeneous bitwidth assignment. However, such methods have a high computational complexity as the bitwidth policy must be learned, which involves training many quantized DNNs.

In this paper, we will use the STE approach and show that the quantizer’s parameters, including the bitwidth, can be learned with gradient methods if a good parametrization is chosen. Specifically, we show that directly learning the bitwidth is not optimal. Instead, we propose to learn the stepsize and dynamic range. The bitwidth can then be inferred from them. Compared to (Wang et al., 2018; Elthakeb et al., 2018), our method has the advantage that training quantized DNNs has nearly the same computational complexity as standard float32 training.

The contributions of this paper are:

1. We show that there are three different parametrizations for *uniform* and *power-of-two* quantization and that, in both cases, one of them has gradients particularly well suited to train quantized DNNs. The other parametrizations have the problem of yielding gradients with an unbounded gradient norm and coupled components.
2. Using this parametrization, we are able to learn all quantization parameters for DNNs with per-tensor quantization and global memory constraints. We formulate the training as a constrained optimization problem, where the quantized DNN is constrained not to exceed a given overall memory budget, and show how to solve it in a penalty framework.
3. We confirm our findings with experiments on CIFAR-10 and ImageNet. For example, we train a heterogeneously quantized MobileNetV2 on ImageNet requiring a total of only 1.65MB to store the weights and only 0.57MB to store its largest feature map. This is equivalent to a homogenous 4bit quantization of both weights and activations. However, our network learns to allocate the bitwidth heterogeneously in an optimal way. Our MobileNetV2 achieves an error of 30.26% compared to 29.82% for the floating point baseline. This is state-of-the-art for such a heavily quantized MobileNetV2.

We use the following notation throughout this paper: x , $\mathbf{x}$, $\mathbf{X}$ and $\mathcal{X}$ denote a scalar, a (column) vector, a matrix and a tensor with three or four dimensions, respectively; $\lfloor \cdot \rfloor$ and $\lceil \cdot \rceil$ are the floor and ceiling operators. Finally, $\delta(\cdot)$ denotes the Dirac delta function.

2 CHOOSING A QUANTIZATION PARAMETRIZATION

Let $Q(x; \theta)$ be a quantizer with the parameters θ , which maps $x \in \mathbb{R}$ to discrete values $\{q_1, \dots, q_I\}$. In this section, we compare different parametrizations of $Q(x; \theta)$ for *uniform quantization* and *power-of-two quantization* and analyze how well the corresponding straight-through gradient estimates $\partial_x Q(x; \theta)$ and $\nabla_\theta Q(x; \theta)$ are suited to optimize the quantizer parameters θ . Our key result is, that the training of quantized DNNs which learns both, the optimal quantized weights and the optimal quantization parameters θ , is very sensitive to the choice of the parametrization of the quantizers. From an optimization point of view, it is best to parametrize the quantizer $Q(x; \theta)$ with the stepsize d and the dynamic range $q_{\max}$ as it leads to gradients with stable norms. Doing so, we can use standard gradient descent to learn the quantization parameters and do not need to use stochastic or reinforcement based algorithms, which are computationally expensive.

2.1 PARAMETRIZATION AND STRAIGHT THROUGH GRADIENT ESTIMATES

A symmetric uniform quantizer $Q_U(x; \theta)$ which maps a real value $x \in \mathbb{R}$ to one of $I = 2k + 1$ quantized values $q \in \{-kd, \dots, 0, \dots, kd\}$ computes

$$q = Q_U(x; \theta) = \text{sign}(x) \begin{cases} d \left\lfloor \frac{|x|}{d} + \frac{1}{2} \right\rfloor & |x| \leq q_{\max} \\ q_{\max} & |x| > q_{\max} \end{cases}, \quad (1)$$

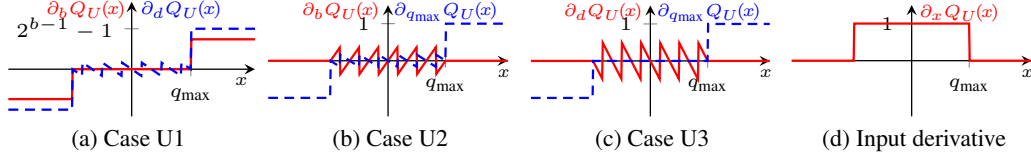


Figure 2: Partial derivatives of $Q_U(x; \theta)$ with respect to the input and the quantization parameters d , $q_{\max}$ and b . Partial derivatives are coupled for “U1” and “U2” but are decoupled for “U3”.

avoid divergence. However, parametrization “U3” does not suffer from such an unbounded gradient norm as both partial derivatives $\partial_d Q_U(x; \theta) \in [-\frac{1}{2}, \frac{1}{2}]$ and $\partial_{q_{\max}} Q_U(x; \theta) \in \{-1, 1\}$ are bounded.

Fig. 2 shows the gradients for the parametrization “U1” to “U3”. For parametrization “U3”, the partial derivatives in $\nabla_{\theta} Q_U(x; \theta)$ are *decoupled*, i.e., $\nabla_{\theta} Q_U(x; \theta)$ is a unit vector, which either points only in the direction of d if $|x| \leq q_{\max}$ or only in the direction of $q_{\max}$, if $|x| > q_{\max}$. We will show in Sec. 2.3 that this implies a diagonal Hessian, which results in a better convergence behavior of gradient descent. In contrast, both parametrizations “U1” and “U2” have partial derivatives that are coupled. In summary, this implies that parametrization “U3” is the best DQ parametrization.

Similar considerations can be made for the power-of-two quantization $Q_P(x; \theta)$, which maps a real-valued number $x \in \mathbb{R}$ to a quantized value $q \in \{\pm 2^k : k \in \mathbb{Z}\}$ by

$$q = Q_P(x; \theta) = \text{sign}(x) \begin{cases} q_{\min} & |x| \leq q_{\min} \\ 2^{\lfloor 0.5 + \log_2 |x| \rfloor} & q_{\min} < |x| \leq q_{\max} \\ q_{\max} & |x| > q_{\max} \end{cases}, \quad (4)$$

where $q_{\min}$ and $q_{\max}$ are the minimum and maximum absolute values of the quantizer for a bitwidth of b bit. Power-of-two quantization is an especially interesting scheme for DNN quantization, since a multiplication of quantized values can be implemented as an addition of the exponents. Using the STE for the floor operation, the derivative $\partial_x Q_P(x; \theta)$ is given by

$$\partial_x Q_P(x) = \begin{cases} 0 & |x| \leq q_{\min} \\ \frac{2^{\lfloor 0.5 + \log_2 |x| \rfloor}}{|x|} & q_{\min} < |x| \leq q_{\max} \\ 0 & |x| > q_{\max} \end{cases}. \quad (5)$$

The power-of-two quantization has the three parameters $[b, q_{\min}, q_{\max}] =: \theta$, which depend on each other with the relationship $q_{\max} = 2^{b-1} q_{\min}$. Therefore, we have again three different parametrizations with $\theta = [b, q_{\min}]$, $\theta = [b, q_{\max}]$ or $\theta = [q_{\min}, q_{\max}]$, respectively. Similarly to the uniform case, one parametrization ($\theta = [q_{\min}, q_{\max}]$) leads to a gradient of a very simple form

$$\nabla_{\theta} Q_P(x; \theta) = \begin{bmatrix} \partial_{q_{\min}} Q_U(x; \theta) \\ \partial_{q_{\max}} Q_U(x; \theta) \end{bmatrix} = \begin{cases} [1, 0]^T & |x| \leq q_{\min} \\ [0, 0]^T & q_{\min} < |x| \leq q_{\max} \\ [0, 1]^T & |x| > q_{\max} \end{cases}, \quad (6)$$

which has again a bounded gradient magnitude and independent components and is, hence, best suited for first order gradient based optimization.

2.2 CONSTRAINTS ON θ

In practice, for an efficient hardware implementation, we need to ensure that the quantization parameters only take specific discrete values: for uniform quantization, only integer values are allowed for the bitwidth b , and the stepsize d must be a power-of-two, see e.g. (Jain et al., 2019); for power-of-two quantization, the bitwidth must be an integer, and the minimum and maximum absolute values $q_{\min}$ and $q_{\max}$ must be powers-of-two.

We fulfill these constraints by rounding the parameters in the forward pass to the closest integer or power-of-two value. In the backward pass we update the original float values, i.e., we used again the STE to propagate the gradients.

2.3 EXPERIMENTAL COMPARISON OF DQ PARAMETRIZATIONS

In the following we compare the parametrizations using two experiments.

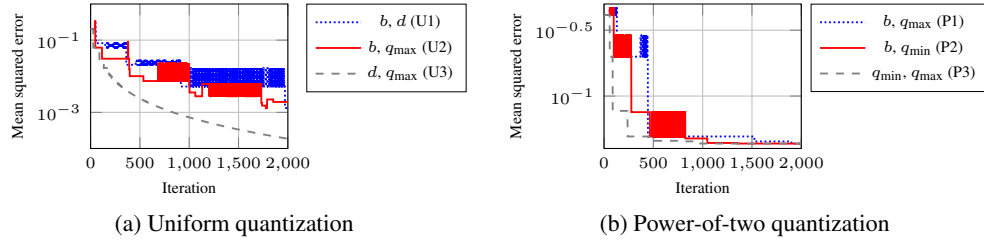


Figure 3: MSE for quantizing Gaussian data $x \sim N(0, 1)$ with uniform and power-of-two quantization. Parametrizations “U3” and “P3” converge to the lowest MSE without any oscillations.

1) *Quantization of Gaussian data* In our first experiment we use DQ to learn the optimal quantization parameters θ^* which minimize the *mean squared error* (MSE) $E[\frac{1}{2}(Q(x; \theta) - x)^2]$ with gradient descent and compare the convergence speed for three possible parametrizations of a uniform and power-of-two quantizer. We choose this example as the gradient $\nabla_{\theta} Q(x; \theta) = E[(Q(x; \theta) - x)\nabla_{\theta} Q(x; \theta)]$ is just a scaled version of $\nabla_{\theta} Q(x; \theta)$, i.e., the gradient direction depends directly on the parametrization of $Q(x; \theta)$ and thus the effects of changing the parametrization can be observed.

It is interesting to study the Hessian $\mathbf{H} = \nabla_{\theta} \nabla_{\theta}^T E[(Q(x; \theta) - x)^2] \in \mathbb{R}^{2 \times 2}$ of the MSE:

$$\mathbf{H} = E[\nabla_{\theta} Q(x; \theta) \nabla_{\theta} Q(x; \theta)^T + (Q(x; \theta) - x) \nabla_{\theta} \nabla_{\theta}^T Q(x; \theta)] \approx E[\nabla_{\theta} Q(x; \theta) \nabla_{\theta} Q(x; \theta)^T].$$

Note that we use the outer-product approximation (Bishop, 2006) in order to simplify our considerations. From this equation it is apparent that the Hessian will be diagonal for the case U3 as $\nabla_{\theta} Q(x; \theta) \nabla_{\theta} Q(x; \theta)^T$ only contains an element in either (1, 1) or (2, 2) and, therefore, $E[\nabla_{\theta} Q(x; \theta) \nabla_{\theta} Q(x; \theta)^T]$ is a diagonal matrix. From this, we can see that gradient descent with an individual learning rate for each parameter is equivalent to Newton’s method and, therefore, efficient. In general this will not be the case for U1 and U2.

We conduct an experiment, using ADAM to optimize the mean squared quantization error on artificially generated data, which is generated by drawing 10^4 samples from $N(0, 1)$. Please note that the same example was studied in (Jain et al., 2019). The results in Fig. 3 clearly show that the parametrizations “U3” and “P3” are best suited to optimize the uniform and power-of-two quantization parameters, respectively. Indeed, these quantizers converge without oscillation to the lowest MSE. It is interesting to see, that even adaptive gradient methods like ADAM can not solve the scaling issue described in Sec. 2.1. In the Appendix A.4 we give further empirical evidence to support this claim and compare the different parametrizations for the training of a quantized ResNet-20 on CIFAR-10 using ADAM. Note that all cases use the same learning rate. For the interested reader, a more detailed visualization of the error surfaces over the quantization parameters can be found in Appendix A.3.

2) *CIFAR-10* In our second experiment we train a ResNet-20 (He et al., 2016) with quantized parameters and activations on CIFAR-10 (Krizhevsky & Hinton, 2009) using the same settings as proposed by (He et al., 2016). Fig. 4 shows the evolution of the training and validation error during training for the case of uniform quantization. The plots for power-of-two quantization can be found in the appendix (Fig. 10). We initialize this network from random parameters or from a pre-trained float network. The quantized DNNs are trained for 160 epochs, using SGD with momentum 0.9 and a learning rate schedule starting with 0.01 and reducing it by a factor of 10 after 80 and 120 epochs, respectively. We use random flips and crops for data augmentation. Each epoch takes about 2.5 min on a single GTX 1080 Ti.

In case of randomly initialized weights, we use an initial stepsize $d_l = 2^{-3}$ for the quantization of weights and activations. Otherwise, we initialize the weights using a pre-trained floating point network and the initial stepsize for a layer is chosen to be $d_l = 2^{\lfloor \log_2(\max |\mathbf{W}_l| / (2^{b-1} - 1)) \rfloor}$. The remaining quantization parameters are chosen such that we start from an initial bitwidth of $b = 4$ bit. This is a reasonable upper limit for b , as in practice no performance degradation can be observed for $b > 4$ bit. Even simple offline algorithms like min/max quantization result in networks with good accuracies. We define no memory constraints during training, i.e., the network can learn to use a large number of bits to quantize weights and activations of each layer. From Fig. 4, we again observe that the parametrization $\theta = [d, q_{\max}]^T$ is best suited to train a uniformly quantized DNN as it converges

Table 3: Homogeneous vs. heterogeneous quantization of ResNet-20 on CIFAR-10.

	Bitwidth Weight/Activ.	$q_{\max}$ Weight/Activ.	Size Weight/Activ.(max)/Activ.(sum)	Uniform quant. Validation error	Power-of-two quant. Validation error
Baseline	32bit/32bit	–	1048KB/64KB/736KB		7.29%
Fixed	2bit/32bit	fixed/–	65.5KB/64KB/736KB	10.81%	8.99%
TQT (Jain et al., 2019)	2bit/32bit	learned/–	65.5KB/64KB/736KB	9.47%	8.79%
Ours (w/ constr. (8a))	learned/32bit	learned/–	70KB/64KB/736KB	8.59%	8.53%
Fixed	2bit/4bit	fixed/learned	65.5KB/8KB/92KB	11.30%	11.62%
TQT (Jain et al., 2019)	2bit/4bit	learned/learned	65.5KB/8KB/92KB	9.62%	11.29%
Ours (w/ constr. (8a) and (8b))	learned/learned	learned/learned	70KB/–/92KB	9.38%	11.29%
Ours (w/ constr. (8a) and (8c))	learned/learned	learned/learned	70KB/8KB/–	8.58%	11.23%

ImageNet, we train the quantized DNNs for 50 epochs, using SGD with momentum 0.9 and a learning rate schedule starting with 0.01 and reducing it by a factor of 10 after 16 and 32 epochs, respectively. Please note that we quantize *all* layers opposed to other papers which use a higher precision for the first and/or last layer.

In our experiments, we noticed that the performance of DQ is not sensitive to the choice of λ_j in (10). For the CIFAR-10 experiments, we use $\lambda = 0.1$ for both constraints (for sizes in kB). For the ImageNet experiments, we kept the same regularization level by scaling λ_j with the square of the size ratio between the ImageNet model and the CIFAR-10 model. We scale with the square-ratio as the constraints in (10) are squared penalty terms.

First, in Table 3/top, we train a ResNet-20 on CIFAR-10 with quantized weights and float32 activations. We start with the most restrictive quantization scheme with fixed $q_{\max}$ and $b = 2\text{bit}$ (“Fixed”). Then, we allow the model to learn $q_{\max}$ while $b = 2\text{bit}$ remains fixed as was done in (Jain et al., 2019) (“TQT”). Finally, we learn both $q_{\max}$ and b with the constraint that the weight size is at most 70KB (“Ours”), which is just 4.5kB larger than the previous 2Bit networks. This allows the model to allocate more than two bits to some layers. From Table 3/top, we observe that the error is smallest when we learn all quantization parameters.

In Table 3/bottom, weights and activations are quantized. For activation quantization, we consider two cases as discussed in Sec. 3. The first one constrains the total activation memory S^x while the second constrains the maximum activation memory $\hat{S}^x$ such that both have the same size as a homogeneously quantized model with 4bit activations. Again, we observe that the error is smallest when we learn all quantization parameters.

We also use DQ to train quantized ResNet-18 (He et al., 2016) and MobileNetV2 (Sandler et al., 2018) on ImageNet (Deng et al., 2009) with 4bit uniform weights and activations or equivalent-sized networks with learned quantization parameters. This is quite aggressive and, thus, a fixed quantization scheme loses more than 6% accuracy while our quantization scheme loses less than 0.5% compared to a float32 precision network.

Our results compare favorably to other recent quantization approaches. To our knowledge, the best result for a 4bit ResNet-18 was reported by (Esser et al., 2019) (29.91% error). This is very close to our performance (29.92% error). Importantly, (Esser et al., 2019) did not quantize the first and last layers, meaning that their network is much bigger. Specifically, compared to our quantized ResNet-18, their model with high precision input and output layers requires 37% more memory to store the weights. Moreover, (Esser et al., 2019) learns stepsizes which are not restricted to powers-of-two. As explained in Sec. 2.2, uniform quantization with power-of-two stepsize leads to more efficient inference, effectively allowing to efficiently compute any multiplication with an integer multiplication and bit-shift. To our knowledge only (Wang et al., 2018) reported results of MobileNetV2 quantized to 4bit. They keep the baseline performance constraining the network to the same size as the 4bit network. However, they do not quantize the activations in this case. In addition, DQ training is efficient since it is comparable to the training of unquantized network. Specifically, one epoch on ImageNet takes 37min for MobileNetV2 and 18min for ResNet-18 on four Nvidia Tesla V100.

Fig. 5 shows the weight bitwidth assignment over layers. We observe that small bitwidths are used for layers with many parameters, i.e., pointwise convolutions and fully connected layers. However, the resulting bitwidth assignments are complex, meaning that there is no simple heuristic. Therefore, it is important to learn the optimal bitwidth assignment.

Table 4: Homogeneous vs. heterogeneous quantization of MobileNetV2 and ResNet-18 on ImageNet.

	Bitwidth		MobileNetV2		ResNet-18	
	Weight/Activ.	$q_{\max}$ Weight/Activ.	Size Weight/Activ(max)	Validation Error	Size Weight/Activ(max)	Validation Error
Baseline	32bit/32bit	—	13.23MB/4.59MB	29.82%	44.56MB/3.04MB	29.72%
Fixed	4bit/4bit	fixed/fixed	1.65MB/0.57MB	36.27%	5.57MB/0.38MB	34.15%
TQT (Jain et al., 2019)	4bit/4bit	learned/learned	1.65MB/0.57MB	32.21%	5.57MB/0.38MB	30.49%
Ours (w/ constr. (8a) and (8c))	learned/learned	learned/learned	1.55MB/0.57MB	30.26%	5.40MB/0.38MB	29.92%
Ours (w/o constr.)	learned/learned	learned/learned	3.14MB/1.58MB	29.41%	10.50MB/1.05MB	29.34%

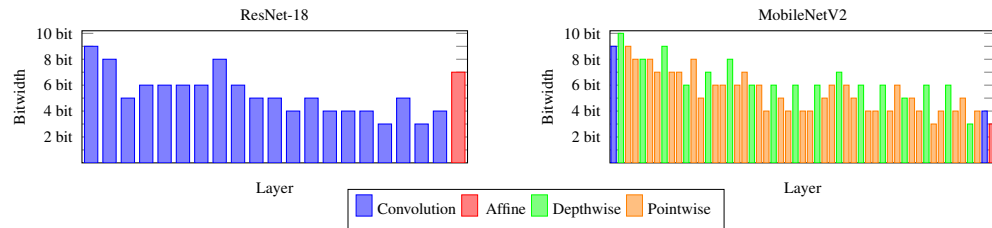


Figure 5: Weight bitwidth assignment over layers for ResNet-18 and MobileNetV2 on ImageNet with weights constrained to a maximum size of 5.57MB. Our method has learned a heterogeneous bitwidth distribution, which gives a better performance than a homogeneous one (see Table 4).

5 CONCLUSIONS

In this paper we discussed differentiable quantization and its application to the training of compact DNNs with memory constraints. In order to fulfill memory constraints, we introduced penalty functions during training and used stochastic gradient descent to find the optimal weights as well as the optimal quantization values in a joint fashion. We showed that there are several possible parametrizations of the quantization function. In particular, learning the bitwidth directly is not optimal; therefore, we proposed to parametrize the quantizer with the stepsize and dynamic range instead. The bitwidth can then be inferred from them. This approach is competitive to other recent quantization methods while it does not require to retrain the network multiple times in contrast to reinforcement learning approaches (Wang et al., 2018; Elthakeb et al., 2018).

ACKNOWLEDGEMENTS

We would like to thank Masato Ishii for many helpful comments during the preparation of this manuscript.

REFERENCES

- Yu Bai, Yu-Xiang Wang, and Edo Liberty. Proxquant: Quantized neural networks via proximal operators. *CoRR*, abs/1810.00861, 2018. URL <http://arxiv.org/abs/1810.00861>.
- Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- Dimitri P Bertsekas. *Constrained optimization and Lagrange multiplier methods*. Academic press, 2014.
- Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- Fabien Cardinaux, Stefan Uhlich, Kazuki Yoshiyama, Javier Alonso García, Stephen Tiedemann, Thomas Kemp, and Akira Nakamura. Iteratively training look-up tables for network quantization. *arXiv preprint arXiv:1811.05355*, 2018.
- Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. Ieee, 2009.

- Ahmed T. Elthakeb, Prannoy Pilligundla, Amir Yazdanbakhsh, Sean Kinzer, and Hadi Esmaeilzadeh. Releg: A reinforcement learning approach for deep quantization of neural networks. *CoRR*, abs/1811.01704, 2018. URL <http://arxiv.org/abs/1811.01704>.
- Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. *CoRR*, abs/1902.08153, 2019. URL <http://arxiv.org/abs/1902.08153>.
- Kaiyuan Guo, Shulin Zeng, Jincheng Yu, Yu Wang, and Huazhong Yang. A survey of fpga-based neural network accelerator. *arXiv preprint arXiv:1712.08934*, 2017.
- Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding. *CoRR*, abs/1510.00149, 2015. URL <http://arxiv.org/abs/1510.00149>.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *CoRR*, abs/1609.07061, 2016. URL <http://arxiv.org/abs/1609.07061>.
- Sambhav R. Jain, Albert Gural, Michael Wu, and Chris Dick. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *CoRR*, abs/1903.08066, 2019. URL <http://arxiv.org/abs/1903.08066>.
- Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical report, Citeseer, 2009.
- Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. *arXiv preprint arXiv:1605.04711*, 2016.
- Zhi-Gang Liu and Matthew Mattina. Learning low-precision neural networks without straight-through estimator (ste). *arXiv preprint arXiv:1903.01061*, 2019.
- Christos Louizos, Matthias Reisser, Tijmen Blankevoort, Efstratios Gavves, and Max Welling. Relaxed quantization for discretized neural networks. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HkxjYoCqKX>.
- Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. *CoRR*, abs/1611.00712, 2016. URL <http://arxiv.org/abs/1611.00712>.
- Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4510–4520, 2018.
- Sony. Neural Network Libraries (NNabla). <https://github.com/sony/nnabla>.
- Kuan Wang, Zhijian Liu, Yujun Lin, Ji Lin, and Song Han. HAQ: hardware-aware automated quantization. *CoRR*, abs/1811.08886, 2018. URL <http://arxiv.org/abs/1811.08886>.
- Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets. *arXiv preprint arXiv:1903.05662*, 2019.
- Aojun Zhou, Anbang Yao, Yiwen Guo, Lin Xu, and Yurong Chen. Incremental network quantization: Towards lossless cnns with low-precision weights. *arXiv preprint arXiv:1702.03044*, 2017.

Bayesian Bits: Unifying Quantization and Pruning

Mart van Baalen*, Christos Louizos*, Markus Nagel, Rana Ali Amjad,
Ying Wang, Tijmen Blankevoort, Max Welling

Qualcomm AI Research[†]

{mart, clouizos, markusn, ramjad, yinwan, tijmen, mwelling}@qti.qualcomm.com

Abstract

We introduce Bayesian Bits, a practical method for joint mixed precision quantization and pruning through gradient based optimization. Bayesian Bits employs a novel decomposition of the quantization operation, which sequentially considers doubling the bit width. At each new bit width, the residual error between the full precision value and the previously rounded value is quantized. We then decide whether or not to add this quantized residual error for a higher effective bit width and lower quantization noise. By starting with a power-of-two bit width, this decomposition will always produce hardware-friendly configurations, and through an additional 0-bit option, serves as a unified view of pruning and quantization. Bayesian Bits then introduces learnable stochastic gates, which collectively control the bit width of the given tensor. As a result, we can obtain low bit solutions by performing approximate inference over the gates, with prior distributions that encourage most of them to be switched off. We experimentally validate our proposed method on several benchmark datasets and show that we can learn pruned, mixed precision networks that provide a better trade-off between accuracy and efficiency than their static bit width equivalents.

1 Introduction

To reduce the computational cost of neural network inference, quantization and compression techniques are often applied before deploying a model in real life. The former reduces the bit width of weight and activation tensors by quantizing floating-point values onto a regular grid, allowing the use of cheap integer arithmetic, while the latter aims to reduce the total number of multiply-accumulate (MAC) operations required. We refer the reader to [18] and [19] for overviews of hardware-friendly quantization and compression techniques, respectively.

In quantization, the default assumption is that all layers should be quantized to the same bit width. While it has long been understood that low bit width quantization can be achieved by keeping the first and last layers of a network in higher precision [34; 5], recent work [7; 35; 36] has shown that carefully selecting the bit width of each tensor can yield a better trade-off between accuracy and complexity. Since the choice of quantization bit width for one tensor may affect the quantization sensitivity of all other tensors, the choice of bit width cannot be made without regarding the rest of the network.

The number of possible bit width configurations for a neural network is exponential in the number of layers in the network. Therefore, we cannot exhaustively search all possible configurations and pick the best one. Several approaches to learning the quantization bit widths from data have been proposed, either during training [35; 24], or on pre-trained networks [36; 7; 6]. However, these works do not consider the fact that commercially available hardware typically only supports efficient computation

*Equal contribution

[†]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

The drawback, similarly to [35], of such an approach is that there is no inherent control over the resulting bit widths.

[38] frames the mixed precision search problem as an architecture search. For each layer in their network, the authors maintain a separate weight tensor for each bit width under consideration. A stochastic version of DARTS [22] is then used to learn the optimal bit width setting jointly with the network’s weights. [36] model the assignment of bit widths as a reinforcement learning problem. Their agent’s observation consists of properties of the current layer, and its action space is the possible bit widths for a layer. The agent receives the validation set accuracy after a short period of fine-tuning as a reward. Besides the reward, the agent receives direct hardware feedback from a target device. This feedback allows the agent to adapt to specific hardware directly, instead of relying on proxy measures.

Learning the scale along with the model parameters for a fixed bit width network was independently introduced by [8] and [15]. Both papers redefine the quantization operation to expose the scale parameter to the learning process, which is then optimized jointly with the network’s parameters. Similarly, [5] reformulate the clipping operation such that the range of activations in a network can be learned from data, leading to activation ranges that are more amenable to quantization.

The recursive decomposition introduced in this paper shares similarities with previous work on residual vector quantization [4], in which the residual error of vectors quantized using K-means is itself (recursively) quantized. [9] apply this method to neural network weight compression: the size of a network can be significantly reduced by only storing the centroids of K-means quantized vectors. Our decomposition also shares similarities with [21]. A crucial difference is that Bayesian bits produces valid fixed-point tensors by construction whereas for [21] this is not the case. Concurrent work [40] takes a similar approach to ours. The authors restrict themselves to what is essentially one step of our decomposition (without handling the scales), and to conditional gating during inference on activation tensors. The decomposition is not extended to multiple bit widths.

4 Experiments

To evaluate our proposed method we conduct experiments on image classification tasks. In every model, we quantized all of the weights and activations (besides the output logits) using per-tensor quantization, and handled the batch norm layers as discussed in [18]. We initialized the parameters of the gates to a large value so that the model initially uses its full 32-bit capacity without pruning.

We evaluate our method on two axes: test set classification accuracy, and bit operations (BOPs), as a hardware-agnostic proxy to model complexity. Intuitively the BOP count measures the number of multiplication operations multiplied by the bit widths of the operands. To compute the BOP count we use the formula introduced by [1], but ignore the terms corresponding to addition in the accumulator since its bit width is commonly fixed regardless of operand bit width. We refer the reader to the Appendix for details. We include pruning by performing group sparsity on the output channels of the weight tensors only, as pruning an output channel of the weight tensor corresponds to pruning that specific activation. Output channel group sparsity can often be exploited by hardware [11].

Finally, we set the prior probability $p(z_{jk} = 1 \mid z_{(j/2)k} = 1) = e^{-\mu\lambda_{jk}}$, where λ_{jk} is proportional to the contribution of gate z_{jk} to the total model BOPs, which is a function of both the tensor k and the bit width j , and μ is a (positive) global regularization parameter. See the Appendix for details. It is worth noting that improvements in BOP count may not directly correspond to reduced latency on specific hardware. Instead, these results should be interpreted as an indication that our method can optimize towards a hardware-like target. One could alternatively encourage low memory networks by e.g. using the regularizer from [35] or even allow for hardware aware pruning and quantization by using e.g. latency timings from a hardware simulator, similar to [36].

We compare the results of our proposed approach to literature that considers both static as well as mixed precision architectures. If BOP counts for a specific model are not provided by the original papers, we perform our own BOP computations, and in some cases we run our own baselines to allow for apples-to-apples comparison to alternative methods (details in Appendix). All tensors (weight and activation) in our networks are quantized, and the bit widths of all quantizers in our network are learned, contrary to common practice in literature to keep the first and last layers of the networks in a higher bit width (e.g. [5; 38]).

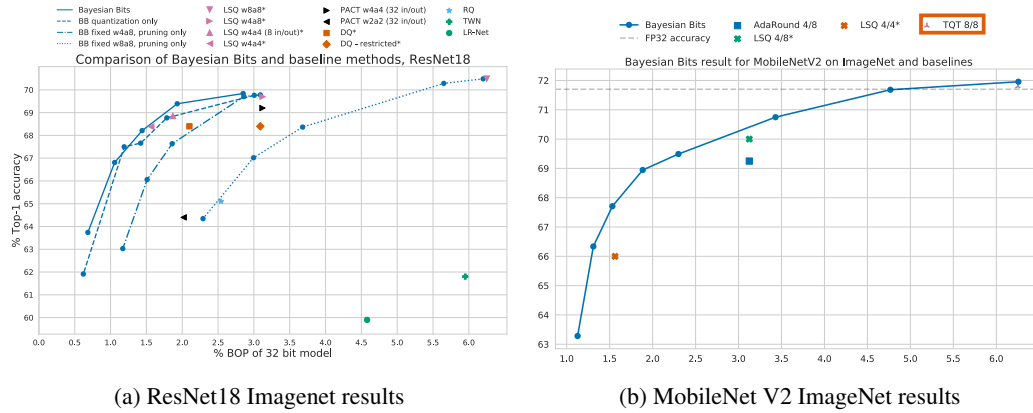


Figure 2: **Imagenet Results.** (a) Bayesian Bits Imagenet validation accuracy on ResNet18. Bayesian Bits and BB Quantization only use $\mu \in \{0.03, 0.05, 0.07, 0.2\}$. BB pruning only uses $\mu \in \{0.05, 0.2, 0.5, 0.7, 1\}$. The Bayesian Bits results show the mean over 3 training runs. The quantization only and pruning only results show the mean over 2 training results. The BOP count per model is presented in the Appendix. The notation ‘wXaY’ indicates a fixed bit width architecture with X bit weights and Y bit activations. ‘Z in/out’ indicates that the weights of the first layer as well as the inputs and weights of the last layer are kept in Z bits. In this plot we additionally compare to PACT [5]. Note that PACT uses 32 bit input and output layers, which negatively affects their BOP count. In the Appendix we compare against a hypothetical setting in which PACT with 8 bit input and output layers yields the same results. * results run by the authors. (b) Bayesian Bits results on MobileNet V2, compared to AdaRound [27], LSQ [8], and TQT [15] * results run by the authors.

parameter μ . Due to differences in experimental setup, we ran our own baseline experiments to obtain results for LSQ [8]. Full details of the differences between the published experiments and ours, as well as experimental setup for baseline experiments can be found in the Appendix.

Besides experiments with combined pruning and quantization, we ran two sets of ablation experiments in which Bayesian Bits was used for pruning a fixed bit width model, and for mixed precision quantization only, without pruning. This was achieved through learning only the 4 bit and higher gates for the quantization only experiment, and only the zero bit gates for the pruning only experiment. In 2a we see that combining pruning with quantization yields superior results.

We provide the tables of the results in the Appendix along with visualizations of the learned architectures. Overall, we observe that Bayesian Bits provides better trade-offs between accuracy and efficiency compared to the baselines. NB: we cannot directly compare our results to those of [36], [7; 6] and [38], for reasons outlined in the Appendix, and therefore omit these results in this paper.

A note on computational cost Bayesian Bits requires the computation of several residual error tensors for each weight and activation tensor in a network. While the computational overhead of these operations is very small compared to the computational overhead of the convolutions and matrix multiplications in a network, we effectively need to store N copies of the model for each forward pass, for N possible quantization levels. To alleviate the resulting memory pressure and allow training with reasonable batch sizes, we use gradient checkpointing [3]. Gradient checkpointing itself incurs extra computational overhead. The resulting total runtime for one ResNet18 experiment, consisting of 30 epochs of training with Bayesian Bits and 10 epochs of fixed-gate fine-tuning, is approximately 70 hours on a single Nvidia TeslaV100. This is a slowdown of approximately 2X compared to 40 epochs of quantization aware training.

4.2.1 Post-training mixed precision

In this experiment, we evaluate the ability of our method to find sensible mixed precision settings by running two sets of experiments on a pre-trained ResNet18 model and a small version of ImageNet. In the first experiment only the values of the gates are learned, while in the second experiment both the values of the gates and the quantization ranges are learned. In both experiments the weights are not updated. We compare this method to an iterative baseline, in which weights and activation

- neural networks. *arXiv preprint arXiv:1804.10969*, 2018.
- [2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
 - [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
 - [4] Yongjian Chen, Tao Guan, and Cheng Wang. Approximate nearest neighbor search by residual vector quantization. *Sensors*, 10(12):11259–11273, 2010.
 - [5] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018.
 - [6] Zhen Dong, Zhewei Yao, Yaohui Cai, Daiyaan Arfeen, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Hawq-v2: Hessian aware trace-weighted quantization of neural networks. *arXiv preprint arXiv:1911.03852*, 2019.
 - [7] Zhen Dong, Zhewei Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. HAWQ: hessian aware quantization of neural networks with mixed-precision. *International Conference on Computer Vision (ICCV)*, 2019.
 - [8] Steven K. Esser, Jeffrey L. McKinstry, Bablani Deepika, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. *International Conference on Learning Representations (ICLR)*, 2020.
 - [9] Yunchao Gong, Liu Liu, Ming Yang, and Lubomir Bourdev. Compressing deep convolutional networks using vector quantization. *International Conference on Learning Representations (ICLR)*, 2015.
 - [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
 - [11] Yihui He, Xiangyu Zhang, and Jian Sun. Channel pruning for accelerating very deep neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1389–1397, 2017.
 - [12] Geoffrey E Hinton and Drew Van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Conference on Computational learning theory (COLT)*, 1993.
 - [13] Andrey Ignatov, Radu Timofte, Andrei Kulik, Seungsoo Yang, Ke Wang, Felix Baum, Max Wu, Lirong Xu, and Luc Van Gool. Ai benchmark: All about deep learning on smartphones in 2019. *International Conference on Computer Vision (ICCV) Workshops*, 2019.
 - [14] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
 - [15] Sambhav R. Jain, Albert Gural, Michael Wu, and Chris Dick. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *arxiv preprint arxiv:1903.08066*, 2019.
 - [16] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.
 - [17] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *International Conference on Learning Representations (ICLR)*, 2014.
 - [18] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.

Bringing AI To Edge: From Deep Learning's Perspective

Di Liu, Hao Kong, Xiangzhong Luo, Weichen Liu, Ravi Subramaniam

Abstract—Edge computing and artificial intelligence (AI), especially deep learning for nowadays, are gradually intersecting to build a novel system, called *edge intelligence*. However, the development of edge intelligence systems encounters some challenges, and one of these challenges is the *computational gap* between computation-intensive deep learning algorithms and less-capable edge systems. Due to the computational gap, many edge intelligence systems cannot meet the expected performance requirements. To bridge the gap, a plethora of deep learning techniques and optimization methods are proposed in the past years: light-weight deep learning models, network compression, and efficient neural architecture search. Although some reviews or surveys have partially covered this large body of literature, we lack a systematic and comprehensive review to discuss all aspects of these deep learning techniques which are critical for edge intelligence implementation. As various and diverse methods which are applicable to edge systems are proposed intensively, a holistic review would enable edge computing engineers and community to know the state-of-the-art deep learning techniques which are instrumental for edge intelligence and to facilitate the development of edge intelligence systems. This paper surveys the representative and latest deep learning techniques that are useful for edge intelligence systems, including hand-crafted models, model compression, hardware-aware neural architecture search and adaptive deep learning models. Finally, based on observations and simple experiments we conducted, we discuss some future directions.

Index Terms—Deep learning, model optimization, edge computing, neural architecture search

I. INTRODUCTION

In 2012, AlexNet [1] broke the record of ImageNet LSVRC contest [2], improving the prediction accuracy by a large margin of 10%, which marked the milestone for the booming development of deep learning (DL) techniques, or more specifically deep neural network (DNN) [3]. In the subsequent years, as the increasing number of DNN applications [4] emerges, designing DNN-based systems attracts a lot of attention and

efforts from academia and industry, spanning from advanced image manipulation and enhancement and convenient voice assistant on mobile phones to Level-4 autonomous driving systems on automobiles [5]. AlphaGo [6] is another prominent example which defeated the top professional player in the ancient Chinese game GO, deemed impossible before due to its extremely high complexity.

The successful application of DNN models relies on two stages: **training** and **inference/deployment**. Training indicates the procedure of learning a predictive model from a huge amount of labeled data, while inference denotes the procedure of using the trained model upon new data to make an accurate prediction. The training procedure involves a complex learning process requiring powerful computational units and a huge amount of data and spending considerable time. For instance, training ResNet50 [7] with ImageNet on 8 Nvidia Tesla P100 takes 29 hours [8], where ImageNet dataset [2] has 1.28 M images of 1000 classes for training. Recently, researchers also measure the environmental effect of the training procedure and indicate that training DNN consumes excessive energy and emits substantial CO₂ [9] and environment-sustainable DNN models should be considered.

The trained DNN model is exploited to make prediction on new data, and this procedure is usually called DNN *inference*. Many companies implement their DNN inference on servers to provide convenient services for their customers such as voice-assistant, machine translation, image retrieval, etc [10]. However, inference from cloud suffers from *responsiveness* (e.g., latency) and *privacy* issues which are critical in some scenarios. For instance, DNNs are seen to be a pivotal technique for self-driving cars, in which most of executions are subject to highly rigorous real-time constraints [5], [11], but the high and uncertain communication overhead makes it difficult to satisfy the temporal requirement. In addition, some inference is conducted on confidential data, e.g., manufacturing and production data, and uploading these data to cloud may risk data leakage which will severely hurt their business. To address these concerns, *edge computing* is proposed [12].

Edge computing systems are placed at the proximity of data producer, like sensors, end-users, etc, to rapidly and locally process data. The emergence of edge computing paves the way for pervasive intelligent systems [13]. IEEE Computer Society identifies ‘Artificial Intelligence at the edge’ as one of the top 12 technology trends to reach adoption in 2020 [14]. Edge systems have a broad spectrum of hardware features, from powerful computation units in communication stations and self-driving cars to small and battery-supplied smart phones and wearable devices. Fig. 1 plots the overview of edge

Di Liu, Hao Kong, and Xiangzhong Luo contribute equally to this work.

This research was conducted in collaboration with HP Inc. and supported by National Research Foundation (NRF) Singapore and the Singapore Government through the Industry Alignment Fund-Industry Collaboration Projects Grant (I1801E0028). This work is also partially supported by NTU NAP M4082282 and SUG M4082087, Singapore.

Di Liu is with HP-NTU Digital Manufacturing Corporate Lab, Nanyang Technological University, Singapore. (Email: liu.di@ntu.edu.sg)

Hao Kong and Weichen Liu are with HP-NTU Digital Manufacturing Corporate Lab, Nanyang Technological University, Singapore, and School of Computer Science and Engineering, Nanyang Technological University, Singapore. (Email: kong.hao@ntu.edu.sg, liu@ntu.edu.sg)

Xiangzhong Luo is with School of Computer Science and Engineering, Nanyang Technological University, Singapore. (Email: xiangzhong001@e.ntu.edu.sg)

Ravi Subramaniam is with Innovations and Experiences–Business Personal Systems, HP Inc., USA. (Email: ravi.subramaniam@hp.com)

criterion for each layer by using a gradient-based method. HRank [99] empirically finds that filters with low-rank is less informative than those with high-rank, and thus uses this observation to prune the unimportant channels.

Wen *et al.* [89] propose SSL to have a training-based pruning method, learning a compact and sparse model from a pre-trained model. Liu *et al.* [100] propose an approach called network slimming, which takes wide and large networks as input models, but during training, insignificant channels are automatically identified and pruned afterwards. Soft filter pruning (SFP) [94] deploy a training-based pruning method to prune a complex network. Huang *et al.* [95] propose the concept of sparsity scaling factor for each filter which is learned during to training, and then filters with scaling factor 0 are removed. Yao *et al.* [92] train an recurrent neural network (RNN) to determine the dropout probability of each filters and prune the network layer by layer according to drop probability.

Most of the above-mentioned *structure pruning* methods are rule-based, i.e., some heuristic algorithms devised according to their own criterion. In contrary, some work employ learning algorithm to automatically prune network model. AMC [96] proposes to use *reinforcement learning* to automatically prune channels of each layer. Anwar *et al.* [101] prune a DNN model at feature level, kernel level, and intra-kernel level (i.e., weight), where they deploy *evolutionary algorithm* to find the best combination of different pruning granularities. However, searching in a discrete space using RL and EA is really costly, so the gradient-based method is recently proposed to find an optimal pruning criterion for each layer [98].

To determine how many channels should be pruned, the above-mentioned works use indirect metrics like FLOPs or compression ratio to prune network. Nevertheless, the reduced FLOPs and compression ratio cannot directly translate to the performance improvement. In addition, a diverse of hardware accelerators have emerged for boosting the execution of DNNs, but these various systems demonstrate different capability to handling network complication. Hence, some pruning studies directly target the direct metric upon a specific hardware, e.g., latency. NetAdapt [47] proposes an automated framework to prune filters in different layers such that the pruned model can be adapted to a target platform. To optimize the latency on a target platform, NetAdapt builds up a look-up-table (LUT) for different operations and layers, so instead of measuring latency on the real platform, it can quickly estimate the latency based on the model architecture and LUT. Yu *et al.* [102] introduce SIMD-aware pruning framework which employs different pruning strategies for different underlying hardware, like weight pruning for low-parallelism CPU and filter pruning for high-parallelism.

Discussion: Table III and IV summarize methods discussed in this section. Pruning is the well-studied topic in model compression and many methods have been proposed to advance the network pruning [103]. These pruning methods show promising results, capable of compressing a DNN model, in some cases more than 40x, while only degrading the prediction accuracy slightly [76]. To effectively adopt model pruning for edge intelligence systems, the existing works suffer from two flaws: 1) the theoretical foundation behind

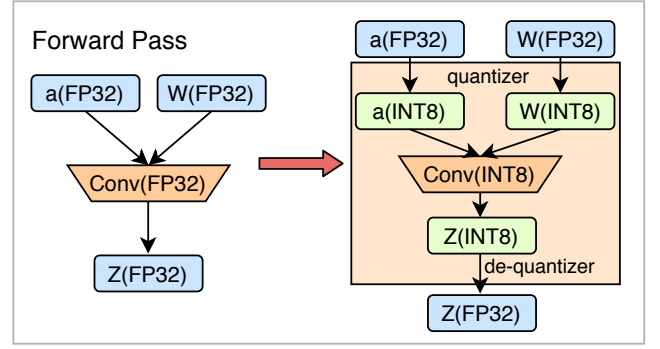


Fig. 6. An example of convolution quantization.

TABLE V
THE CLASSIFICATION OF QUANTIZATION METHODS.

Methodologies	Bits	Weights	Activations	Year
Gupta <i>et al.</i> [109]	16-bit	Yes	No	2015
Q-CNN [110]	1-bit	Yes	No	2016
BinaryConnect [111]	1-bit	Yes	No	2016
Ternary [112]	2-bit	Yes	No	2016
DoReFa-Net [113]	Arbitrary	Yes	Yes	2016
XNOR-Net [114]	1-bit	Yes	Yes	2016
INQ [115]	5/4-bit [†]	Yes	Yes	2017
ABC-Net [116]	1-bit	Yes	Yes	2017
Zhu <i>et al.</i> [117]	2-bit	Yes	No	2017
Jacob [118]	8-bit	Yes	Yes	2018
Bi-Real-Net [119]	1-bit	Yes	Yes	2018
LO-Net [120]	Arbitrary	Yes	Yes	2018
TQT [121]	8-bit	Yes	Yes	2019
HAQ [122]	(1~8)-bit [§]	Yes	Yes	2019
HAWQ [123]	2/4-bit [†]	Yes	Yes	2019
Jung <i>et al.</i> [124]	(2~4)-bit	Yes	Yes	2019
Zhuang <i>et al.</i> [125]	2-bit	Yes	Yes	2020
XOR-Net [126]	2-bit	Yes	Yes	2020

a/b -bit[†] denotes the method quantizes the network weights for a bits while operating intermediate activation feature maps for b bits.

$(a \sim b)$ -bit[§] represents the method quantizes the network weights or activations from a bits to b bits.

pruning lacks, and thus there is a debate whether we should prune a complex model or directly train a compact model for resource constrained hardware platforms. Some recent research [104] [105] [106] [107] strives to empirically or theoretically find an answer for this question. Since pruning is a time-consuming procedure involving large model training and iterative pruning-retraining procedure, a theoretical foundation or proof may drastically change the way we use model pruning to design edge DNN models; 2) Almost all of pruning methods are *hardware-agnostic* and depend on *hardware-independent* metrics, like MACs and FLOPs. Since different hardware architectures demonstrate different degrees of parallelism, the state-of-the-art methods may unnecessarily prune models without performance improvement (e.g., latency reduction) while reducing the capacity of DNN models which is proven to have a significant impact on model's accuracy [108]. Therefore, we need to devise pruning method with hardware-awareness such that the model can be tailored for various hardware.

B. Quantization

Network pruning reduces the complexity of DNN models by removing redundant weights or channels. However, the

state-of-the-art models have more than billions of parameters and at the same time during inference a model produces a large portion of intermediate results (activation/feature maps) which usually occupy a large memory space. As a result, the huge memory requirement prohibits DNN models from implementing on memory-limited edge devices [127], [128]. For example, ResNet-50 [7] has 26 million parameter weights, generates 16 million activations in one inference, requires around 168 MB memory space, and needs at least 3GB/s memory bandwidth. It is not difficult to see that it is unlikely to deploy these state-of-the-art models to edge devices, which have limited storage and computational resources.

In this case, *quantization* becomes a promising approach to address the aforementioned issue, which encodes full-precision (FP32) weights and activations with low-precision ones (e.g., FP16, INT8, binary) while preserving even the same level of accuracy. Some early work has shown that using FP16 to train DNN models can reduce the computational cost while retaining accuracy [109]. Quantization significantly benefits DNN models on resource-limited devices, and it is capable of fitting the whole model into on-chip memory of edge devices such that the high overhead occurred by off-chip memory access can be mitigated. In addition, since operations with low-bit representation usually consume less energy and execute faster, quantization reduces energy consumption and latency as well on some hardware platforms [129]. In this section, we discuss some state-of-the-art quantization studies.

Wu *et al.* [110] propose a unified quantization framework, which improves the quantization performance by minimizing the estimation error of each layer's response. On this basis, an error correction training strategy is included. Jain *et al.* [121] propose a trained uniform quantization method for accurate and efficient neural network inference on fixed-point hardware. Jacob *et al.* [118] quantize models into 8-bit integer and propose a quantize-aware training method to eliminate the accuracy loss caused by quantization. Instead of quantizing all weights in a DNN, IQN [115] adopts a group-wise quantization method to gradually quantize weights of a DNN model. The advantage of IQN is that it is able to derive the quantized model without accuracy loss.

The above works uniformly convert all weights and activation into the same low-bit representation, but many emergent hardware and accelerators support mixed precision operations, e.g., Nvidia Turing GPU architecture supports 1-bit, 4-bit, 8-bit and 16-bit arithmetic operations [130]. This provides a more effective and flexible way to quantize weights and activations of a DNN model. Wang *et al.* [122] propose a hardware-aware automated quantization approach, namely HAQ. HAQ exploits reinforcement learning to select different quantization width for each layer upon a target hardware. Additionally, the hardware architecture is involved into the learning loop, so that it can directly reduce the inference latency, energy and storage on the target hardware. HAWQ [123] also considers to quantize a full-precision model into a mix-precision model where the quantized precision is determined for each layer based on Hessian matrix.

Some works use binary or ternary quantization to maximally compress DNN models. Then, binarized or ternarized network

can use cheap bit operation to boost the efficiency on dedicated hardware [131]. In [111], Courbariaux *et al.* propose the binaryconnect, which targets to transform the full-precision weights into the binary format. A very straightforward binarization method would be based on the sign function:

$$w_b = \begin{cases} +1 & \text{if } w \geq 0 \\ -1 & \text{otherwise} \end{cases} \quad (1)$$

where w_b is the binary weight and w is the full-precision weight. Due to the fact that this is a deterministic operation, averaging the discretization over the many input weights of a hidden unit could compensate for the loss of information. An alternative that allows a finer and more correct averaging process to take place is to binarize stochastically, which helps to improve the model generalization capability:

$$w_b = \begin{cases} +1 & \text{with probability } p = \delta(w) \\ -1 & \text{with probability } 1 - p \end{cases} \quad (2)$$

where δ is the *hard sigmoid* formula:

$$\delta(x) = \text{clip}\left(\frac{x+1}{2}, 0, 1\right) = \max(0, \min(1, \frac{x+1}{2})) \quad (3)$$

Intuitively, applying a binarized method is an easy way to quantize the full precision weights. However, this will be harder for the training process to converge due to the highly discrete parameter space, which drastically degrades the model performance.

Later, Rastegari *et al.* [114] present an efficient approximation strategy, which constrains the full precision weights to +1 and -1 instead of directly rounding them. With a scaling factor α , a convolutional operation can be approximated as follows:

$$\mathcal{I} * \mathcal{W} \approx \mathcal{I} * (\alpha \mathcal{B}) = \alpha \mathcal{I} * \mathcal{B} \quad (4)$$

where $\mathcal{I}$ is the input image and $\mathcal{B} \in \{-1, +1\}$ is the binarized weight. Other binarized methods are proposed to improve the degraded accuracy caused by binarization like [113], [116], [119]. XOR-Net [126] takes the implementation of BNNs into account and simplifies the number of instructions used in binary dot product to accelerate BNN inference on edge devices.

Inspired by [114], Li *et al.* [112] propose ternary weight networks (TWNs) with weights constrained to $\{-1, 0, +1\}$, which minimizes the Euclidian distance between full precision weights $\mathcal{W}$ and the ternary weights $\mathcal{W}^t$ along with a scaling factor α . Here the quantized weights are obtained with a threshold-based ternary function:

$$\mathcal{W}_i^t = \begin{cases} +1, & \text{if } \mathcal{W}_i > \Delta \\ +0, & \text{if } |\mathcal{W}_i| \leq \Delta \\ -0, & \text{if } \mathcal{W}_i < -\Delta \end{cases} \quad (5)$$

where Δ is a positive threshold parameter. Thus, the optimization objective can be formulated as follows:

$$\alpha^*, \mathcal{W}^{t*} = \arg \min J(\alpha, \mathcal{W}^t) = \|\mathcal{W} - \alpha \mathcal{W}^t\|_2^2 \quad (6)$$

- [96] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "Amc: Automl for model compression and acceleration on mobile devices," in *The European Conference on Computer Vision (ECCV)*, September 2018.
- [97] Z. You, K. Yan, J. Ye, M. Ma, and P. Wang, "Gate decorator: Global filter pruning method for accelerating deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2019, pp. 2130–2141.
- [98] Y. He, Y. Ding, P. Liu, L. Zhu, H. Zhang, and Y. Yang, "Learning filter pruning criteria for deep convolutional neural networks acceleration," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2009–2018.
- [99] M. Lin, R. Ji, Y. Wang, Y. Zhang, B. Zhang, Y. Tian, and L. Shao, "Hrank: Filter pruning using high-rank feature map," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1529–1538.
- [100] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2736–2744.
- [101] S. Anwar, K. Hwang, and W. Sung, "Structured pruning of deep convolutional neural networks," *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 13, no. 3, pp. 1–18, 2017.
- [102] J. Yu, A. Lukefahr, D. J. Palfreman, G. S. Dasika, R. Das, and S. A. Mahlke, "Scalpel: Customizing dnn pruning to the underlying hardware parallelism," *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 548–560, 2017.
- [103] D. Blalock, J. J. G. Ortiz, J. Frankle, and J. Gutttag, "What is the state of neural network pruning?" *arXiv preprint arXiv:2003.03033*, 2020.
- [104] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=rJlnB3C5Ym>
- [105] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=rJl-b3RcF7>
- [106] Y. Wang, X. Zhang, L. Xie, J. Zhou, H. Su, B. Zhang, and X. Hu, "Pruning from scratch," *arXiv preprint arXiv:1909.12579*, 2019.
- [107] E. Malach, G. Yehudai, S. Shalev-Shwartz, and O. Shamir, "Proving the lottery ticket hypothesis: Pruning is all you need," *arXiv preprint arXiv:2002.00585*, 2020.
- [108] S. Zagoruyko and N. Komodakis, "Wide residual networks," *arXiv preprint arXiv:1605.07146*, 2016.
- [109] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *International Conference on Machine Learning*, 2015, pp. 1737–1746.
- [110] J. Wu, C. Leng, Y. Wang, Q. Hu, and J. Cheng, "Quantized convolutional neural networks for mobile devices," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4820–4828.
- [111] M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," in *Advances in neural information processing systems*, 2015, pp. 3123–3131.
- [112] F. Li, B. Zhang, and B. Liu, "Ternary weight networks," *arXiv preprint arXiv:1605.04711*, 2016.
- [113] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients," *arXiv preprint arXiv:1606.06160*, 2016.
- [114] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," in *European conference on computer vision*. Springer, 2016, pp. 525–542.
- [115] A. Zhou, A. Yao, Y. Guo, L. Xu, and Y. Chen, "Incremental network quantization: Towards lossless cnns with low-precision weights," *arXiv preprint arXiv:1702.03044*, 2017.
- [116] X. Lin, C. Zhao, and W. Pan, "Towards accurate binary convolutional neural network," in *Advances in Neural Information Processing Systems*, 2017, pp. 345–353.
- [117] C. Zhu, S. Han, H. Mao, and W. J. Dally, "Trained ternary quantization," *arXiv preprint arXiv:1612.01064*, 2016.
- [118] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [119] Z. Liu, B. Wu, W. Luo, X. Yang, W. Liu, and K.-T. Cheng, "Bi-real net: Enhancing the performance of 1-bit cnns with improved representational capability and advanced training algorithm," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 722–737.
- [120] D. Zhang, J. Yang, D. Ye, and G. Hua, "Lq-nets: Learned quantization for highly accurate and compact deep neural networks," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 365–382.
- [121] S. R. Jain, A. Gural, M. Wu, and C. Dick, "Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware," *arXiv preprint arXiv:1903.08066*, 2019.
- [122] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "Hq: Hardware-aware automated quantization with mixed precision," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.
- [123] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "Hawq: Hessian aware quantization of neural networks with mixed-precision," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 293–302.
- [124] S. Jung, C. Son, S. Lee, J. Son, J.-J. Han, Y. Kwak, S. J. Hwang, and C. Choi, "Learning to quantize deep networks by optimizing quantization intervals with task loss," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4350–4359.
- [125] B. Zhuang, L. Liu, M. Tan, C. Shen, and I. Reid, "Training quantized neural networks with a full-precision auxiliary module," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1488–1497.
- [126] L. H. K. D. Shien Zhu and W. Liu, "Xor-net: An efficient computation pipeline of binary neural network inference on edge devices," in *The 26th IEEE International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2020.
- [127] N. D. Lane, S. Bhattacharya, P. Georgiev, C. Forlivesi, and F. Kawsar, "An early resource characterization of deep learning on wearables, smartphones and internet-of-things devices," in *Proceedings of the 2015 international workshop on internet of things towards applications*, 2015, pp. 7–12.
- [128] H. Cai, C. Gan, L. Zhu, and S. Han, "Tiny transfer learning: Towards memory-efficient on-device learning," *arXiv preprint arXiv:2007.11622*, 2020.
- [129] W. J. Dally, Y. Turakhia, and S. Han, "Domain-specific hardware accelerators," *Communications of the ACM*, vol. 63, no. 7, pp. 48–57, 2020.
- [130] J. Burgess, "Rtx on—the nvidia turing gpu," *IEEE Micro*, vol. 40, no. 2, pp. 36–44, 2020.
- [131] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, "Finn: A framework for fast, scalable binarized neural network inference," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2017, pp. 65–74.
- [132] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," 2017.
- [133] M. Abadi *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: <http://tensorflow.org/>
- [134] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.
- [135] J. Lin, C. Gan, and S. Han, "Defensive quantization: When efficiency meets robustness," *arXiv preprint arXiv:1904.08444*, 2019.
- [136] C. Gong, Z. Jiang, D. Wang, Y. Lin, Q. Liu, and D. Z. Pan, "Mixed precision neural architecture search for energy efficient deep learning," in *ICCAD*, 2019, pp. 1–7.
- [137] G. Chen, H. Meng, Y. Liang, and K. Huang, "Gpu-accelerated real-time stereo estimation with binary neural network," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 12, pp. 2896–2907, 2020.
- [138] T. Wang, K. Wang, H. Cai, J. Lin, Z. Liu, H. Wang, Y. Lin, and S. Han, "Apq: Joint search for network architecture, pruning and quantization policy," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2078–2087.
- [139] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "Fitnets: Hints for thin deep nets," *arXiv preprint arXiv:1412.6550*, 2014.
- [140] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

HMQ: Hardware Friendly Mixed Precision Quantization Block for CNNs

Hai Victor Habi, Roy H. Jennings, and Arnon Netzer

Sony Semiconductor Israel
{hai.habi, roy.jennings, arnon.netzer}@sony.com

July 21, 2020

Abstract

Recent work in network quantization produced state-of-the-art results using mixed precision quantization. An imperative requirement for many efficient edge device hardware implementations is that their quantizers are uniform and with power-of-two thresholds. In this work, we introduce the Hardware Friendly Mixed Precision Quantization Block (HMQ) in order to meet this requirement. The HMQ is a mixed precision quantization block that repurposes the Gumbel-Softmax estimator into a smooth estimator of a pair of quantization parameters, namely, bit-width and threshold. HMQs use this to search over a finite space of quantization schemes. Empirically, we apply HMQs to quantize classification models trained on CIFAR10 and ImageNet. For ImageNet, we quantize four different architectures and show that, in spite of the added restrictions to our quantization scheme, we achieve competitive and, in some cases, state-of-the-art results.

1 Introduction

In recent years, convolutional neural networks (CNNs) produced state-of-the-art results in many computer vision tasks including image classification [14, 17, 21, 22, 38, 39], object detection [29, 36, 40], semantic segmentation [31, 37], etc. Deploying these models on embedded devices is a challenging task due to limitations on available memory, computational power and power consumption.

Many works address these issues using different methods. These include pruning [16, 45, 47], efficient neural architecture design [14, 21, 24, 38], hardware and CNN co-design [14, 20, 43] and quantization [6, 13, 15, 23, 24, 46].

In this work, we focus on quantization, an approach in which the model is compressed by reducing the bit-widths of weights and activations. Besides re-

The code of this work is available in <https://github.com/sony-si/ai-research>.

duction in memory requirements, depending on the specific hardware, quantization usually also results in the reduction of both latency and power consumption. The challenge of quantization is to reduce the model size without compromising its performance. For high compression rates, this is usually achieved by *fine-tuning* a pre-trained model for quantization. In addition, recent work in quantization focused on making quantizers more *hardware friendly* (amenable to deployment on embedded devices) by restricting quantization schemes to be: per-tensor, uniform, symmetric and with thresholds that are powers of two [24, 41].

Recently, *mixed-precision* quantization was studied in [12, 41, 42, 44]. In these works, the bit-widths of weights and activations are not equal across the model and are learned during some optimization process. In [42], reinforcement learning is used, which requires the training of an agent that decides the bit-width of each layer. In [44], neural architecture search is used, which implies duplication of nodes in the network and that the size of the model grows proportionally to the size of the search space of bit-widths. Both of these methods limit the bit-width search space because of their computational cost. In [12], the bit-widths are not searched during training, but rather, this method relies on the relationship between the layer's Hessian and its sensitivity to quantization.

An imperative requirement for many efficient edge device hardware implementations is that their quantizers are symmetric, uniform and with power-of-two thresholds (see [24]). This removes the cost of special handling of zero points and real value scale factors. In this work, we introduce a novel quantization block we call the *Hardware Friendly Mixed Precision Quantization Block* (HMQ) that is designed to search over a finite set of quantization schemes that meet this requirement. HMQs utilize the Gumbel-Softmax estimator [25] in order to optimize over a categorical distribution whose samples correspond to quantization scheme parameters.

We propose a method, based on HMQs, in which both the bit-width and the quantizer's threshold are searched simultaneously. We present state-of-the-art results on MobileNetV1, MobileNetV2 and ResNet-50 in most cases, in spite of the hardware friendly restriction applied to the quantization schemes. Additionally, we present the first (that we know of) mixed precision quantization results of EfficientNet-B0. In particular, our contributions are the following:

- We introduce HMQ, a novel, hardware friendly, mixed precision quantization block which enables a simple and efficient search for quantization parameters.
- We present an optimization method, based on HMQs, for mixed precision quantization in which we search simultaneously for both the bit-width and the threshold of each quantizer.
- We present competitive and, in most cases, state-of-the-art results using our method to quantize ResNet-50, EfficientNet-B0, MobileNetV1 and MobileNetV2 classification models on ImageNet.

2 Related Work

Quantization lies within an active area of research that tries to reduce memory requirements, power consumption and inference latencies of neural networks. These works use techniques such as pruning, efficient network architectures and distillation (see e.g. [7, 14, 15, 18, 19, 21, 30, 34, 35, 38, 39, 48]). Quantization is a key method in this area of research which compresses and accelerates the model by reducing the number of bits used to represent model weights and activations.

Quantization. Quantization techniques can be roughly divided into two families: post-training quantization techniques and quantization-aware training techniques. In post-training quantization techniques, a trained model is quantized without retraining the model (see e.g. [1, 5]). In quantization-aware training techniques, a model undergoes an optimization process during which the model is quantized. A key challenge in this area of research, is to compress the model without significant degradation to its accuracy. Post-training techniques suffer from a higher degradation to accuracy, especially for high compression rates.

Since the gradient of quantization functions is zero almost everywhere, most quantization-aware training techniques use the straight through estimator (STE) [4] for the estimation of the gradients of quantization functions. These techniques mostly differ in their choice of quantizers, the quantizers' parametrization (thresholds, bit-widths, step size, etc.) and their training procedure. During training, the network weights are usually stored in full-precision and are quantized before they are used in feed-forward. The full-precision weights are then updated via back-propagation. Uniform quantizers are an important family of quantizers that have several benefits from a hardware point-of-view (see e.g. [13, 24, 41]). Non-uniform quantizers include clustering, logarithmic quantization and others (see e.g. [3, 33, 46, 49]).

Mixed precision. Recent works on quantization produced state-of-the-art results using mixed precision quantization, that is, quantization in which the bit-widths are not constant across the model (weights and activations). In [42], reinforcement learning is used to determine bit-widths. In [12], second order gradient information is used to determine bit-widths. More precisely, the bit-widths are selected by ordering the network layers using this information. In [41], bit-widths are determined by learnable parameters whose gradients are estimated using STE. This work focuses on the choice of parametrization of the quantizers and shows that the threshold (dynamic range) and step size are preferable over parametrizations that use bit-widths explicitly.

In [44], a mixed precision quantization-aware training technique is proposed where the bit-widths search is converted into a network architecture search (based on [27]). More precisely, in this solution, the search space of all possible quantization schemes is, in fact, a search for a sub-graph in a super-net. The disadvantage of this approach, is that the size of the super net grows substantially with every optional quantization edge/path that is added to the super

net. In practice, this limits the architecture search space. Moreover, this work deals with bit-widths and thresholds as two separate problems where thresholds follow the solution in [8].

3 The HMQ Block

The *Hardware Friendly Mixed Precision Quantization Block* (HMQ) is a network block that learns, via standard SGD, a uniform and symmetric quantization scheme. The scheme is parametrized by a pair (t, b) of threshold t and bit-width b . During training, an HMQ searches for (t, b) over a finite space $T \times B \subseteq \mathbb{R}^+ \times \mathbb{N}$. In this work, we make HMQs “hardware friendly” by also forcing their thresholds to be powers of two. We do this by restricting

$$T = \{2^M, 2^{M-1}, \dots, 2^{M-8}\} \quad (1)$$

where $M \in \mathbb{Z}$ is an integer we configure per HMQ (see Section 4).

The step size Δ of a uniform quantization scheme is the (constant) gap between any two adjacent quantization points. Δ is parametrized by (t, b) differently for a signed quantizer, where $\Delta = \frac{2t}{2^b}$, and an unsigned one, where $\Delta = \frac{t}{2^b}$. Note that Δ ties the bit-width and threshold values into a single parameter but Δ is not uniquely defined by them. The definition of the quantizer that we use in this work is similar to the one in [24]. The signed version Q^s of a quantizer of an HMQ is defined as follows:

$$Q^s(x, \Delta, t) = \text{clip} \left(\Delta \cdot \left\lceil \frac{x}{\Delta} \right\rceil, -(t - \Delta), t \right) \quad (2)$$

where $\text{clip}(x, a, b) = \min(\max(x, a), b)$ and $\lceil x \rceil$ is the rounding function. Similarly, the unsigned version Q^{us} is defined as follows:

$$Q^{\text{us}}(x, \Delta, t) = \text{clip} \left(\Delta \cdot \left\lceil \frac{x}{\Delta} \right\rceil, 0, t - \Delta \right). \quad (3)$$

In the rest of this section we assume that the quantizer Q of an HMQ is signed, but it applies to both signed and unsigned quantizers.

In order to search over a discrete set, the HMQ represents each pair in $T \times B$ as a sample of a categorical random variable of the Gumbel-Softmax estimator (see [25, 32]). This enables the HMQ to search for a pair of threshold and bit-width. The Gumbel-Softmax is a continuous distribution on the simplex that approximates categorical samples. In our case, we use this approximation as a joint discrete probability distribution of thresholds and bit-widths $P_{T,B}(T=t, B=b|g_{t,b})$ on $T \times B$:

$$P_{T,B}(T=t, B=b|g_{t,b}) = \frac{\exp(\frac{\log(\hat{\pi}_{t,b}) + g_{t,b}}{\tau})}{\sum_{t' \in T} \sum_{b' \in B} \exp(\frac{\log(\hat{\pi}_{t',b'}) + g_{t',b'}}{\tau})} \quad (4)$$

- [20] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1314–1324, 2019.
- [21] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- [22] J. Hu, L. Shen, and G. Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7132–7141, 2018.
- [23] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.
- [24] S. R. Jain, A. Gural, M. Wu, and C. Dick. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066*, 2019.
- [25] E. Jang, S. Gu, and B. Poole. Categorical reparametrization with gumble-softmax. In *International Conference on Learning Representations (ICLR 2017)*. OpenReview. net, 2017.
- [26] A. Krizhevsky, G. Hinton, et al. Learning multiple layers of features from tiny images. Technical report, Citeseer, 2009.
- [27] H. Liu, K. Simonyan, and Y. Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations*, 2019.
- [28] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han. On the variance of the adaptive learning rate and beyond. In *International Conference on Learning Representations*, 2020.
- [29] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. Ssd: Single shot multibox detector. In *European conference on computer vision*, pages 21–37. Springer, 2016.
- [30] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang. Learning efficient convolutional networks through network slimming. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2736–2744, 2017.
- [31] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015.

QKD: Quantization-aware Knowledge Distillation

Jangho Kim^{*†}
Seoul National University
kjh91@snu.ac.kr

Yash Bhalgat^{*}
Qualcomm AI Research[‡]
Qualcomm Technologies, Inc.
ybhalgat@qti.qualcomm.com

Jinwon Lee
Qualcomm AI Research[‡]
Qualcomm Technologies, Inc.
jinwonl@qti.qualcomm.com

Chirag Patel
Qualcomm AI Research[‡]
Qualcomm Technologies, Inc.
cpatel@qti.qualcomm.com

Nojun Kwak[§]
Seoul National University
nojunk@snu.ac.kr

Abstract

Quantization and Knowledge distillation (KD) methods are widely used to reduce memory and power consumption of deep neural networks (DNNs), especially for resource-constrained edge devices. Although their combination is quite promising to meet these requirements, it may not work as desired. It is mainly because the regularization effect of KD further diminishes the already reduced representation power of a quantized model. To address this shortcoming, we propose Quantization-aware Knowledge Distillation (QKD) wherein quantization and KD are carefully coordinated in three phases. First, Self-studying (SS) phase fine-tunes a quantized low-precision student network without KD to obtain a good initialization. Second, Co-studying (CS) phase tries to train a teacher to make it more quantization-friendly and powerful than a fixed teacher. Finally, Tutoring (TU) phase transfers knowledge from the trained teacher to the student. We extensively evaluate our method on ImageNet and CIFAR-10/100 datasets and show an ablation study on networks with both standard and depthwise-separable convolutions. The proposed QKD outperformed existing state-of-the-art methods (e.g., 1.3% improvement on ResNet-18 with W4A4, 2.6% on MobileNetV2 with W4A4). Additionally, QKD could recover the full-precision accuracy at as low as W3A3 quantization on ResNet and W6A6 quantization on MobilenetV2. (See Fig. 1)

^{*}Equal contribution

[†]Work done while interning at Qualcomm AI Research.

[‡]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

[§]Currently a Visiting Researcher at Qualcomm Technologies, Inc.

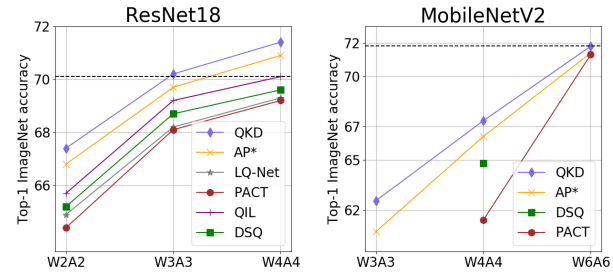


Figure 1: Top-1 accuracy on ImageNet with ResNet-18 and MobileNetV2. Our QKD is compared with various methods such as AP* [30], LQ-Net [50], PACT [6], QIL [18] and DSQ [10]. Dotted line indicates the full-Precision accuracy. More details are in Table 3 and 4.

1. Introduction

Deploying complex DNNs on resource-constrained edge devices such as smartphones or IoT devices is still challenging due to their tight memory and computation requirements. To address this, several research works have been proposed which can be roughly categorized into three folds: weight pruning [12, 11, 27, 9], quantization, [23, 50, 26, 47, 5] and knowledge distillation [15, 20, 48].

Today, a majority of edge devices require fixed-point inference for compute- and power-efficiency. Hence, quantizing the weights and activations of deep networks to a certain bit-width becomes a necessity to make them compatible to edge devices. Recently, hardware accelerators like NPUs (Neural Processing Units), NVIDIA’s Tensor Core units and even CIM (Compute-in-memory) devices have targeted support for sub-byte level processing such as 4-bit $\times$ 4-bit or 2-bit $\times$ 2-bit matrix multiply-and-accumulate operations

[28, 33, 40, 42]. These are much more power- and compute-efficient than conventional 8-bit processing. Thus, there are increased demands for lower-bit quantization.

The accuracy of very low-bit quantized networks such as using 4-, 3- or 2-bits inevitably decreases because of their low representation power compared to typical 8-bit quantization. Some works [30, 34] used knowledge distillation (KD) to improve the accuracy of quantized networks by transferring knowledge from a full-precision teacher network to a quantized student network. However, even though KD is one of the widely used techniques to enhance a DNN's accuracy [15], directly applying KD to very low-bit quantized networks incurs several challenges:

1. Due to its limited representation power, quantized networks generally show lower training and test accuracies compared to the networks with full precision. On the other hand, because KD uses not only the ground truth labels but also the teacher's estimated class probability distributions, it acts as a heavy regularizer [15]. Combining these contradictory characteristics of quantization and KD could sometimes result in further degradation of performance from that of quantization-only.
2. In general, it has been shown that a powerful teacher with high accuracy can teach a student better than a weaker one [54]. Also, [20, 29] show that if there is a large gap from the capacity or inherent differences between the teacher and student, it can hinder the knowledge transfer because this knowledge is unadaptable to the student. Previous works of quantization with KD [30, 34] directly applied conventional KD to quantization along with fixed teacher network, and thus they suffer from above mentioned limitations.

In this paper, we propose Quantization-aware Knowledge Distillation (QKD) to address the above mentioned challenges, especially for very low-precision quantization. QKD consists of three phases. In the first '**self-studying**' phase, instead of directly applying knowledge distillation to the quantized student network from the beginning, we first try to find a good initialization. In the second '**co-studying**' phase, we improve the teacher by using the knowledge of the student network. This phase makes the teacher more powerful and quantization-friendly (See Section 4.5), which is essential to improve accuracy. In final '**tutoring**' phase, we freeze the teacher network and transfer its knowledge to the student. This phase saves unnecessary training time and memory of the teacher network which tends to have already saturated in the co-studying phase. The overall process of QKD is depicted in the Figure 2. Our key contributions are the following:

- We propose Quantization-aware Knowledge Distillation which can be effectively applied to very low-bit

(2-,3-,4-bit) quantized networks. Considering the characteristics of low bit networks and distillation methods, we design a combination of three phases for QKD that overcome the shortcomings posed by conventional quantization + KD methods as described above.

- We empirically verify that QKD works well on depth-wise convolution networks which are known to be difficult to quantize and our work is the first to apply KD to train low-bit depth-wise convolution networks (viz. MobileNetV2 and EfficientNet)
- We show that our QKD obtains state-of-the-art accuracies on CIFAR and ImageNet datasets compared to other existing methods. (See Figure 1.)

2. Related work

Quantization Reducing the precision of neural networks [7, 25, 53, 32, 43] has been studied extensively due to its computational and storage benefits. Although binary $\{-1, +1\}$ [7] or ternary $\{-1, 0, +1\}$ [25] quantizations are typically used, their methods only consider the quantization of weights. To fully utilize bit-wise operations, activation maps also should be quantized in the same way as weights. Some researches consider quantizing both weights and activation maps [16, 41, 35, 52, 46]. Binary neural networks [16] quantize both weights and activation maps as binary and compute gradients with binary values. XNOR-Net [35] also quantizes its weights and activation maps with scaling factors obtained by constrained optimization. Furthermore, HAQ [46] adaptively changes bit-width per layer by leveraging reinforcement learning and HW-awareness.

Recent works have tried to improve quantization further by learning the range of weights and activation maps [6, 18, 8, 45]. These approaches can easily outperform previous methods which do not train quantization-related parameters. PACT [6] proposes a clipping activation function using trainable a parameter which limits the maximum value of activation. LSQ [8] and TQT [17] introduce uniform quantization using trainable interval values. QIL [18] uses a trainable quantizer which performs both pruning and clipping. Our QKD leverages these trainable approaches in the baseline quantization implementation. On top of this quantization-only method, our elaborated knowledge distillation process boosts the accuracy, thereby achieving the state of art accuracy on low-bit quantization.

Knowledge Distillation KD is one of the most popular methods in model compression. It is widely used in many computer vision tasks. This framework transfers the knowledge of a teacher network to a smaller-sized student network in two ways: Offline and Online. First, offline KD uses a fixed pre-trained teacher networks and the knowledge transfer can happen in different ways. In [15], they

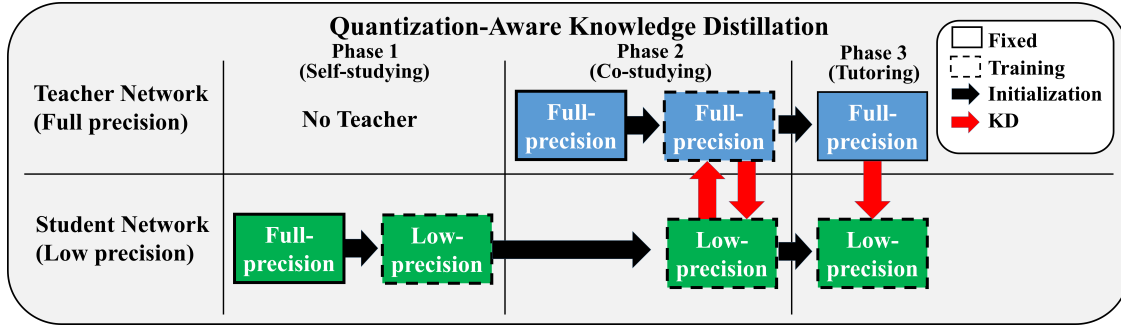


Figure 2: The overall process of QKD. Self-studying (SS) phase give a good starting point to alleviate low representative power and regularization effect of KD. Co-studying (CS) phase makes a teacher adaptable to a student and powerful than the fixed teacher. In tutoring (TU) phase, the teacher transfers its adaptable and powerful knowledge to the student.

encourage the student network to mimic the softened distribution of the teacher network. Other works [49, 14, 20, 37] transfer the information using different forms of activation maps from the teacher network. Second, online KD methods [54, 19, 51] train both the teacher and the student networks simultaneously without pre-trained teacher models. Deep Mutual Learning (DML) [51] have tried to transfer each knowledge of independent networks using KL loss, although it is not studied in the context of quantization. Our QKD uses both online and offline KD methods sequentially.

Quantization + Knowledge distillation Some researches have tried to use distillation methods to train low precision network [30, 34]. They use a full precision network as the teacher and the low bit network as a student. In Apprentice (AP) [30], the teacher and student networks are initialized with the corresponding pre-trained full precision networks and the student is then fine-tuned using distillation. Due to AP’s initialization of the student, AP tends to get stuck in a local minimum in the case of very low-bit quantized student networks. The self-study phase of QKD directly mitigates this issue. Also, using a fixed teacher, as in [30, 34], can limit the knowledge transfer due to the inherent differences between the distributions of the full-precision teacher and low-precision student network. We tackle this problem via online co-studying (CS) and offline tutoring (TU).

3. Proposed Method

In this section, we first explain the quantization method used in QKD and then describe the proposed QKD method.

3.1. Quantization

In QKD, we use a trainable uniform quantization scheme for our baseline quantization implementation. We choose uniform quantization because of its hardware-friendly characteristics. In this work, we quantize both weights and activations. So, we introduce two trainable parameters for the interval values of each layer’s weight parameters and in-

put activations similar to [8, 17, 45]. These parameters can be trained with either the task loss or a combination of the task and distillation losses. Considering k -bit quantization for weights and activations, the weight and activation quantizers are defined as follows.

Weight Quantizer: Since weights can take positive as well as negative values, we quantize each weight to integers in the range $[-2^{k-1}, 2^{k-1} - 1]$. Before rounding, the weights are first constrained to this range using a clamping function $F_W(w) \triangleq F(w, -2^{k-1}, 2^{k-1} - 1)$ where

$$F(x, \min, \max) = \begin{cases} \max & \text{if } x > \max \\ \min & \text{if } x < \min \\ x & \text{otherwise.} \end{cases} \quad (1)$$

We use a trainable parameter I_W for the interval value of weight quantization. It is trained along with the weights of the network. We can calculate the quantization level of the input weight w using a rounding operation on F_W . The overall quantization-dequantization scheme for the weights of the network is defined as follows:

$$\hat{w} = Q_W(w) = \left\lfloor F_W\left(\frac{w}{I_W}\right) + \frac{1}{2} \right\rfloor \times I_W \quad (2)$$

where $\lfloor \cdot \rfloor$ is the flooring operation. Note that the *dequantization* step (multiplication by I_W) just brings the quantized values ($\hat{w}$) back to their original range and is commonly used when emulating the effect of quantization [3, 17, 36].

Activation Quantizer: Since most of the networks today use ReLU as the activation function, the activation values are unsigned¹. Hence, to quantize the activations, we use an quantization function with the range $[0, 2^k - 1]$. The activation quantizer Q_X can be obtained as

$$\hat{x} = Q_X(x) = \left\lfloor F_X\left(\frac{x}{I_X}\right) + \frac{1}{2} \right\rfloor \times I_X \quad (3)$$

¹EfficientNet uses Swish extensively which introduces a small proportion of negative activation values, but we can save one bit per activation by clamping these negative values at 0 using unsigned quantization

Table 3: ImageNet performance comparison on architectures with standard convolutions

Bitwidth	Method	ResNet18		ResNet34		ResNet50	
		top-1	top-5	top-1	top-5	top-1	top-5
W32A32	-	70.1	89.5	73.5	91.7	76.3	93.0
W4A4	PACT [6]	69.2	89.0	-	-	76.5	93.2
	LQ-Nets [50]	69.3	88.8	-	-	75.1	92.4
	DSQ [10]	69.6	-	72.8	-	-	-
	QIL [18]	70.1	-	73.7	-	-	-
	BL	70.7	89.4	73.8	91.5	76.6	93.2
	SS + BL	70.8	89.4	74.9	91.5	76.6	93.2
	AP*	70.9	89.5	74.0	91.7	76.7	93.2
	SS + AP*	71.1	89.6	74.2	91.8	76.9	93.3
	CS + TU	71.1	90.1	74.4	91.9	77.0	93.5
	QKD	71.4	90.3	74.6	92.1	77.3	93.6
W3A3	PACT [6]	68.1	88.2	-	-	75.3	92.6
	LQ-Nets [50]	68.2	87.9	71.9	90.2	74.2	91.6
	DSQ [10]	68.7	-	72.5	-	-	-
	QIL [18]	69.2	-	73.1	-	-	-
	BL	69.4	88.5	73.3	90.5	75.3	92.6
	SS + BL	69.5	88.5	73.3	90.6	75.4	92.7
	AP*	69.7	88.7	73.5	90.7	76.0	92.7
	SS + AP*	69.9	88.7	73.7	90.8	76.1	92.8
	CS + TU	70.0	89.4	73.7	91.2	76.2	92.9
	QKD	70.2	89.9	73.9	91.4	76.4	93.2
W2A2	PACT [6]	64.4	85.6	-	-	72.2	90.5
	LQ-Nets [50]	64.9	85.9	69.8	89.1	71.5	90.3
	DSQ [10]	65.2	-	70.0	-	-	-
	QIL [18]	65.7	-	70.6	-	-	-
	BL	66.7	86.7	71.0	89.8	73.3	91.0
	SS + BL	66.8	86.9	71.0	89.9	73.3	91.0
	AP*	66.8	86.9	71.1	89.9	73.4	91.0
	SS + AP*	67.2	87.3	71.3	90.2	73.7	91.2
	CS + TU	67.1	87.2	71.3	90.1	73.6	91.2
	QKD	67.4	87.5	71.6	90.3	73.9	91.6

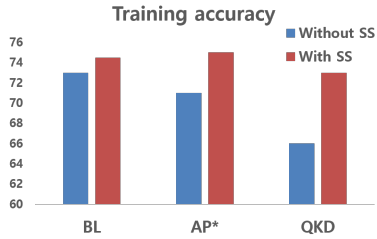


Figure 3: Training accuracy of student network using BL, AP* and QKD at W2A2 on CIFAR-100. Red and blue represent the accuracy with and without Self-studying (SS)

With W2A2, we observe a drastic drop in top-1 accuracy. This was expected since depth-wise convolution layers are known to be highly sensitive to quantization [46, 17]. So, we ran another set of experiments where the weights and input activations of the depth-wise separable layers are quantized to W8A8 whereas the rest of the layers are quantized to W2A2, W3A3 or W4A4. We include these results in the supplementary details.

Table 4: ImageNet performance comparison on architectures with depthwise separable convolutions

Bitwidth	Method	MobileNetV2		EfficientNet-B0	
		top-1	top-5	top-1	top-5
W32A32	-	71.8	90.3	76.3	93.2
W6A6	PACT [6]	71.3	90.0	-	-
	BL	71.2	89.9	74.5	92.3
	SS + BL	71.3	90.0	74.6	92.3
	AP*	71.4	90.0	74.8	92.4
	SS + AP*	71.4	90.0	75.0	92.4
	CS + TU	71.5	90.1	75.1	92.6
	QKD	71.8	90.2	75.4	92.6
W4A4	PACT [6]	61.4	83.7	-	-
	DSQ [10]	64.8	-	-	-
	BL	66.1	86.2	71.9	90.4
	SS + BL	66.3	86.3	72.0	90.4
	AP*	66.4	86.4	72.3	90.6
	SS + AP*	66.6	86.5	72.6	90.7
	CS + TU	66.7	86.7	72.8	91.0
	QKD	67.4	87.0	73.1	91.2
W3A3	BL	60.1	83.0	67.5	85.8
	SS + BL	61.5	83.4	67.7	85.9
	AP*	60.7	83.2	68.4	86.1
	SS + AP*	62.0	83.5	68.7	86.2
	CS + TU	62.0	83.5	68.8	86.4
	QKD	62.6	84.0	69.2	86.9
W2A2	BL	37.7	64.1	43.5	67.5
	SS+BL	42.7	66.2	44.9	67.6
	AP*	39.8	65.3	46.4	68.2
	SS+AP*	43.9	67.3	47.7	68.5
	CS + TU	43.6	66.9	48.0	68.7
	QKD	45.7	68.1	50.0	69.3

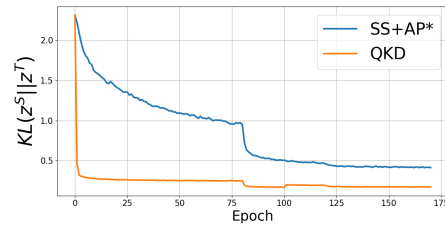


Figure 4: KL-divergence between teacher & student distributions on CIFAR100 with QKD and SS+AP* (SS part is not shown in both because it doesn't involve the teacher network).

4.5. Discussion

Effectiveness of Self-studying Figure 3 shows the training accuracy of the student in the final epoch with and without the Self-Studying phase on the CIFAR-100 dataset. At very low-bit quantization, the model has low representative power and without SS, it gets stuck in a local minima as is reflected from the lower training accuracy. The SS phase helps the student start from a better initialization point and KD then guides it to a better local minima, hence increasing the training accuracy. Note that although training accuracy

each of these methods on the quantized model accuracy. We show how self-studying is important in alleviating the regularization effect imposed by KD and how co-studying enhances the quantization performance by making the teacher more adaptable and more powerful in terms of accuracy. Overall, with an extensive set of experiments, we show that QKD gives significant performance boost over our baseline quantization-only method and outperforms the existing state-of-the-art approaches. We demonstrate QKD's results on networks with both standard and depthwise separable convolutions and show that we can recover the full-precision accuracy at as low as W3A3 quantization of the ResNet architectures and W6A6 quantization of MobileNetV2.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org. 4
- [2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013. 4
- [3] Aishwarya Bhandare, Vamsi Sripathi, Deepthi Karkada, Vivek Menon, Sun Choi, Kushal Datta, and Vikram Sale-tore. Efficient 8-bit quantization of transformer neural machine language translation model. *arXiv preprint arXiv:1906.00532*, 2019. 3
- [4] Léon Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010*, pages 177–186. Springer, 2010. 5
- [5] Zhaowei Cai, Xiaodong He, Jian Sun, and Nuno Vasconcelos. Deep learning with low precision by half-wave gaussian quantization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5918–5926, 2017. 1
- [6] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018. 1, 2, 6, 7
- [7] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. Binaryconnect: Training deep neural networks with binary weights during propagations. In *Advances in neural information processing systems*, pages 3123–3131, 2015. 2
- [8] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization, 2019. 2, 3
- [9] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018. 1
- [10] Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. Differentiable soft quantization: Bridging full-precision and low-bit neural networks, 2019. 1, 6, 7
- [11] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015. 1
- [12] Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. In *Advances in neural information processing systems*, pages 1135–1143, 2015. 1
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 5, 6
- [14] Byeongho Heo, Jeesoo Kim, Sangdoo Yun, Hyojin Park, Nojun Kwak, and Jin Young Choi. A comprehensive overhaul of feature distillation. *arXiv preprint arXiv:1904.01866*, 2019. 3, 8
- [15] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 1, 2
- [16] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. In *Advances in neural information processing systems*, pages 4107–4115, 2016. 2
- [17] Sambhav R Jain, Albert Gural, Michael Wu, and Chris H Dick. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066*, 2019. 2, 3, 7
- [18] Sangil Jung, Changyong Son, Seohyung Lee, Jinwoo Son, Jae-Joon Han, Youngjun Kwak, Sung Ju Hwang, and Changkyu Choi. Learning to quantize deep networks by optimizing quantization intervals with task loss. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4350–4359, 2019. 1, 2, 5, 6, 7
- [19] Jangho Kim, Minsung Hyun, Inseop Chung, and Nojun Kwak. Feature fusion for online mutual knowledge distillation. *arXiv preprint arXiv:1904.09058*, 2019. 3
- [20] Jangho Kim, SeongUk Park, and Nojun Kwak. Paraphrasing complex network: Network compression via factor transfer. In *Advances in Neural Information Processing Systems*, pages 2760–2769, 2018. 1, 2, 3, 8
- [21] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 5
- [22] Guillaume Klein, Yoon Kim, Yuntian Deng, Jean Senellart, and Alexander M. Rush. OpenNMT: Open-source toolkit for neural machine translation. In *Proc. ACL*, 2017. 5

Differentiable Model Compression via Pseudo Quantization Noise

Alexandre Défossez*

Meta AI, FAIR Team, Paris, France

defossez@fb.com

Yossi Adi*

Meta AI, FAIR Team, Tel-Aviv, Israel

adiyoss@fb.com

Gabriel Synnaeve

Meta AI, FAIR Team, Paris, France

gab@fb.com

Reviewed on OpenReview: <https://openreview.net/forum?id=DijnKziche>

Abstract

We propose DIFFQ a differentiable method for model compression for quantizing model parameters without gradient approximations (e.g., *Straight Through Estimator*). We suggest adding independent pseudo quantization noise to model parameters during training to approximate the effect of a quantization operator. DIFFQ is differentiable both with respect to the unquantized weights and the number of bits used. Given a single hyper-parameter balancing between the quantized model size and accuracy, DIFFQ optimizes the number of bits used per individual weight or groups of weights, in end-to-end training. We experimentally verify that our method is competitive with STE based quantization techniques on several benchmarks and architectures for image classification, language modeling, and audio source separation. For instance, on the ImageNet dataset, DIFFQ compresses a 12 layers transformer-based model by more than a factor of 8, (lower than 4 bits precision per weight on average), with a loss of 0.3% in model accuracy. Code is available at github.com/facebookresearch/diffq.

1 Introduction

An important factor in the adoption of a deep learning model for real-world applications is how easily it can be pushed to remote devices. It has been observed that larger models usually lead to better performance, for instance with larger ResNets (He et al., 2016) achieving higher accuracies than smaller ones. In response, the community has worked toward smaller, and more efficient models (Tan & Le, 2019). Yet an EfficientNet-B3 is still almost 50MB, a considerable amount if the model is to be included in online applications, or updated with limited network capabilities. For other applications, such as language modeling (Vaswani et al., 2017) or source separation (Défossez et al., 2019), the typical model size is closer to 1GB, ruling out any kind of mobile usage. Efficient model compression is thus important for on device adoption of deep learning models. Thus, we focus in the present work on reducing model size, rather than achieving computational gains.

The simplest method to reduce model size consists in decreasing the number of bits used to encode individual weights. For instance, using 16 bits floating point numbers halves the model size, while retaining a sufficient approximation of the set of real numbers, $\mathbb{R}$, to train with first-order optimization methods (Micikevicius et al., 2018). When considering lower precision, for instance, 8 or 4 bits, the set of possible values is no longer a good approximation of $\mathbb{R}$, hence preventing the use of first-order optimization methods. Specifically, uniform quantization requires using the `round` function, which has zero gradients wherever it is differentiable.

Quantization can be done as a post-processing step to regular training. However, errors accumulate in a multiplicative fashion across layers, with a possibly uncontrolled decrease in the model accuracy. Courbariaux et al.

*Equal contribution.

performance than DIFFQ. Additive noise was also studied in the context of image compression (Ballé et al., 2017; Choi et al., 2019) in order to provide a differentiable pseudo-quantization operator. However, those work rely on an explicit estimation of the quantized values entropy, in particular with respect to a distribution of images. This formalism breaks down when having to quantize a single model, not a distribution, and DIFFQ uses a simpler approach where the bitwidth is directly tuned. More recently, Park et al. (2022) extended our method for activation quantization.

An important contribution from DIFFQ is the automatic tuning of the bitwidth using mixed-precision. Other mixed-precision quantization methods are based on Reinforcement Learning (Wang et al., 2019; Elthakeb et al., 2020; Liu et al., 2021), second-order optimization (Dong et al., 2019; 2020; Yao et al., 2021), and differentiable quantization methods (Uhlich et al., 2020; Wang et al., 2020). Comparing to DIFFQ, such methods are more complex (e.g., require plenty of parameter tuning), more computationally heavy, and most importantly based on STE approximations. Wang et al. (2019); Elthakeb et al. (2019) suggested learning a bitwidth assignment policy using reinforcement learning methods. In contrast, our method select bitwidth along training, using only first order optimization. Jain et al. (2019); Esser et al. (2020), and Bhalgat et al. (2020) proposed learning the quantizer step-size or dynamic-range using STE, but do not allow to select the bitdwidth. Our experiments show that DIFFQ outperforms (Esser et al., 2020) (LSQ) both on most vision and natural language tasks. Uhlich et al. (2020) proposed a re-parametrization that allows to select the bitwidth for each layer through first order optimization, while also relying on STE. The re-parametrization is more complex than the additive noise used in DIFFQ, and suffers from the biased gradient of STE. Results suggest that DIFFQ achieves similar or better trade-offs between model size and accuracy. Besides, in the present work we explore setting a bitwidth for individual groups of weights within each layer, rather than layer-wise.

The limitations of STE methods for quantization were first noticed by Liu & Mattina (2019). They recommend using a linear combination of the unquantized and quantized weight, with the gradient flowing only through the unquantized contribution. In a similar spirit, Fan et al. (2021) sample for each layer and iteration whether to use the quantized or unquantized weight. Both methods reduce the bias from STE, but also remove some of the quantization noise during training. In contrast our method allows to keep a full pseudo quantization noise without the STE bias. Liu et al. (2022) proposed the *Generalized STE* method to deal with gradient instabilities by calculating the expectation of the stochastic quantization during the backward phase. Finally, Nagel et al. (2022) extend the analysis we present in Section 3.3 on the oscillations of weights when using STE and suggest tracking the weight oscillations in order to freeze them when needed, as an ad-hoc solution.

A last line of related work is Product Quantization (PQ) Stock et al. (2019), where code words are being learned to quantize blocks of weights rather than single weights. This method achieves a higher compression level than per-weight quantization but also requires carefully choosing the size of the codebooks for each layer. In contrast, our method requires only choosing a single hyper-parameter to balance between model size and accuracy. Besides, as noted by Fan et al. (2021), per-weight quantization and PQ can be combined. We compare with PQ on vision and language tasks: while PQ can reach smaller model size than DIFFQ, it can also suffer from unacceptable accuracy loss, in particular for language modeling.

3 Background

Let us consider a weight vector $w \in \mathbb{R}^d$, where $d \in \mathbb{N}$, typically the weights of convolution or linear layer. Each entry of the vector is typically coded over 32 bits with floating-point precision. We aim to reduce the number of possible states to 2^B , where $B \ll 32$ is the number of bits of precision. First, we assume $w_i \in [0, 1]$ for all $1 \leq i \leq d$. In practice, one would first normalize w as

$$\hat{w} = \frac{w - \min(w)}{\max(w) - \min(w)},$$

and provide the tuple $(\min(w), \max(w))$ separately as a 32 bits IEEE float. Given that for typical deep learning models $d \gg 1$, storing this range has a negligible cost. For readability, we describe the method for scalar values $w \in [0, 1]$, however, this can be easily extended to vectors $w \in \mathbb{R}^d$.

- Alexandre Défossez, Nicolas Usunier, Léon Bottou, and Francis Bach. Music source separation in the waveform domain. *arXiv preprint arXiv:1911.13254*, 2019.
- J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR*, 2009.
- Zhen Dong, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Hawq: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 293–302, 2019.
- Zhen Dong, Zhewei Yao, Daiyaan Arfeen, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Hawq-v2: Hessian aware trace-weighted quantization of neural networks. *Advances in neural information processing systems*, 33:18518–18529, 2020.
- Ahmed Elthakeb, Prannoy Pilligundla, FatemehSadat Miresghallah, Amir Yazdanbakhsh, Sicuan Gao, and Hadi Esmaeilzadeh. Releq: An automatic reinforcement learning approach for deep quantization of neural networks. In *NeurIPS ML for Systems workshop, 2018*, 2019.
- Ahmed T Elthakeb, Prannoy Pilligundla, Fatemehsadat Miresghallah, Amir Yazdanbakhsh, and Hadi Esmaeilzadeh. Releq: A reinforcement learning approach for automatic deep quantization of neural networks. *IEEE micro*, 40(5):37–45, 2020.
- Steven K Esser, Jeffrey L McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S Modha. Learned step size quantization. In *Proc. of the International Conference on Learning Representations*, 2020.
- Angela Fan, Edouard Grave, and Armand Joulin. Reducing transformer depth on demand with structured dropout. In *Proc. of the International Conference on Learning Representations*, 2019.
- Angela Fan, Pierre Stock, Benjamin Graham, Edouard Grave, Remi Gribonval, Herve Jegou, and Armand Joulin. Training with quantization noise for extreme model compression. In *ICLR 2021*, 2021.
- Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. Differentiable soft quantization: Bridging full-precision and low-bit neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*, 2019.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.
- Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9): 1098–1101, 1952. doi: 10.1109/JRPROC.1952.273898.
- Yerlan Idelbayev. Proper ResNet implementation for CIFAR10/CIFAR100 in PyTorch. https://github.com/akamaster/pytorch_resnet_cifar10, 2018.
- Sambhav R Jain, Albert Gural, Michael Wu, and Chris H Dick. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066*, 2019.
- Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *Proc. of the International Conference on Learning Representations*, 2017.
- Sangil Jung, Changyong Son, Seohyung Lee, Jinwoo Son, Jae-Joon Han, Youngjun Kwak, Sung Ju Hwang, and Changkyu Choi. Learning to quantize deep networks by optimizing quantization intervals with task loss. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4350–4359, 2019.

ANALOGNETS: ML-HW CO-DESIGN OF NOISE-ROBUST TINYML MODELS AND ALWAYS-ON ANALOG COMPUTE-IN-MEMORY ACCELERATOR

Chuteng Zhou¹ Fernando Garcia Redondo² Julian Büchel^{3,4} Irem Boybat³ Xavier Timoneda Comas³
S. R. Nandakumar³ Shidhartha Das² Abu Sebastian³ Manuel Le Gallo^{*3} Paul N. Whatmough^{*1}

ABSTRACT

Always-on TinyML perception tasks in IoT applications require very high energy efficiency. Analog compute-in-memory (CiM) using non-volatile memory (NVM) promises high efficiency and also provides self-contained on-chip model storage. However, analog CiM introduces new practical considerations, including conductance drift, read/write noise, fixed analog-to-digital (ADC) converter gain, etc. These additional constraints must be addressed to achieve models that can be deployed on analog CiM with acceptable accuracy loss. This work describes *AnalogNets*: TinyML models for the popular always-on applications of keyword spotting (KWS) and visual wake words (VWW). The model architectures are specifically designed for analog CiM, and we detail a comprehensive training methodology, to retain accuracy in the face of analog non-idealities, and low-precision data converters at inference time. We also describe AON-CiM, a programmable, minimal-area phase-change memory (PCM) analog CiM accelerator, with a novel layer-serial approach to remove the cost of complex interconnects associated with a fully-pipelined design. We evaluate the AnalogNets on a calibrated simulator, as well as real hardware, and find that accuracy degradation is limited to 0.8%/1.2% after 24 hours of PCM drift (8-bit) for KWS/VWW. AnalogNets running on the 14nm AON-CiM accelerator demonstrate 8.58/4.37 TOPS/W for KWS/VWW workloads using 8-bit activations, respectively, and increasing to 57.39/25.69 TOPS/W with 4-bit activations.

1 INTRODUCTION

The origins of deep learning were with high-performance GPU hardware consuming hundreds of Watts of power and costing thousands of dollars. However, there is currently great interest in developing networks that can be deployed for inference tasks on constrained devices that can run autonomously on a few milliwatts of power and cost a few dollars, which is often referred to as *TinyML*. The applications for TinyML are numerous and varied, but including things like smart hearing aids (Fedorov et al., 2020), machinery maintenance monitoring (Banbury et al., 2021), human activity recognition (Kodali et al., 2017) and so on. In this paper we focus on two popular always-on perception tasks: visual wake words (VWW) (Chowdhery et al., 2019) and key-word spotting (KWS) (Warden, 2018), which turn on the more sophisticated ML functions when a person is in the camera frame or when a keyword is heard (Figure 1).

In many TinyML scenarios, the limitations of a constrained hardware platform introduce very significant challenges for

^{*}Equal contribution ¹Arm Research, Boston, US ²Arm Research, Cambridge, UK ³IBM Research, Zürich, Switzerland ⁴ETH Zürich, Zürich, Switzerland. Correspondence to: Chuteng Zhou <chu.zhou@arm.com>.

Preprint, under review.

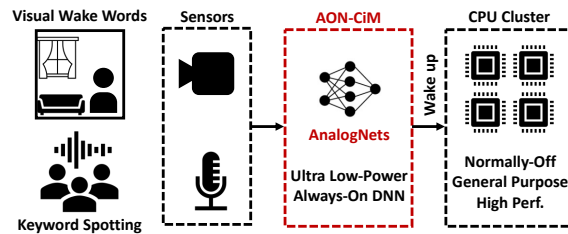


Figure 1. Overview of always-on (AON) TinyML AnalogNets DNNs on self-contained compute-in-memory (CiM) hardware.

DNN deployment. Typical microcontroller (MCU) hardware platforms are cheap ($\sim \$1$) and low power (~ 10 s mW) (Fedorov et al., 2019). However, they often have very limited persistent storage (~ 1 MB), memory (~ 100 s KB) and compute throughput (~ 100 s MOPS) (Banbury et al., 2021). This limits the design of the DNN model in terms of: 1) number of parameters, 2) activation footprint, and 3) number of operations (ops) per inference. These three factors will largely determine the accuracy, latency and energy per inference that can be achieved by a given model.

The high compute and memory demands of DNN inference workloads has recently driven significant research interest in analog compute-in-memory (CiM) hardware platforms (Zidan et al., 2018; Hamdioui et al., 2019; Li et al., 2019; Sebastian et al., 2020). The basic idea is to perform computation

an ML training abstraction, we model the DACs and ADCs with simple effective number of bits (ENOB) quantizers. Quantization, whether for model weights or activations, is becoming ubiquitous in ML model training and there are many tools available. In the ML training graph (Figure 4), we model DACs and ADCs as separate quantization nodes. The input activation tensor is quantized by the DAC coming into the operator and the output pre-activation tensor is quantized by the ADC. Hardware constraints dictate that the ranges of these quantizers are symmetric, and we set the ENOB of the DAC to be 1-bit more than the ADC to accommodate the positivity of input activations after a ReLU activation function:

$$b_{\text{DAC}} = b_{\text{ADC}} + 1. \quad (3)$$

Setting the ranges for these quantizers, or equivalently choosing the scaling/gain factors for DAC/ADCs, is not easy especially because of the varying dynamic ranges between the signals on different filters and layers for a deep neural network with many layers. Furthermore, for low-precision DAC/ADCs, sub-optimal choices of these quantities leads to accuracy degradation. To address this problem, we adopt a fully differentiable approach to learn the ranges using gradient descent. Following a similar approach to Jain et al. (2019), the quantization function can be written as

$$q(x; b) = \text{round}_{\text{STE}} \left(\frac{\text{clip}(x; -r_{\text{max}}, r_{\text{max}})}{r_{\text{max}} / (2^{b-1} - 1)} \right). \quad (4)$$

The rounding operation rounds the number to its nearest integer and its gradient is calculated using the straight-through estimator (STE) (Bengio et al., 2013). This quantization function is differentiable with respect to both x and r_{max} .

Figure 4 also describes the ADC gain constraint discussed in Section 3. The analog signal coming from the bitlines in the CiM array need to be scaled in the analog domain before being converted by the ADC. This analog scaling represents the ADC gain and is set at calibration time, and therefore it should be considered a constant across different layers. The scaling factor is related to the ranges of quantizers and maximum weight values to make sure that the ranges of ADCs and DACs are fully utilized. This constraint can be expressed as

$$S \equiv \frac{r_{\text{DAC},l} W_{l,\text{max}}}{r_{\text{ADC},l}}, \forall l. \quad (5)$$

$r_{\text{DAC},l}$ and $r_{\text{ADC},l}$ are respectively the maximum ranges of quantizers representing the DAC and the ADC in layer l . $W_{l,\text{max}}$ is related to the noise injected into model weights by Equation 1. To eliminate one moving piece, we fix the value of $W_{l,\text{max}}$ during a two-stage training process.

In the first training stage, we train a model without any quantization nodes or noise injection and only with weight

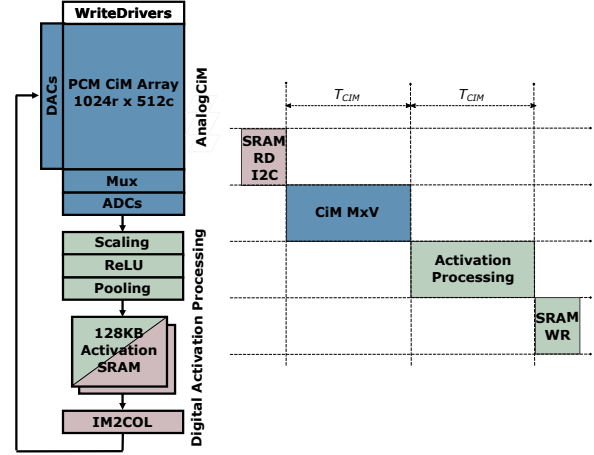


Figure 5. AON-CiM accelerator architecture and operation. Following Khaddam-Aljameh et al. (2021), each ADC quantized value requires a floating point activation/scaling operation pipelined to adjust to the SRAM read/writes and crossbar operation.

clipping in the range $[-2\sigma_{\mathbf{W}_{l,0}}, 2\sigma_{\mathbf{W}_{l,0}}]$, where $\sigma_{\mathbf{W}_{l,0}}$ is the standard deviation of weights calculated with the unclipped weights $\mathbf{W}_{l,0}$. $\sigma_{\mathbf{W}_{l,0}}$ is updated every 10 time steps until the model is trained to convergence.

In the second training stage, we initialize the model with the trained weights from the first stage and compute the clipping ranges as $W_{l,\text{min}} = -2\sigma_{\mathbf{W}_{l,0}}$, $W_{l,\text{max}} = 2\sigma_{\mathbf{W}_{l,0}}$. $W_{l,\text{min}}$ and $W_{l,\text{max}}$ are then fixed throughout the training. We add noise injection and the quantization nodes representing the ADCs and DACs. We treat S and $r_{\text{ADC},l}$ as independent trainable parameters and initialize them at 1. Therefore $r_{\text{DAC},l} = r_{\text{ADC},l} |S| / W_{l,\text{max}}$ and we take the absolute value of S here to make sure that the ranges are all positive since S may become negative during gradient descent. The gradient of S can be computed as

$$\frac{\partial \mathcal{L}}{\partial S} = \sum_l \frac{\partial \mathcal{L}}{\partial r_{\text{DAC},l}} \frac{r_{\text{ADC},l}}{W_{l,\text{max}}} \frac{d|S|}{dS}, \quad (6)$$

where $d|S|/dS$ denotes the subgradient for the absolute value function. The gradients of $r_{\text{ADC},l}$ are also slightly altered since there is now an extra differentiable path through $r_{\text{DAC},l}$. There are no gradients for $W_{l,\text{max}}$ as they are constant. Modern deep learning frameworks support automatic differentiation and can handle these gradient calculations seamlessly. We use TensorFlow to train our models.

5 AON-CiM ACCELERATOR

Always-on (AON) accelerators operate alone while the remaining blocks in an SoC are unused and powered-down. Therefore, an AON accelerator must be self-contained, including all required memory. Below, we describe the fully-programmable AON-CiM accelerator used to evaluate the

- Gonugondla, S. K., Kang, M., and Shanbhag, N. A 42pj/decision 3.12 tops/w robust in-memory machine learning classifier with on-chip training. In *2018 IEEE International Solid-State Circuits Conference-(ISSCC)*, pp. 490–492. IEEE, 2018.
- Guo, R., Liu, Y., Zheng, S., Wu, S.-Y., Ouyang, P., Khwa, W.-S., Chen, X., Chen, J.-J., Li, X., Liu, L., Chang, M.-F., Wei, S., and Yin, S. A 5.1pj/neuron 127.3us/inference rnn-based speech recognition processor using 16 computing-in-memory sram macros in 65nm cmos. In *2019 Symposium on VLSI Circuits*, pp. C120–C121, 2019. doi: 10.23919/VLSIC.2019.8778028.
- Gupta, C., Suggala, A. S., Goyal, A., Simhadri, H. V., Paranjape, B., Kumar, A., Goyal, S., Udupa, R., Varma, M., and Jain, P. Protonn: Compressed and accurate knn for resource-scarce devices. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML'17*, pp. 1331–1340. JMLR.org, 2017.
- Hamdioui, S., Du Nguyen, H. A., Taouil, M., Sebastian, A., Gallo, M. L., Pande, S., Schaafsma, S., Catthoor, F., Das, S., Redondo, F. G., Karunaratne, G., Rahimi, A., and Benini, L. Applications of Computation-In-Memory Architectures based on Memristive Devices. In *2019 Des. Autom. Test Eur. Conf. Exhib.*, pp. 486–491. IEEE, mar 2019. ISBN 978-3-9819263-2-3. doi: 10.23919/DATE.2019.8715020. URL <https://ieeexplore.ieee.org/document/8715020/>.
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Wey, T., Andreetto, M., and Adam, H. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *CoRR*, abs/1704.04861, 2017. URL <http://arxiv.org/abs/1704.04861>.
- Hu, M., Graves, C., Li, C., Li, Y., Ge, N., Montgomery, E., Davila, N., Jiang, H., Williams, R., Yang, J., et al. Memristor-based analog computation and neural network classification with a dot product engine. *Advanced materials (Deerfield Beach, Fla.)*, 30(9), 2018.
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2704–2713, 2018.
- Jain, S., Sengupta, A., Roy, K., and Raghunathan, A. Rx-caffe: Framework for evaluating and training deep neural networks on resistive crossbars. *arXiv preprint arXiv:1809.00072*, 2018.
- Jain, S. R., Gural, A., Wu, M., and Dick, C. H. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066*, 2019.
- Jia, H., Ozatay, M., Tang, Y., Valavi, H., Pathak, R., Lee, J., and Verma, N. A programmable neural-network inference accelerator based on scalable in-memory computing. In *IEEE International Solid-State Circuits Conference (ISSCC)*, volume 64, pp. 236–238, 2021. doi: 10.1109/ISSCC42613.2021.9365788.
- Joshi, V., Le Gallo, M., Haefeli, S., Boybat, I., Nandakumar, S. R., Piveteau, C., Dazzi, M., Rajendran, B., Sebastian, A., and Eleftheriou, E. Accurate deep neural network inference using computational phase-change memory. *Nature Communications*, 11(1):1–13, 2020.
- Khaddam-Aljameh, R., Stanisavljevic, M., Mas, J. F., Karunaratne, G., Braendli, M., Liu, F., Singh, A., Müller, S., Egger, U., Petropoulos, A., et al. Hermes core—a 14nm CMOS and PCM-based in-memory compute core using an array of 300ps/LSB linearized CCO-based ADCs and local digital processing. In *2021 Symposium on VLSI Circuits*, pp. 1–2. IEEE, 2021.
- Kodali, S., Hansen, P., Mulholland, N., Whatmough, P., Brooks, D., and Wei, G. Applications of deep neural networks for ultra low power iot. In *2017 IEEE International Conference on Computer Design (ICCD)*, pp. 589–592, Nov 2017. doi: 10.1109/ICCD.2017.102.
- Kumar, A., Goyal, S., and Varma, M. Resource-efficient machine learning in 2 kb ram for the internet of things. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML'17*, pp. 1935–1944. JMLR.org, 2017.
- Le Gallo, M., Krebs, D., Zipoli, F., Salinga, M., and Sebastian, A. Collective structural relaxation in phase-change memory devices. *Advanced Electronic Materials*, 4(9):1700627, 2018. doi: <https://doi.org/10.1002/aelm.201700627>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/aelm.201700627>.
- Lee, J., Valavi, H., Tang, Y., and Verma, N. Fully row/column-parallel in-memory computing sram macro employing capacitor-based mixed-signal computation with 5-b inputs. In *2021 Symposium on VLSI Circuits*, pp. 1–2, 2021. doi: 10.23919/VLSICircuits52068.2021.9492444.
- Li, C., Hu, M., Li, Y., Jiang, H., Ge, N., Montgomery, E., Zhang, J., Song, W., Dávila, N., Graves, C. E., Li, Z., Strachan, J. P., Lin, P., Wang, Z., Barnell, M., Wu, Q., Williams, R. S., Yang, J. J., and Xia, Q. Analogue signal and image processing with large memristor crossbars. *Nat. Electron.*, 1(1):

Article

Integer-Only CNNs with 4 Bit Weights and Bit-Shift Quantization Scales at Full-Precision Accuracy

Maarten Vandersteegen , Kristof Van Beeck  and Toon Goedemé 

KU Leuven, EAVISE-Jan Pieter De Nayerlaan 5, 2860 Sint-Katelijne-Waver, Belgium;
kristof.vanbeeck@kuleuven.be (K.V.B.); toon.goedeme@kuleuven.be (T.G.)

* Correspondence: maarten.vandersteegen@kuleuven.be; Tel.: +32-15-31-69-44

Abstract: Quantization of neural networks has been one of the most popular techniques to compress models for embedded (IoT) hardware platforms with highly constrained latency, storage, memory-bandwidth, and energy specifications. Limiting the number of bits per weight and activation has been the main focus in the literature. To avoid major degradation of accuracy, common quantization methods introduce additional scale factors to adapt the quantized values to the diverse data ranges, present in full-precision (floating-point) neural networks. These scales are usually kept in high precision, requiring the target compute engine to support a few high-precision multiplications, which is not desirable due to the larger hardware cost. Little effort has yet been invested in trying to avoid high-precision multipliers altogether, especially in combination with 4 bit weights. This work proposes a new quantization scheme, based on power-of-two quantization scales, that works on-par compared to uniform per-channel quantization with full-precision 32 bit quantization scales when using only 4 bit weights. This is done through the addition of a low-precision lookup-table that translates stored 4 bit weights into nonuniformly distributed 8 bit weights for internal computation. All our quantized ImageNet CNNs achieved or even exceeded the Top-1 accuracy of their full-precision counterparts, with ResNet18 exceeding its full-precision model by 0.35%. Our MobileNetV2 model achieved state-of-the-art performance with only a slight drop in accuracy of 0.51%.

Keywords: quantization; neural networks; nonuniform; power-of-two scales; low-cost hardware



Citation: Vandersteegen, M.; Van Beeck, K.; Goedemé, T. Integer-Only CNNs with 4 Bit Weights and Bit-Shift Quantization Scales at Full-Precision Accuracy. *Electronics* **2021**, *10*, 2823. <https://doi.org/10.3390/electronics10222823>

Academic Editor: Luis Gomes

Received: 31 October 2021

Accepted: 15 November 2021

Published: 17 November 2021

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Copyright: © 2021 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Quantization of neural networks dates back to the 1990s [1,2], where the discretization of models was a necessity to make their implementation feasible on the available hardware. More recently, neural networks became popular again because of the ImageNet challenge [3] and the availability of powerful GPU hardware. This breakthrough started a new area of research with hundreds of new potential applications. Today, neural networks are found in various electronic devices such as smartphones, wearables, robots, self-driving cars, smart sensors, and many others. The embedded electronic processors found in these applications are often very limited in capabilities to make them affordable, compact, and energy efficient in the case of battery-powered applications. Most neural networks are too large and require too many computations to be implemented directly on such processors, and therefore need to be compressed first. One of the most effective ways to reduce latency, storage cost, memory-bandwidth, energy efficiency, and silicon area among popular compression techniques such as model pruning [4] and network architecture search [5] is model quantization [6]. The quantization of neural networks is a frequently visited research topic with numerous publications that mostly focus on reducing the number of bits per weight or activation as much as possible in order to achieve high compression rates [7–11].

The commonly accepted method to achieve low-precision quantization introduces high-precision scale factors, and in some cases zero-points, to adapt the quantized values to the diverse weight and activation ranges present in full-precision neural networks. This

zation scales. Lookup-table approaches seemed the most interesting, given that they can represent any value, compared to the limited number of steps representable through (sums of) power-of-two values.

2.3.4. Mixed Precision

Most works consider *homogeneous* quantization, which means that all weight or all activations across a model share the same bit-widths. Several recent works [28–31] explored the quantization of heterogeneous models, where each weight tensor in a layer can have its own bit-width, also commonly referred to as mixed-precision models. Assigning low bit-widths to layers that are less sensitive to quantization can increase compression and bandwidth efficiency even further. Moreover, a fine-grained trade-off can be selected between model accuracy and model compression by choosing the right configuration. The major challenge for mixed-precision quantization algorithms is to determine the bit-width configuration for an entire model. Several works [28–30] designed a sensitivity metric that represents the sensitivity to quantization for a given layer. A bit-width, proportional to this metric, is then assigned to that layer. Cai et al. [31] solved the selection of the right mixed-precision model through differential NAS.

Although mixed precision has many great advantages, it requires specialized hardware to support it. In this work, we did not consider using mixed precision; however, our method is suited to be used in combination with mixed precision, which can be explored in future work.

2.4. Quantizer Parameters

This section describes the different data types that are used in practice to represent quantizer parameters (Section 2.4.1) and how these parameters are estimated in the PTQ and QAT algorithms (Section 2.4.2).

2.4.1. Quantizer Parameter Data Types

The (simulated) data type of quantizer parameters such as scale factors and zero-points matters, since they influence the requirements of the target hardware. If, for example, s would be a full-precision floating-point number during simulation, the accelerator would also need to support scaling operations in full-precision. Since floating-point operations in integer-based accelerators are usually not preferred, other solutions have been proposed in the literature for scale factors: dynamic fixed-point integers [13], which are 32 bit or 16 bit fixed-point integers plus a shift operation to enlarge their dynamic range, and powers-of-two values [12]. Zero-points are less critical and in practice represented through integers with the same bit-width as their integer data $\mathbf{q}$. Dynamic fixed-point integers are mostly used for scale factors since they closely resemble the dynamics and precision of floating-point numbers, which allows simulators to just represent them in floating-points. Power-of-two (POT) scale factors, according to Jain et al. [13], can be simulated with:

$$s_{POT} = 2^{\text{ceil}(\log_2(s))} \quad (23)$$

s is a full-precision scale factor; s_{POT} is the power-of-two scale factor; ceil is a ceiling operator that rounds upwards. In this work, we preferred to use these power-of-two scale factors because they can be executed with a single bit-shift operation.

2.4.2. Quantizer Parameter Estimation

We found different techniques to estimate the quantizer parameters in both the PTQ and QAT frameworks. More traditional works estimate these parameters using simple data statistics, such as observed minimum and maximum data values [13,14,19], and use them as lower and upper bounds of the quantization grid, respectively. This approach is however sensitive to outliers and tends to overestimate the range, resulting in large quantization errors, especially for bit-widths smaller than eight. To overcome this issue,

many PTQ algorithms minimize some distance function D between the full-precision data and the quantized data:

$$\arg \min_{s,z} D(\mathbf{x}, Q_f(\mathbf{x}, s, z)) \quad (24)$$

Different metrics for D include the Kullback–Leibler divergence between both distributions [12,32] and the mean-squared error [18,33]. Modern QAT methods make quantizer parameters trainable and optimize them together with the weights [6,9,12,21,24,34,35]. Using the chain rule and the STE, the gradient w.r.t. the scale factor can be calculated. For the uniform symmetric quantizer, this will be:

$$\frac{\partial}{\partial s} \left[s \text{ clamp} \left(\text{round} \left(\frac{\mathbf{x}}{s} \right), n, p \right) \right] = \begin{cases} \text{round} \left(\frac{\mathbf{x}}{s} \right) - \frac{\mathbf{x}}{s} & \text{if } n \leq \text{round} \left(\frac{\mathbf{x}}{s} \right) \leq p \\ n & \text{if } \text{round} \left(\frac{\mathbf{x}}{s} \right) < n \\ p & \text{if } \text{round} \left(\frac{\mathbf{x}}{s} \right) > p \end{cases} \quad (25)$$

$n = -2^{b-1}$ and $p = 2^{b-1} - 1$ for signed and $n = 0$ and $p = 2^b - 1$ for unsigned data.

Some works define $s = 1$ and $z = 0$ [11,36,37], which completely eliminates scaling on the target device. The consequence, however, is that all layers have to learn quantized values that lie more or less within the same range, which works since these authors trained their models from scratch. Training from scratch with quantizers is, however, less preferable since it takes much longer to train and sometimes results in less accurate models.

As mentioned in Section 2.4.1, we preferred power-of-two scales. We optimized the scales through backpropagation as proposed by Jain et al. [12]. In the forward pass, the power-of-two scale is calculated with Equation (23). In the backward pass, the STE is used for the ceil function so $\frac{\partial \text{ceil}(x)}{\partial x} = 1$, and $s_{\log} = \log_2 s$ is treated as a training parameter rather than s itself. The latter enables training in the $\log_2$ domain, which increases the numerical stability and provides better convergence. The gradient w.r.t. $s_{\log}$ of Equation (23) is:

$$\frac{\partial}{\partial s_{\log}} \left[2^{\text{ceil}(s_{\log})} \right] = 2^{\text{ceil}(s_{\log})} \ln 2 \quad (26)$$

2.5. Quantizing Other Layers

Sections 2.1 and 2.2 discuss the internals of a fixed-point compute engine and how to simulate it, respectively. In this section, we discuss how other common layers, such as elementwise adders or concatenation layers, need to be handled in hardware in order to correctly deal with scale factors and zero-points:

Max pooling: This layer does not need special handling because it only has a single input branch and does not alter these input values numerically;

Average pooling: The average of multiple integers is not necessarily an integer, which requires inserting a *fake* quantization node at the output. In practice, average pooling is often executed through a depthwise convolution layer without bias and with weights $\mathbf{w} = 1/K^2 \mathbf{J}$ where K is the kernel size of the average pool layer and $\mathbf{J}$ is an all-ones tensor. This way, the convolution compute engine can be reused for average pool operations, and quantization simulation happens in the same way as for the convolution engine;

Elementwise add: If a scale or zero-point of both input branches of the adder do not match exactly, the quantization grids will be different and the addition will be incorrect. Krishnamoorthi et al. [14] placed a *fake* quantizer node at both input branches and one at the output. Their hardware implementation used a scale operation at both input branches and a round operator at the output. Jain et al. [12] had a slightly different approach. They forced the *fake* quantizer nodes at both input branches to share the same quantizer parameters, as depicted in Figure 5a by the dashed line. This way, the hardware implementation, shown in Figure 5b, only needs a requantization step at the output. In case a ReLU activation function directly follows the elementwise adder, which is for instance the case with ResNet, this function can be fused into the elementwise add operation through the clipping function from the requantization step;

Concatenation: The input branches of a concatenation function likely do not share the same scale or zero-point. Just as elementwise addition, the quantization parameters of the input branches need to be shared. This can be done in simulation by inserting *fake* quantization nodes on every input branch with shared parameters or by using a single *fake* quantization node at the output, which has the same effect (Figure 5c). The hardware implementation is shown in Figure 5d.

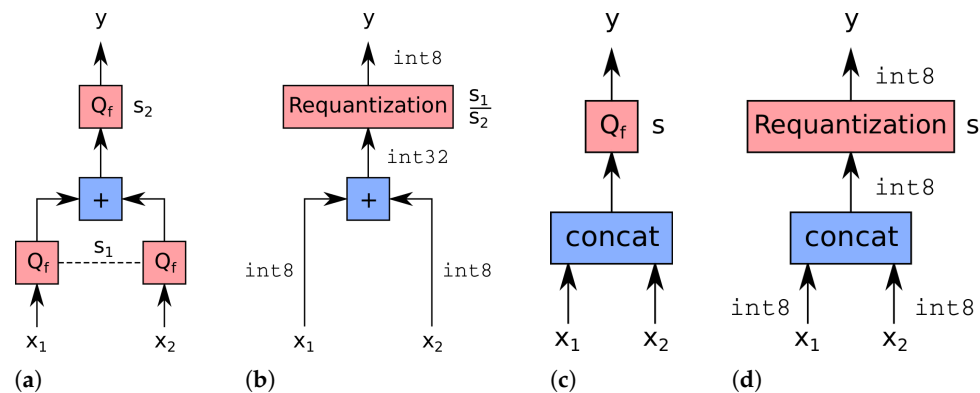


Figure 5. Fake quantized elementwise add (a), its implementation in hardware (b), fake quantized concat (c), and its implementation in hardware (d). The figures assume a uniform symmetric quantization scheme (only scale factors) for the sake of simplicity.

2.6. QAT Initialization

The superior performance of QAT over PTQ is significantly influenced by proper initialization of the QAT process. It is known that initializing QAT with pretrained floating-point weights gives better results compared to training a quantized model from scratch [12–14]. The proper initialization of the quantizer parameters is however equally important, which is why QAT algorithms first perform PTQ to give the quantizer parameters a meaningful starting point. Nagel et al. [6] experimented with different initialization techniques for uniform QAT. They concluded that the mean-squared-error-based estimation of the quantization parameters is favorable compared to min-max-based estimation, though the advantage becomes small towards the end of the training. Other techniques to nudge the weights or biases prior to training such as AdaRound [38] or bias-correction [22] seem not to be beneficial, with the exception of Cross-Layer Equalization (CLE) [22] which, according to their experiments, is required in order to successfully train MobileNets with 4 bit weights.

In addition to PTQ initialization, QAT-trained models with higher bit-widths are also used to initialize QAT trainings with lower bit-widths, in which case the bit-width is progressively reduced in multiple training stages. Commonly known as *progressive* quantization, this strategy has proven to work well and gives a small boost to the final accuracy. Progressive quantization can also mean that parts of the network are quantized progressively rather than the whole network at once. Zhuang et al. [7] proposed to first quantize the weights and leave the activations in full-precision, followed by quantizing both the weights and activations in a second stage. Park et al. [9] exploited this idea even further and proposed PROFIT, a technique to identify layers that cause more instability during QAT than others in MobileNets. By freezing the most unstable layers (mainly depthwise convolutions) in a consecutive training stage, that stage would be trained more smoothly, resulting in better performance. Progressive freezing, by Park et al. [9], was performed in three stages: (1) all layers were trained; (2) 2/3 of the layers were trained; finally, (3) 1/3 of the layers were trained. Each training stage was initialized with the weights from the previous training stage.

We applied CLE for all our MobileNetV2 experiments and used PROFIT in some experiments to further close the gap between quantized and full-precision models.

3. Materials and Methods

As discussed in our related work, most methods rely on high-precision scaling to cope with the different dynamic data ranges, present in full-precision neural networks. High-precision scaling requires additional high-precision multipliers, which require larger silicon area and more energy and, in case FPGAs are used, leads to more expensive platforms. We propose a novel method that avoids high-precision scales in fixed-point compute engines by using power-of-two scales, which can be applied through a single bit-shift. Jain et al. [12] already proved that quantized models with 8 bit weights and power-of-two quantization scales can achieve full-precision accuracy or even better. Although much harder, we prove that we can achieve full-precision accuracy with 4 bit weights and power-of-two scales. Our method was inspired by the superior performance of nonuniform quantizers over uniform quantizers, but does not require complex hardware.

The difference between a typical compute engine and our proposed engine is depicted in Figure 1a,b, respectively. Our compute engine relies on bit-shift-based scaling with a single scale per layer, and we propose to use a lookup-table to translate 4 bit weights into 8 bit weights for internal computation. The lookup-table allows the compute engine to have nonuniform properties, boosting its expressiveness significantly. Since the LUT can hold any number representable through eight bits, different value combinations can be chosen to match the underlying weight distribution within a specific convolution layer. A different set of LUT values is used for every layer to best match the layer-specific data distribution. Note that a single LUT can be shared among multiple parallel compute engines, allowing an additional hardware simplification.

Section 3.1 presents our quantization scheme, the design changes to the compute engine, and the *fake* quantizer that we propose to simulate our design. Section 3.2 discusses the proposed optimization algorithm to estimate the 8 bit lookup-table values for each layer, and Section 3.3 describes the initialization method of both lookup-table values and power-of-two scales, used prior to training.

3.1. Lookup-Table Quantizer

Our quantization scheme can be expressed by Equation (27), where a floating-point vector $\mathbf{x}$ is approximated by a power-of-two scale factor s , multiplied by the output of LUT function L . L simply returns the integer values from LUT content $\mathbf{v} \in [-2^{b-1} \dots 2^{b-1} - 1]^K$, which correspond to quantized values $\mathbf{q}$ (the lookup addresses). Here, K is the number of values in the LUT and b is the number of bits to represent them.

$$\mathbf{x} \approx sL(\mathbf{q}, \mathbf{v}) \quad s.t. \quad s = \frac{2^l}{2^{b-1}} \tag{27}$$

If, for example, $\mathbf{q}$ contains 4 bit addresses, the LUT will contain $K = 16$ values, each with b bits (typical $b = 8$). Because $\mathbf{v}$ is a set of discrete b bit values rather than floating-point values, we used the power-of-two scale factor s to increase the dynamic range of the scheme, where $l \in \mathbb{Z}$ is the amount of left-shifting. Figure 6 illustrates this scheme graphically.

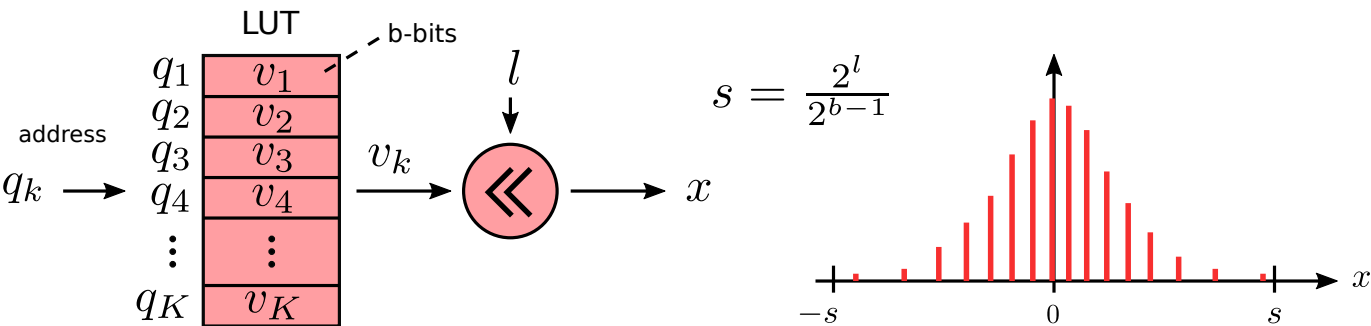


Figure 6. Our quantization scheme, which calculates a value x by translating a lookup-address q_k into a b bit value v_k and bit-shifts the results by l places.

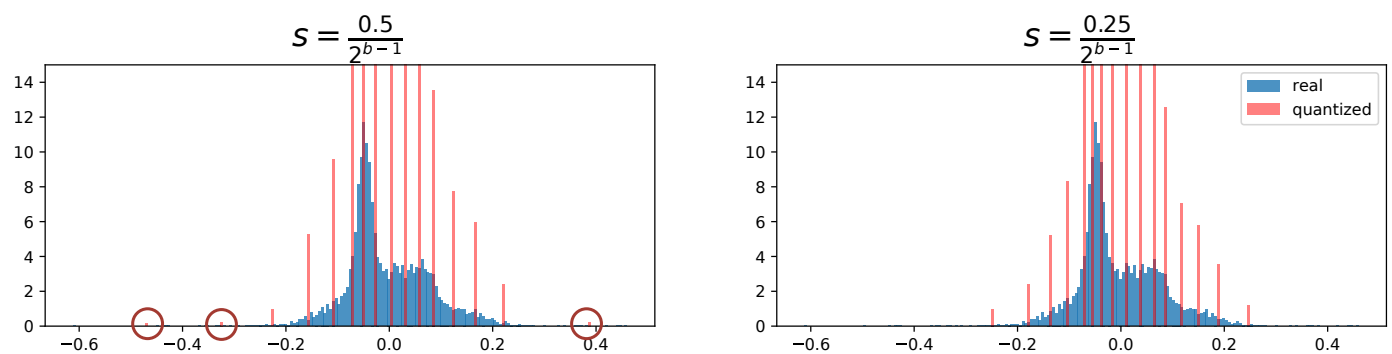


Figure 7. Real and quantized (16 bins) distributions of a weight tensor from MobileNetV2, generated using Algorithm 1. On the left, a few quantization bins are assigned to outliers due to the larger scale factor s . On the right, a smaller s results in a more compact quantized distribution because of the clamp operator.

3.2.1. Stopping Optimization of the LUT Values

Each weight layer has its own LUT vector $\mathbf{v}$ within a model. During training, when a LUT vector $\mathbf{v}$ is stabilized, it is rounded and its optimization procedure is stopped, which we call *freezing* (setting *enable_optimization* to false). We considered a vector stabilized when the following criteria are met:

$$\text{round}(\mathbf{v}) = \text{round}(\mathbf{v}_{\text{smooth}}) \quad (33)$$

where $\mathbf{v}_{\text{smooth}}$ is a smoothed version of $\mathbf{v}$ calculated by an EMA filter with a high decay (0.999). We start checking all LUT vectors after the first 1000 training iterations every 50 cycles. We only froze one LUT vector at a time. In case the criteria are met for multiple $\mathbf{v}$, the vector with the smallest round error $\|\mathbf{v} - \text{round}(\mathbf{v})\|_2$ is preferred.

3.3. Initialization

Both scale factor s and LUT values $\mathbf{v}$ are initialized prior to training using our PTQ as the initialization method. Our initialization method, summarized in Algorithm 3, starts with uniformly distributed LUT values $\mathbf{v}$ and a power-of-two scale factor determined from the maximum value of $\mathbf{x}$. It then optimizes $\mathbf{v}$ through Algorithm 2 for that specific scale factor followed by calculating the mean-squared error ϵ between quantized data $\mathbf{q}$ and $\mathbf{x}$. This is repeated for five smaller power-of-two values of s . Optimal scale factor $\hat{s}$ and LUT vector $\hat{\mathbf{v}}$ that produced the lowest ϵ are selected as the final initialization values for training. Note that our PTQ does not yet round the values in $\mathbf{v}$.

Algorithm 3 is applied for all weight tensors. Batch normalization layers are folded prior to quantization as discussed in Section 2.2.2. We did not quantize bias parameters in our experiments since we assumed 32 bit integer bias parameters and accumulators.

3.4. Quantizing Feature-Map Data

For quantizing feature-map data or activations, 8 bit quantization with power-of-two quantization scales was used. This was implemented through the symmetric uniform quantizer from Jain et al. [12] with learnable power-of-two scales. Their hyperparameter settings and scale freezing technique for optimizing the scales were adopted as well. Note that we deliberately did not choose to use asymmetric quantizers for feature-map data. Although this is possible, it introduces additional zero-points that need optimization, which complicates the training process. In addition, we empirically discovered that the added value is virtually nothing at 8 bit compared to a mix of signed and unsigned symmetric quantizers at the right locations. Throughout the model, quantizers are inserted at the inputs, activation functions, and other nonparametric layers such as elementwise adders, concatenations and average pooling layers. This is done to achieve correct integer-only inference simulation, as discussed in Section 2.5.

Algorithm 3: Initialization of LUT values $\mathbf{v}$ and scale factor s .**Input:** Floating-point data $\mathbf{x} = [x_1, x_2, \dots, x_N]$ **Output:** Optimal scale factor $\hat{s}$ and LUT values $\hat{\mathbf{v}} = [v_1, v_2, \dots, v_K]$ **Procedure:** $\hat{\epsilon} \leftarrow \infty$ $\hat{s} \leftarrow s \leftarrow \frac{2^l}{2^{b-1}} \quad \text{s.t.} \quad l = \text{ceil}(\log_2 \max |\mathbf{x}|)$ **for** $i = 1$ **to** 5 **do** $\mathbf{v} = [v_1, v_2, \dots, v_K]$ with uniformly distributed values in range $[-2^{b-1}, 2^{b-1} - 1]$ **for** $j = 1$ **to** 100 **do** $\mathbf{q}, \mathbf{v} \leftarrow \text{Algorithm 2}(\mathbf{x}, s, \mathbf{v}) \quad // \text{ Optimization enabled}$ **end** $\epsilon \leftarrow \|\mathbf{x} - \mathbf{q}\|_2^2$ **if** $\epsilon < \hat{\epsilon}$ **then** $\hat{\epsilon} \leftarrow \epsilon \quad \hat{s} \leftarrow s \quad \hat{\mathbf{v}} \leftarrow \mathbf{v}$ **end** $s \leftarrow s/2$ **end**

4. Results

To evaluate the performance of our LUT-based method, we conducted several experiments on CNN-based image classifier models and an object detection model. Section 4.1 compares the PTQ and QAT performance of our proposed method against other uniform quantizers with both full-precision and power-of-two quantization scales in both per-layer and per-channel mode. Section 4.2 compares our results against results from other state-of-the-art works for five different image classifier models. Section 4.3 presents the results of our method, tested on an object detector model. Section 4.4 presents a small study on the added value of optimizing the LUT values during QAT. Section 4.5 discusses additional tools to further improve the accuracy of MobileNetV2. Finally, in Section 4.6, we present the simulated inference speed of a few CNN models with both 8 and 4 bit weights on an FPGA platform.

4.1. Comparison to Different Quantizers

In this first set of experiments, we compared our proposed nonuniform weight quantization method against uniform quantization methods with the POT scale and float scale in both the per-layer and per-channel configurations. Table 1 lists the Top-1 ImageNet accuracies of quantized ResNet18 [39] and MobileNetV2 [40] with 4 bit weights and 8 bit activations. We included both QAT's and PTQ's results, where the PTQ models were used to initialize the QAT models. Note that all these results were produced using the same framework for a fair comparison.

Table 1 shows that our method achieved the highest scores for both models, even slightly outperforming per-channel quantized models with full-precision scales. The advantage of the lookup-table clearly made the difference: our method can be used as a worthy alternative for uniform quantizers with both per-layer and per-channel quantization with floating-point scales. We also noticed that uniform quantization with POT scales in both per-layer and per-channel quantization did not completely recover the baseline accuracy of ResNet18, while our method did with a safe margin of +0.35%, which proves the superiority of our method for POT scales.

It is known that the accuracy of QAT models can exceed the accuracy of their full-precision counterparts [6,12,14], which can also be seen with our method. This phenomenon can be explained by the additional training cycles of QAT and the regularization effect of the quantization noise during QAT.

Table 1. ImageNet Top-1 results using different quantizers with 4 bit weights and 8 bit activations. The first number represents the relative accuracy or difference between the quantized model and the full-precision model, while the second number in parenthesis represents the absolute accuracy.

Model (FP32 Accuracy)	ResNet18 (69.76)		MobileNetV2 (71.88)	
	PTQ	QAT	PTQ	QAT
Float scale per-layer	−50.18 (19.58)	+0.08 (69.84)	−67.11 (4.77)	−1.51 (70.37)
Float scale per-channel	−15.85 (53.91)	+0.23 (69.99)	−64.96 (6.92)	−0.97 (70.91)
POT scale per-layer	−60.14 (9.62)	−0.27 (69.49)	−71.05 (0.83)	−1.73 (70.15)
POT scale per-channel	−31.75 (38.01)	−0.08 (69.68)	−69.87 (6.34)	−1.66 (70.22)
POT scale per-layer + LUT (ours)	−12.51 (57.25)	+0.35 (70.11)	−55.95 (15.93)	−0.92 (70.96)

Although per-channel quantization with POT-scales was never attempted by Jain et al. [12], their per-layer results were consistent with ours. Per-channel quantization with POT scales has, to the best of our knowledge, never been tried in the literature before. Our procedure for uniform per-channel quantization with POT scales adopted the gradual freezing method of Jain et al. [12] to freeze the POT scales once stabilized, but applied it on a per-channel level. Instead of allowing a single POT scale to be frozen every 50 training iterations, we allowed up to 400 per-channel scales to be frozen, using the same stabilization criteria from Jain et al. [12].

For MobileNetV2, however, no quantizer succeeded in recovering the full-precision accuracy, because it is very sensitive to quantization due to the depthwise separable convolutions. Although CLE [22] was applied prior to quantization to improve MobileNetV2’s QAT accuracy in all our experiments, as suggested by Nagel et al. [6], a drop in accuracy of approximately one percent still existed. In Section 4.5, we further analyze the effect of CLE on POT-scale-based quantizers in more detail, and we discuss what can be done in addition to further improve the results of MobileNetV2.

Both models were trained for 20 epochs on ImageNet, with standard preprocessing, data augmentation, and batch sizes of 256 and 128 for ResNet18 and MobileNetV2, respectively. We used the Adam optimizer to optimize both weights and quantization scales with its default parameters. For the weights, learning rates of 1×10^{-5} and 3×10^{-5} were used for ResNet18 and MobileNetV2, respectively, with a cosine learning rate decay that reduced the learning rate to zero at the end of the training. The quantization scales had a learning rate of 1×10^{-2} that quickly decayed in a step-based fashion, as suggested by Jain et al. [12] for their POT scale quantizer. We discovered that this learning rate schedule worked better for both POT-scale and float-scale models compared to the same learning rate schedule as used for the weights. Note that other best practices to train our POT-scale models, including gradual freezing of the scales once stabilized, were also adopted from Jain et al. [12], for our experiments that optimize POT scales.

The PTQ methods for the uniform models used mean-squared error estimation to initialize both weight and activation scales, as suggested by Nagel et al. [6]. Note that all experiments used symmetric quantizers for the activations.

4.2. Comparison to the State-of-the-Art

In this section, we compare our method against results from other state-of-the-art methods for a number of different models. The selection criteria used to select works to compare to were twofold: first, mainly QAT methods and, second, works that reported results with 4 bit weights and 8 bit activations. Table 2 presents the Top-1 ImageNet accuracies of ResNet18 [39], ResNet50 [39], MobileNetV2 [40], VGG16 [41] without batch normalization, and InceptionV3 [42]. The accuracies of the full-precision models among the related works slightly differed and are therefore listed separately (32/32) in addition to the accuracies of the quantized models with 4 bit weights and 8 bit activations (4/8). In addition to the results of our method (*Ours LUT*), we also present our own results of uniform quantization with POT scales (*Ours uniform*), to highlight the added value of our LUT.

Results from the other works were directly taken from their papers. AB [18], QAT1 [14], and QAT2 [6] are all QAT methods that use per-channel quantization with full-precision quantization scales, where AB and QAT2 also experimented with per-layer quantization. TQT [12] is a QAT method that uses POT scales with per-layer quantization, and PWLQ [43] is a PTQ method that uses multiple uniform quantizers to obtain nonuniform quantization.

We can conclude that all models using our method achieved and even exceeded full-precision model accuracy with a safe margin, with the exception of MobileNetV2. Although the latter model was not fully recovered, we still achieved the highest accuracy compared to other methods. Nagel et al. [6] achieved a slightly higher accuracy on ResNet50 and InceptionV3 using per-channel quantization with float scales, but comparable to ours.

For ResNet50, VGG16, and InceptionV3, we used a learning rate 3×10^{-6} and their standard respective preprocessing and data augmentation. All other hyperparameters were as described in the previous Section 4.1. InceptionV3 was fine-tuned without auxiliary loss, as suggested by Jaint et al. [12].

In Table 3, we also present the ImageNet results of uniform quantized models with power-of-two scales and 8 bit weights instead of 4 bit weights, to further put our 4/8 results into perspective. Overall, these results were slightly higher compared to our results from Table 2, which was however to be expected. Similar training conditions were used as with our 4/8 experiments.

Table 2. Top-1 ImageNet accuracies of different works. The first value represents the relative accuracy or difference between the quantized model and the full-precision model, while the second number in parenthesis represents the absolute accuracy. Columns Per-Ch and POT scale indicate whether per-channel quantization or POT scales were used, respectively.

Method	w/a	Per-Ch	POT Scale	ResNet18	ResNet50	MobileNetV2	VGG16	InceptionV3
AB [18]	32/32			/	75.20	/	/	/
	4/8	no	no	/	−1.40 (73.80)	/	/	/
	4/8	yes	no	/	−0.90 (74.30)	/	/	/
PWLQ [43]	32/32			/	76.13	71.88	/	77.49
	4/8	yes	no	/	−0.51 (75.62)	−2.68 (69.22)	/	−1.04 (76.45)
TQT [12]	32/32			/	75.20	/	70.90	78.00
	4/8	no	yes	/	−0.80 (74.40)	/	+0.60 (71.50)	−1.60 (76.4)
QAT1 [14]	32/32			/	75.20	71.90	/	78.00
	4/8	yes	no	/	−2.00 (73.2)	−9.90 (62.00)	/	−2.00 (76.00)
QAT2 [6]	32/32			69.76	76.07	71.72	/	77.40
	4/8	no	no	+0.08 (69.76)	−0.18 (75.89)	−1.55 (70.17)	/	+0.44 (77.84)
	4/8	yes	no	+0.33 (70.01)	+0.45 (76.52)	−1.24 (70.48)	/	+0.72 (78.12)
Ours uniform	32/32			69.76	76.13	71.88	71.59	77.34
Ours LUT	4/8	no	yes	−0.27 (69.49)	−0.11 (76.02)	−1.73 (70.15)	+0.09 (71.68)	+0.23 (77.57)
	4/8	no	yes	+0.35 (70.11)	+0.18 (76.31)	−0.92 (70.96)	+0.28 (71.87)	+0.39 (77.73)

Table 3. Top-1 ImageNet accuracies of uniform quantized models with 8 bit weights and 8 bit activations using POT scales.

w/a	ResNet18	ResNet50	MobileNetV2	VGG16	InceptionV3
32/32	69.76	76.13	71.88	71.59	77.34
8/8	+0.74 (70.50)	+0.54 (76.67)	+0.00 (71.88)	+0.37 (71.96)	+0.99 (78.33)

4.3. Object Detection

We also tested our method on the Tiny YOLOV2 [44] object detector. Table 4 presents the mean-average-precision results of Tiny YOLOV2 on the MS COCO dataset with 4 bit weights and 8 bit activations. The model was quantized with per-layer POT scales, both with and without our LUT method. We can conclude that also for this object detector, our method improved the final QAT result compared to uniform quantization. We trained the

In future work, our method can be used easily in mixed-precision computation, where each weight tensor in a layer can have its own bit-width, to achieve a more fine-grained trade-off between model compression and model accuracy. Models with a mix of 2 bit, 4 bit, and 8 bit weights could be constructed and executed on our proposed compute engine without any modification. A 2 bit layer could be configured by simply using only four entries of the LUT, while an 8 bit layer could be configured through a simple bypass of the LUT.

Author Contributions: Methodology, M.V.; software, M.V.; supervision, T.G.; validation, K.V.B.; writing—original draft, M.V.; writing—review and editing, M.V., K.V.B., and T.G. All authors have read and agreed to the published version of the manuscript.

Funding: This work was funded by VLAIO and EASICS <https://www.easics.com> (accessed on 16 November 2021) through the Start to Deep Learn TETRA project and FWO via the OmniDrone SBO project.

Data Availability Statement: The following publicly available datasets were used in this research: the ImageNet dataset can be found here <https://www.image-net.org/download.php> and the MS COCO dataset can be found here <https://cocodataset.org/#download> (both accessed on 16 November 2021). The code used to conduct this research can be found here <https://gitlab.com/EAVISE/lut-model-quantization> (accessed on 16 November 2021).

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Fiesler, E.; Choudry, A.; Caulfield, H.J. Weight discretization paradigm for optical neural networks. *Optical interconnections and networks. Int. Soc. Opt. Photonics* **1990**, *1281*, 164–173.
2. Balzer, W.; Takahashi, M.; Ohta, J.; Kyuma, K. Weight quantization in Boltzmann machines. *Neural Netw.* **1991**, *4*, 405–409. [CrossRef]
3. Deng, J.; Dong, W.; Socher, R.; Li, L.J.; Li, K.; Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In Proceedings of the 2009 IEEE Conference on Computer Vision and Pattern Recognition, Miami, FL, USA, 20–25 June 2009; pp. 248–255.
4. Molchanov, P.; Tyree, S.; Karras, T.; Aila, T.; Kautz, J. Pruning convolutional neural networks for resource efficient transfer learning. *arXiv* **2016**, arXiv:1611.06440.
5. Banbury, C.; Zhou, C.; Fedorov, I.; Matas, R.; Thakker, U.; Gope, D.; Janapa Reddi, V.; Mattina, M.; Whatmough, P. Micronets: Neural network architectures for deploying tinyml applications on commodity microcontrollers. *Proc. Mach. Learn. Syst.* **2021**, *3*, 517–532.
6. Nagel, M.; Fournarakis, M.; Amjad, R.A.; Bondarenko, Y.; van Baalen, M.; Blankevoort, T. A White Paper on Neural Network Quantization. *arXiv* **2021**, arXiv:2106.08295.
7. Zhuang, B.; Shen, C.; Tan, M.; Liu, L.; Reid, I. Towards effective low-bitwidth convolutional neural networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–23 June 2018; pp. 7920–7928.
8. Rastegari, M.; Ordonez, V.; Redmon, J.; Farhadi, A. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European Conference on Computer Vision*; Springer: Berlin/Heidelberg, Germany, 2016; pp. 525–542.
9. Park, E.; Yoo, S. Profit: A novel training method for sub-4 bit mobilenet models. In *European Conference on Computer Vision*; Springer: Berlin/Heidelberg, Germany, 2020; pp. 430–446.
10. Zhang, D.; Yang, J.; Ye, D.; Hua, G. Lq-nets: Learned quantization for highly accurate and compact deep neural networks. In Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 365–382.
11. Zhou, S.; Wu, Y.; Ni, Z.; Zhou, X.; Wen, H.; Zou, Y. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv* **2016**, arXiv:1606.06160.
12. Jain, S.; Gural, A.; Wu, M.; Dick, C. Trained Quantization Thresholds for Accurate and Efficient Fixed-Point Inference of Deep Neural Networks. *Proc. Mach. Learn. Syst.* **2020**, *2*, 112–128.
13. Jacob, B.; Kligys, S.; Chen, B.; Zhu, M.; Tang, M.; Howard, A.; Adam, H.; Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–23 June 2018; pp. 2704–2713.
14. Krishnamoorthi, R. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv* **2018**, arXiv:1806.08342.
15. Ioffe, S.; Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Proceedings of the International Conference on Machine Learning, Lille, France, 6–11 July 2015; pp. 448–456.
16. Bengio, Y.; Léonard, N.; Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv* **2013**, arXiv:1308.3432.
17. Yang, J.; Shen, X.; Xing, J.; Tian, X.; Li, H.; Deng, B.; Huang, J.; Hua, X.S. Quantization networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 16–17 June 2019; pp. 7308–7316.

Towards Machine Learning for Placement and Routing in Chip Design: a Methodological Overview

Junchi Yan^{1*}, Xianglong Lyu¹, Ruoyu Cheng¹, Yibo Lin^{2*}

¹Department of CSE, and MoE Key Lab of Artificial Intelligence, Shanghai Jiao Tong University

²Department of EECS, Peking University

{yanjunchi,kyle_lyu,roy_account}@sjtu.edu.cn yibolin@pku.edu.cn

Abstract

Placement and routing are two indispensable and challenging (NP-hard) tasks in modern chip design flows. Compared with traditional solvers using heuristics or expert-well-designed algorithms, machine learning has shown promising prospects by its data-driven nature, which can be of less reliance on knowledge and priors, and potentially more scalable by its advanced computational paradigms (e.g. deep networks with GPU acceleration). This survey starts with the introduction of basics of placement and routing, with a brief description on classic learning-free solvers. Then we present detailed review on recent advance in machine learning for placement and routing. Finally we discuss the challenges and opportunities for future research.

1 Introduction

The scale of integrated circuits (ICs) has increased dramatically, posing a challenge to the scalability of existing Electronic Design Automation (EDA) techniques and technologies. The increasing circuit density incurs additional issues for very large scale integration (VLSI) placers and routers as the feature size of modern VLSI design continues to drop and on-chip connectivity gets increasingly sophisticated. Due to increased on-chip connectivity, concentrated needs, and restricted resources, modern designs are prone to congestion issues and wire-length minimization, which has become a critical task at every stage of the design process. Accordingly, placement and routing – the elements of the design cycle physically arranges the locations and the courses of nets – becomes more crucial in modern VLSI.

Placement and routing are two of the most critical but time-consuming steps of the chip design process. Placement assigns various logic components like logic gates and functional blocks into discrete sites in the physical layout of a chip, and routing finishes the physical wiring of interconnections. The goal of placement and routing is to optimize power, performance, and area (PPA) metrics within constraints, e.g., placement density and routing congestion.

*Correspondence author.

Placement can be regarded as a much more complicated variation of 2D bin packing problem with geometrical constraints. The latter is known to be NP-hard [Hartmanis, 1982]. The objective of placement is correlated to both the logical interconnection of the circuit design and the geometrical locations of logical components. As the quality of a placement solution cannot be accurately evaluated until routing, resulting in a long feedback loop in the design flow, modern placement needs to reduce routing congestion and improve routability in early stage. Routing has been proven to be an NP-hard problem [Hartmanis, 1982], even in the simplest case with only a few two-pin networks. Given the placement solution, it connects the pins of millions of networks with physical wires subjecting to the limited routing resources and complicated geometrical design rules. Routing is tightly coupled with placement and an excellent placement solution leads to better chip area utilization, timing performance, and routability.

There are also surveys on routing and placement. [Huang *et al.*, 2021] summaries a comprehensive review of existing ML studies for EDA field, most of which belongs to the four categories: decision making in traditional methods, performance prediction, black-box optimization, and automated design, ordered by increasing degree of automation. Although it covers various stages in the EDA flow, it provides shallow analysis for specific tasks, placement and routing, compared to ours. [Hamolia and Melnyk, 2021] introduces the ISPD 2015 dataset for comparison between classic methods and ML algorithms. [Rapp *et al.*, 2021] categorizes how ML may be used and is used for design-time/run-time optimization and exploration strategies of ICs, along with trends in the employed ML algorithms. [Markov *et al.*, 2015] reviews the history of placement research and the progress leading up to the state of the art. However, they only consider classic placer in the past 50 years, which is complementary to our survey which focuses on learning-based placer in recent years.

2 Problem Background and Classic Solvers

This section starts with the problem formulation and mainstream solvers which are traditionally learning-free. We then introduce recent applicable learning techniques.

Task type	Feature from	Label at	Publication	Backbone models	Benchmark
Congestion Count (Scalar or Vector as Labels)	Global Routing	Detailed Routing	[Qi <i>et al.</i> , 2014]	MARS	ASIC
			[Zhou <i>et al.</i> , 2019]	MARS	
	Cell Placement	Global Routing	[Tabrizi <i>et al.</i> , 2018]	MLP	ASIC
			[Maarouf <i>et al.</i> , 2018]	LR, RF, MLP	FPGA
	Cell Placement	Detailed Routing	[Zhou <i>et al.</i> , 2015]	MARS	ASIC
			[Chan <i>et al.</i> , 2016]	MARS, SVM	
Congestion Locations (2D Map as Labels)	Macro Placement	Global Routing	[Cheng <i>et al.</i> , 2018b]	LR, RF, Boosting, MLP	ASIC
	Cell Placement	Global Routing	RouteNet [Xie <i>et al.</i> , 2018]	FCN	ASIC
			PROS [Chen <i>et al.</i> , 2020]	FCN	ASIC
			[Pui <i>et al.</i> , 2017]	LR, SVM	FPGA
			[Yu and Zhang, 2019]	Conditional GAN	
			[Alawieh <i>et al.</i> , 2020]	Conditional GAN	
			DLRoute [Al-Hyari <i>et al.</i> , 2021]	CNN	
	Cell Placement	Detailed Routing	J-Net [Liang <i>et al.</i> , 2020]	U-Net	ASIC

Table 1: Summary of recent publications on learning-aided routability prediction in terms of congestion count and locations.

to predict global routing congestion at placement stage with linear regression (LR), random forest (RF), and MLP models for datasets from ASIC and FPGA. Other studies like [Zhou *et al.*, 2015; Chan *et al.*, 2016] aim at predicting the congestion count at detailed routing given cell placement, and [Cheng *et al.*, 2018b] tries to predict the congestion count at global routing given only macro placement.

Congestion location. Its accurate prediction is necessary to effectively guide the placement and routing optimization, as it can help reserve enough space for congested regions. RouteNet [Xie *et al.*, 2018] is the first attempt to utilize CNN to forecast the locations of design rule checking (DRC) hotspots given cell placement and global routing information. A customized fully convolutional network (FCN) is constructed taking features like rectangle uniform wire density (RUDY), as a pre-routing congestion estimator, and global routing congestion map. Predicting global routing congestion locations at placement is helpful to guide routability optimization in early stages. Thus, PROS [Chen *et al.*, 2020], [Pui *et al.*, 2017], [Yu and Zhang, 2019], [Alawieh *et al.*, 2020], and DLRoute [Al-Hyari *et al.*, 2021] attempt to learn the correlation between congestion locations at global routing and cell placement. As both the features and labels can be represented as image-like tensors, many studies transform the problem into image generation tasks and leverage FCN and conditional GAN to build the correlation. [Liang *et al.*, 2020] moves one step further to directly predict congestion locations at detailed routing (i.e., locations of design rule violations) from cell placement. A customized CNN architecture, J-Net (an extension of U-Net architecture), is proposed as opposed to a plug-in use of machine learning modules. This work converts the density of pins and macros in placement results into images and optimizes an encoder-decoder model using a pixel-wise loss function. The network outputs a heat map, showing where detailed routing congestion might occur.

Among learning-aided prediction tasks, the more stages to skip in Fig. 1, the more difficult the tasks are, as we need to build models correlating with more stages. Thus, the accuracy requirement varies from task to task. Predicting congestion locations at detailed routing at early stages like cell placement or even macro placement can expedite the design.

Publication	Target for placement		Learning protocol	
	Macro	Standard cell	Module	Reward using
[Cheng and Yan, 2021]	RL	DNN backprop	CNN+GNN	cell placement
[Mirhoseini <i>et al.</i> , 2021]	RL	NA	GNN	macro placement
[Vashisht <i>et al.</i> , 2020]	RL	NA	MLP	macro placement
[He <i>et al.</i> , 2020]	RL	NA	MLP	macro placement
[Lin <i>et al.</i> , 2020]	heuristic	DNN backprop	NA	NA

Table 2: Comparison of learning-based placers. RL is the most popular paradigm for macro placement, while learning for standard cell placement still relies on traditional placers for further enhancement. Note that [Cheng and Yan, 2021] is a joint learning approach for solving placement of macro and standard cells, whereby the two kinds of components are sequentially arranged by reinforcement learning and neural network formed gradient optimization respectively and reward is based on the full placement result. [Lin *et al.*, 2020] is not a RL method, thus there is no reward function.

4.2 Deep Neural Networks for Routing

Rather than the methods mentioned above predicting the congestion information of placement, some works employ DNNs to directly handle routing problems, with a little or even without the assistance of traditional routing techniques.

[Jain and Okabe, 2017] presents a fully convolutional neural network learning to route a circuit layout with appropriate choices of metal tracks and wire class combinations. Encoded layouts containing spatial location of pins to be routed are fed into the network, and after 15 fully convolutional layers followed by a comparator, 8 layout layers are produced, which are then decoded to obtain the routed layouts. This work formulates routing as a binary segmentation problem on a per-pixel per-layer basis, where the network is trained to correctly classify pixels in each layout layer to be *on* or *off*. [He and Bao, 2020] models the circuit routing as a sequential decision-making problem, and solve it by Monte Carlo tree search (MCTS) with DNN guided rollout. A recent study [Utyamishv and Partin-Vaisband, 2020] proposes a global router that learns from routed circuits and autonomously routes unseen layouts. Different from traditional routing flow, this approach redefines the global routing as a classical image-to-image processing problem and handles the imaging problem in a unified, single-step non-iterative manner with a deep learning system, comprising a variational autoencoder and a custom loss.

- [Chan *et al.*, 2016] W. Chan, Y. Du, A. Kahng, S. Nath, and K. Samadi. Beol stack-aware routability prediction from placement using data mining techniques. In *ICCD*, 2016.
- [Chan *et al.*, 2017] W. Chan, P. Ho, A. Kahng, and P. Saxena. Routability optimization for industrial designs at sub-14nm process nodes using machine learning. In *ISPD*, 2017.
- [Chang and Lin, 2004] Y. Chang and S. Lin. Mr: A new framework for multilevel full-chip routing. *TCAD*, 2004.
- [Chang *et al.*, 2010] Y.-J. Chang, Y.-T. Lee, J.-R. Gao, P.-C. Wu, and T.-C. Wang. Nthru-route 2.0: a robust global router for modern designs. *TCAD*, 2010.
- [Chen *et al.*, 2008a] H. Chen, Y. Chuang, Y.-W. Chang, and Y. Chang. Constraint graph-based macro placement for modern mixed-size circuit designs. In *ICCAD*, 2008.
- [Chen *et al.*, 2008b] T.-C. Chen, Z.-W. Jiang, T.-C. Hsu, H.-C. Chen, and Y.-W. Chang. Ntuplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints. *TCAD*, 2008.
- [Chen *et al.*, 2009] H. Chen, C. Hsu, and Y. Chang. High-performance global routing with fast overflow reduction. In *ASP-DAC*, 2009.
- [Chen *et al.*, 2019] G. Chen, C. Pui, H. Li, J. Chen, B. Jiang, and E. Young. Detailed routing by sparse grid graph and minimum-area-captured path search. In *ASP-DAC*, 2019.
- [Chen *et al.*, 2020] J. Chen, J. Kuang, G. Zhao, D. Huang, and E. Young. Pros: A plug-in for routability optimization applied in the state-of-the-art commercial eda tool using deep learning. In *ICCAD*, 2020.
- [Cheng and Yan, 2021] R. Cheng and J. Yan. On joint learning for solving placement and routing in chip design. *NeurIPS*, 2021.
- [Cheng *et al.*, 2018a] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang. Replace: Advancing solution quality and routability validation in global placement. *TCAD*, 2018.
- [Cheng *et al.*, 2018b] W.-K. Cheng, Y.-Y. Guo, and C.-S. Wu. Evaluation of routability-driven macro placement with machine-learning technique. In *ISNE*. IEEE, 2018.
- [Cheon *et al.*, 2005] Y. Cheon, P. Ho, A. Kahng, S. Reda, and Q. Wang. Power-aware placement. In *DAC*, 2005.
- [Cho and Pan, 2007] M. Cho and D. Pan. Boxrouter: A new global router based on box expansion and progressive ilp. *TCAD*, 2007.
- [Chu and Wong, 2007] C. Chu and Y.-C. Wong. Flute: Fast lookup table based rectilinear steiner minimal tree algorithm for vlsi design. *TCAD*, 2007.
- [Cong *et al.*, 2001] J. Cong, J. Fang, and K.-Y. Khoo. Dune-a multilayer gridless routing system. *TCAD*, 2001.
- [Fiduccia and Mattheyses, 1982] C. M. Fiduccia and R. M. Mattheyses. A linear-time heuristic for improving network partitions. In *DAC*. IEEE, 1982.
- [Gao *et al.*, 2008] J.-R. Gao, P.-C. Wu, and T.-C. Wang. A new global router for modern designs. In *ASP-DAC*, 2008.
- [Hamolia and Melnyk, 2021] V. Hamolia and V. Melnyk. A survey of machine learning methods and applications in electronic design automation. In *ACIT*, 2021.
- [Hartmanis, 1982] J. Hartmanis. Computers and intractability: a guide to the theory of np-completeness (michael r. Garey and david s. Johnson). *Siam Review*, 1982.
- [He and Bao, 2020] Y. He and F. S. Bao. Circuit routing using monte carlo tree search and deep neural networks. *arXiv preprint arXiv:2006.13607*, 2020.
- [He *et al.*, 2020] Z. He, Y. Ma, L. Zhang, P. Liao, N. Wong, B. Yu, and M. D. Wong. Learn to floorplan through acquisition of effective local search heuristics. In *ICCD*, 2020.
- [Ho *et al.*, 2004] S.-Y. Ho, S.-J. Ho, Y.-K. Lin, and W.-C. Chu. An orthogonal simulated annealing algorithm for large floorplanning problems. *VLSI Systems*, 2004.
- [Hsu *et al.*, 2011] M.-K. Hsu, S. Chou, T.-H. Lin, and Y.-W. Chang. Routability-driven analytical placement for mixed-size circuit designs. In *ICCAD*, 2011.
- [Huang *et al.*, 2019] Y. Huang, Z. Xie, G. Fang, T. Yu, H. Ren, S. Fang, Y. Chen, and J. Hu. Routability-driven macro placement with embedded cnn-based prediction model. In *DATE*, 2019.
- [Huang *et al.*, 2021] G. Huang, J. Hu, Y. He, J. Liu, M. Ma, Z. Shen, J. Wu, et al. Machine learning for electronic design automation: A survey. *TODAES*, 26(5):1–46, 2021.
- [Jain and Okabe, 2017] S. R. Jain and K. Okabe. Training a fully convolutional neural network to route integrated circuits. *arXiv preprint arXiv:1706.08948*, 2017.
- [Kahng and Wang, 2005] A. Kahng and Q. Wang. Implementation and extensibility of an analytic placer. *TCAD*, 2005.
- [Kahng *et al.*, 2018] A. B. Kahng, L. Wang, and B. Xu. Triton-route: An initial detailed router for advanced vlsi technologies. In *ICCAD*, 2018.
- [Kastner *et al.*, 2002] R. Kastner, E. Bozorgzadeh, and M. Sarrafzadeh. Pattern routing: Use and theory for increasing predictability and avoiding coupling. *TCAD*, 2002.
- [Kim *et al.*, 2011] M.-C. Kim, J. Hu, D.-J. Lee, and I. Markov. A simple method for routability-driven placement. In *ICCAD*, 2011.
- [Kim *et al.*, 2015] M. Kim, J. Hu, J. Li, and N. Viswanathan. Iccad-2015 cad contest in incremental timing-driven placement and benchmark suite. In *ICCAD*, 2015.
- [Kirkpatrick *et al.*, 1983] S. Kirkpatrick, D. Gelatt, and M. Vecchi. Optimization by simulated annealing. *Science*, 1983.
- [Lee and Markov, 2012] D. Lee and I. Markov. Obstacle-aware clock-tree shaping during placement. *TCAD*, 2012.
- [Lee, 1961] C. Y. Lee. An algorithm for path connections and its applications. *IRE Transactions on Electronic Computers*, 1961.
- [Li *et al.*, 2018] H. Li, W.-K. Chow, G. Chen, E. F. Young, and B. Yu. Routability-driven and fence-aware legalization for mixed-cell-height circuits. In *DAC*, 2018.
- [Li *et al.*, 2019] H. Li, G. Chen, B. Jiang, J. Chen, and E. F. Young. Dr. cu 2.0: A scalable detailed routing framework with correct-by-construction design rule satisfaction. In *ICCAD*, 2019.
- [Liang *et al.*, 2020] R. Liang, H. Xiang, D. Pandey, L. Reddy, S. Ramji, G.-J. Nam, and J. Hu. Drc hotspot prediction at sub-10nm process nodes using customized convolutional network. In *ISPD*, 2020.
- [Liao *et al.*, 2020a] H. Liao, Q. Dong, X. Dong, W. Zhang, W. Zhang, W. Qi, E. Fallon, and L. B. Kara. Attention routing: track-assignment detailed routing using attention-based reinforcement learning. In *IDETC-CIE*, 2020.
- [Liao *et al.*, 2020b] H. Liao, W. Zhang, X. Dong, B. Poczos, K. Shimada, and L. Burak Kara. A deep reinforcement learning approach for global routing. *Journal of Mechanical Design*, 2020.

Grid Synchronization Phase-Locked Loop Strategy for Unbalance and Harmonic Distortion Conditions

Hicham Chaoui¹ · Okezie Okoye¹ · Mehdy Khayamy¹

Received: 9 February 2016 / Revised: 29 March 2016 / Accepted: 19 May 2016 / Published online: 1 June 2016
© Brazilian Society for Automatics–SBA 2016

Abstract The synthesis, design and analysis of a three-phase phase-locked loop (PLL) algorithm under grid voltage uncertainties is presented. Unlike other techniques, the proposed strategy is simple but yet, robust against unbalanced and distorted voltage conditions. The method does not rely on the symmetry of the three-phase voltages, like conventional PLL implementation techniques. Under phase lock, the reference voltage and the output voltage are in synchronism, hence they cross the zero axis at the same time. The proposed PLL implementation exploits this fact, by counting the difference between the zero crossing times of the grid voltage and the output voltage waveform, and generating an error signal proportional to this difference. This error is fed into a PI regulator which forces it to zero over time. The proposed algorithm is compared against two popular PLL techniques. Simulation and experimental results show better performance of the proposed PLL implementation in unbalanced grid voltage conditions.

Keywords Phase-locked loop · Grid voltage distortion · Power electronics converters · Phase detection · Grid synchronization

1 Introduction

Renewable energy has, in the last decade, emerged as a viable tool for combating climate change, hence its widespread popularity in industry and academia. It is estimated that a third of global electricity generation will be supplied by renewable energy by 2035 (State Grid Corporation of China 2012; Hammons et al. 2000). A comprehensive survey of renewable energy systems, as they pertain to sustainable energy, was undertaken by Nehrir et al. (2011). New trends in power electronic technology are examined by Carrasco et al. (2006) for interfacing these renewable energy sources with the grid to maintain power supply reliability. The wind turbine is one of the many forms of renewable energy generator that has experienced rapid growth. However, interconnecting such a renewable energy system to the power grid requires a robust phase-locked loop (PLL) algorithm capable of dealing with uncertainties. In light of this, alternative synchronization strategies are necessary to fully exploit this energy source and keep pace with ever-growing energy needs.

A PLL is a closed-loop control system that modulates the phase angle of an output signal, so that it closely matches the phase angle of a reference input signal. It essentially minimizes the phase error between both signals, thus matching their oscillatory frequencies. The general technique of doing this is generating a voltage signal corresponding to the phase difference/frequency difference between the output signal and the reference signal. Thus, if the input signal is frequency- or phase-modulated, the output becomes the demodulated output of the PLL (Phipps et al. 2006).

A PLL implementation can be linear (S-domain), digital, or software. The general structure of any PLL consists of a phase detector (PD), a loop filter (LF), and a voltage controlled oscillator (VCO). The phase detector compares the phase angle of the reference signal, usually a

✉ Hicham Chaoui
hchaoui@ntech.edu

Okezie Okoye
oookoye42@students.ntech.edu

Mehdy Khayamy
mkhayamy42@students.ntech.edu

¹ Intelligent Robotics and Energy Systems (IRES), Center for Manufacturing Research, Tennessee Technological University, Cookeville, TN 38501, USA

continuous-time signal, with the phase angle of the VCO output signal. The error signal generated by the phase detection process is filtered by the low pass filter, as we are only interested in the dc varying components. The filtered voltage controls an oscillator to produce a feedback signal with adjusted frequency and phase, such that its phase angle locks into that of the reference signal. In general, it can be stated that in designing a phase-locked loop for most purposes, a trade-off has to be made between achieving high system stability at the expense of system performance or vice-versa. By using

A practical three phase power system is far from ideal and involves various single-phase loads distributed asymmetrically between the feeders, loads constantly being connected and disconnected, and nonlinear loads that distort the voltage of the grid at the point of common coupling (PCC). Additionally, the reference signal obtained from the grid for grid-tie inverters may be distorted by harmonics, which may be local to the power converter itself or generated at some other point on the grid. It is usual to find nonlinear loads operate in an asymmetrical manner; hence, these harmonics may not always be evenly distributed among the three-phase. This uneven distribution of harmonics causes distortions to the grid voltage. Another causative factor of asymmetrically distributed harmonics on the grid is the connection of nonlinear single-phase loads (Wang and Li 2010). Voltage offsets are another form of waveform distortion introduced into three-phase system often due to failure of grid-connected rectifiers (Seymour 2005). Passos et al. (2015) also cites voltage sags as a common disturbance in the power grid at the point of common coupling. Under these conditions, the frequency and phase of the grid is uncertain and difficult to determine, but remains crucial for converter operation and control. The past couple of years have seen various implementations of the PLL with varying adaptations to make them more responsive to changing grid conditions and a comparative study of some of them have been carried out by Gao and Barnes (2012).

The design, analysis and optimization of a PLL, which used the d-q transform as a phase detector (PD), for a three-phase utility interface inverter was carried out in Chung (2000). Particularly, the errors due to distorted voltages were analyzed. The use of coordinate transformation to implement a phase detection scheme is popular in multiphase systems, with the synchronous reference frame (SRF) enjoying the most widespread use (Karimi-Ghartemani 2012; Liccardo et al. 2010; Perez et al. 2008; Rodriguez et al. 2007). However, in Phipps et al. (2006), a novel PLL design was proposed which, unlike traditional three-phase PLL, required no coordinate transformation. An optimized digital implementation of PLLs is presented in Freijedo et al. (2011), where the digital oscillator is used to execute trigonometric functions. Jain et al. (2014) focuses on optimizing the dynamic response of the PLL for fast tracking under disturbance. Park

et al. (2014) deal with grid voltage distortion by constructing an observer to extract the positive sequence voltage and hence enhance the performance of the PLL. Most of the aforementioned techniques are complex in their circuit implementation due to the numerous subsystems deployed, while others are far from robust and make impractical assumptions about grid voltage conditions.

This paper proposes a PLL strategy for renewable energy systems. Phase detection is carried out by counting the time difference between the zero crossings of the reference signal and the output signal. While both signals are out of phase, (cross the zero axis at different times), their amplitudes sampled at any point in time would be unequal. An error signal corresponding to this difference in amplitudes is fed to a PI regulator which forces the error to zero over time. This methodology is as simple as it is robust. It does not rely on the symmetry between the amplitudes of the three-phase voltages, so even under conditions of voltage imbalance, it is still able to track the reference frequency. In a similar way, the presence of distortions in the grid voltage does not impede the performance of the PLL system as it would be seen in the simulation and experimental results. The performance of this approach is compared against two classical schemes. The layout of the rest of the paper is as follows: Two popular PLL schemes are briefly introduced in Sect. 2. For simplicity and ease of reference, the proposed PLL strategy would be referred to as “PLL Counter,” the strategy implemented in Chung (2000) is labeled “Synchronous Frame PLL (SRF-PLL),” and the scheme presented in Phipps et al. (2006) is labeled “PW PLL.” The synthesis, operational principle and modeling of the proposed PLL technique is described in Sect. 3. In Sect. 4, simulation results of the three implementations are reported and discussed. Section 5 presents the experimental results of the three different PLL strategies and the conclusion follows.

2 Conventional PLL Schemes

2.1 Synchronous Reference Frame (SRF) PLL

This PLL design technique employs the Direct Clarke and Givens rotating matrix to determine the frequency and phase difference between the utility grid voltage and the PLL output voltage. This method is common in three-phase power systems and the block diagram is shown above in (Fig. 1).

Assuming balanced conditions in the utility voltage, the three-phase utility voltage can be represented as:

$$v_{abc} = V_m \cdot \begin{pmatrix} \cos \theta \\ \cos \left(\theta - \frac{2\pi}{3} \right) \\ \cos \left(\theta + \frac{2\pi}{3} \right) \end{pmatrix} \quad (1)$$

Figure 11c shows the response of the PLL Counter when the voltage distortions are introduced to the PLL counter at 5.29 s; we observe an initial transient in the phase error which decays to zero in about 0.2 s.

6 Conclusion

In this paper, a PLL strategy is proposed for renewable energy systems. The phase detection is implemented using a counter, which gives the system a high level of robustness to distorted utility conditions. The proposed technique achieves robust tracking of the phase angle under grid voltage uncertainties like phase imbalance, distortions and voltage offset. This is demonstrated by its response to the simulations of various grid conditions which were carried out. The three strategies were also validated experimentally using a real-time digital simulator. The performance of the proposed PLL technique is compared to two widely used implementations in the industry: the Synchronous Reference Frame (SRF) PLL and the PW PLL. Simulation and experimental results show effective tracking in the face of practical grid disturbances. The structure of this implementation of this proposed strategy is simple and can easily be realized. The performance of the presented technique is instrumental in achieving the high performance necessary for fully exploiting renewable energies.

References

- Carrasco, J., Franquelo, L., Bialasiewicz, J., Galvan, E., Guisado, R., Prats, M., et al. (2006). Power-electronic systems for the grid integration of renewable energy sources: A survey. *IEEE Transactions on Industrial Electronics*, 53(2), 1002–1016.
- Chung, S. K. (2000). A phase tracking system for three-phase utility interface inverters. *IEEE Transactions on Power Electronics*, 15(3), 431–438.
- Freijedo, F., Yepes, A., Lopez, O., Fernandez-Comesana, P., & Doval-Gandoy, J. (2011). An optimized implementation of phase locked loops for grid applications. *IEEE Transactions on Instrumentation and Measurement*, 60(9), 3110–3119.
- Gao, S., & Barnes, M. (2012). Phase-locked loop for AC systems: Analyses and comparisons. In *Power Electronics, Machines and Drives (PEMD 2012)*.
- Hammons, T., Boyer, J., Conners, S., Davies, M., Ellis, M., Fraser, M., et al. (2000). Renewable energy alternatives for developed countries. *IEEE Transactions on Energy Conversion*, 15(2), 481–493.
- Jain, S. R., Ravikirithi, P., & Chilakapati, N. (2014). A novel self-consistent model based optimal filter design for the improved dynamic performance of 3-phase PLLs for phase tracking under grid imperfections. *Journal of Control, Automation and Electrical Systems*, 22(5), 620–628.
- Karimi-Ghartemani, M. (2012). A unifying approach to single-phase synchronous reference frame PLLs. *IEEE Transactions on Power Electronics*, 28(10), 4550–4556.
- Liccardo, F., Marino, P., & Raimondo, G. (2010). Robust and fast three-phase PLL tracking system. *IEEE Transactions on Industrial Electronics*, 58(1), 221–231.
- Nehrir, M. H., Wang, C., Strunz, K., Aki, H., Ramakumar, R., Bing, J., et al. (2011). A review of hybrid renewable/alternative energy systems for electric power generation: Configurations, control, and applications. *IEEE Transactions on Sustainable Energy*, 2(4), 392–403.
- Park, Y., Sul, S. K., Kim, W. C., & Young Lee, H. (2014). Phase-locked loop based on an observer for grid synchronization. *IEEE Transactions on Industry Applications*, 50(2), 1256–1265.
- Passos, F. O., Carvalho Filho, J. M., Leborgne, R. C., Silveira, P. M., & Ribeiro, P. F. (2015). An alternative approach to locating voltage sag source side at the point of common coupling based on power-flow information. *Journal of Control, Automation and Electrical Systems*, 26(5), 579–587.
- Perez, M., Espinoza, J., Moran, L., Torres, M., & Araya, E. (2008). A robust phase-locked loop algorithm to synchronize static-power converters with polluted AC systems. *IEEE Transactions on Industrial Electronics*, 55(5), 2185–2192.
- Phipps, W. S., Duke, R. M., & Harrison, M. J. (2006). A novel three-phase software phase-locked loop. In *Power electronics and motion control conference*.
- Rodriguez, P., Pou, J., Bergas, J., Candela, J., Burgos, R., & Boroyevich, D. (2007). Decoupled double synchronous reference frame PLL for power converters control. *IEEE Transactions on Power Electronics*, 22(2), 584–592.
- Seymour, J. (2005). *The seven types of power problems*. Tech. rep., Schneider Electric Data Center Science Center.
- State Grid Corporation of China. (October 2012). *Grid integration of large-capacity renewable energy sources and use of large-capacity electrical energy storage*. Tech. rep., International Electrotechnical Commission (IEC).
- Wang, Y. F., & Li, Y. W. (2010). Grid synchronization PLL based on cascaded delayed signal cancellation. *IEEE Transactions on Power Electronics*, 26(7), 1987–1997.
- Xiu, L., Li, W., Meiners, J., & Padakanti, R. (2004). A novel all-digital PLL with software adaptive filter. *IEEE Journal of Solid-State Circuits*, 39(3), 476–483.